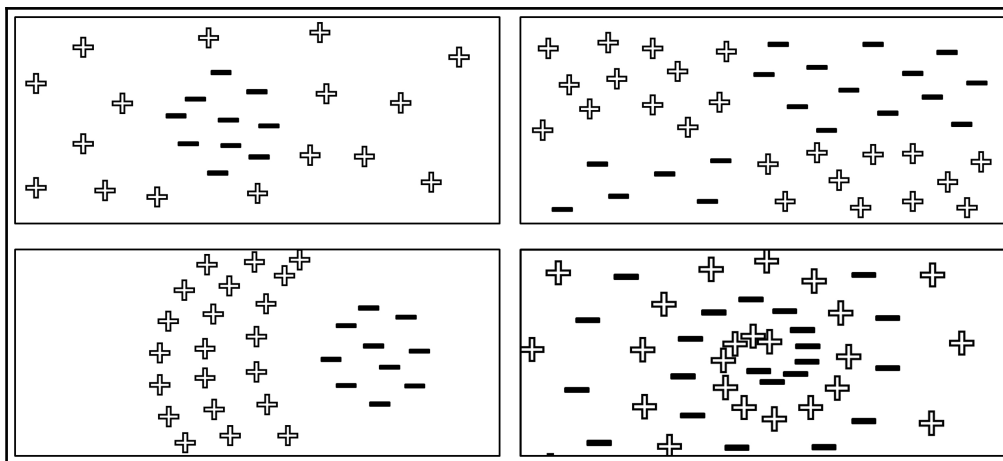




Different datasets that can be classified with SVMs

Think you know the answer? Code it up!



You can use the pregenerated datasets in the Jupyter Notebook `notebooks/06.03-Additional-SVM-Exercises.ipynb` on GitHub. There you will also find the right answers.

Summary

In this chapter, we learned about SVMs in all their forms and flavors. We now know how to draw decision boundaries in 2D and hyperplanes in high-dimensional spaces. We learned about different SVM kernels, and got to know how to implement them in OpenCV.

In addition, we also applied our newly gained knowledge to the practical example of pedestrian detection. For this, we had to learn about the HOG feature descriptor, and how to collect suitable data for the task. We used bootstrapping to improve the performance of our classifier and combined the classifier with OpenCV's multi-scale detection mechanism.

Not only was that a lot to digest in a single chapter, but you have also made it through half of the book. Congrats!

In the next chapter, we will shift gears a bit and revisit a topic from earlier chapters: spam filters. However, this time we want to build a much smarter one than in the first chapter, using the power of Bayesian decision theory.

7

Implementing a Spam Filter with Bayesian Learning

Before we get to grips with advanced topics, such as cluster analysis, deep learning, and ensemble models, let's turn our attention to a much simpler model that we have overlooked so far: the **naive Bayes classifier**.

Naive Bayes classifiers have their roots in **Bayesian inference**, named after famed statistician and philosopher Thomas Bayes (1701-1761). **Bayes' theorem** famously describes the **probability** of an event based on **prior knowledge** of conditions that might lead to the event. We can use Bayes' theorem to build a statistical model that can not only classify data but also provide us with an estimate of how likely it is that our classification is correct. In our case, we can use Bayesian inference to dismiss an email as spam with high confidence, and to determine the probability of a woman having breast cancer, given a positive screening test.

We have now gained enough experience with the mechanics of implementing machine learning methods, and so we should no longer be afraid to try and understand the theory behind them. Don't worry, we won't write a book on it (ahem), but we need some understanding of the theory to appreciate a model's inner workings. After that, I am sure you will find that Bayesian classifiers are easy to implement, computationally efficient, and tend to perform quite well on relatively small datasets.

Along the way, we also want to address the following questions:

- What makes a naive Bayes classifier so darn naive in the first place?
- What is the difference between discriminative and generative models?
- How do I implement Bayesian classifiers in OpenCV and how do I use them to detect fraudulent email messages?

Understanding Bayesian inference

Although Bayesian classifiers are relatively simple to implement, the theory behind them can be quite counter-intuitive at first, especially if you are not too familiar with **probability theory** yet. However, the beauty of Bayesian classifiers is that they understand the underlying data better than all the classifiers we have encountered so far. For example, standard classifiers, such as the k -nearest neighbor algorithm or decision trees, might be able to tell us the target label of a never-before-seen data point. However, these algorithms have no concept of how likely it is for their predictions to be right or wrong. We call them **discriminative models**. Bayesian models, on the other hand, have an understanding of the underlying **probability distribution** that caused the data. We call them **generative models** because they don't just put labels on existing data points- they can also generate new data points with the same statistics.

If this last paragraph was a bit over your head, you might enjoy the following briefer on probability theory. It will be important for the upcoming sections.

Taking a short detour on probability theory

In order to appreciate Bayes' theorem, we need to get a hold of the following technical terms:

- **Random variable:** This is a variable whose value depends on chance. A good example is the act of flipping a coin, which might turn up heads or tails. If a random variable can take on only a limited number of values, we call it **discrete** (such as a coin flip or a dice roll), otherwise we call it a **continuous** random variable (such as the temperature on a given day). Random variables are often typeset as capital letters.
- **Probability:** This is a measure of how likely it is for an event to occur. We denote the probability of an event e happening as $p(e)$, which must be a number between 0 and 1 (or between 0 and 100%). For example, for the random variable X denoting a coin toss, we can describe the probability of getting heads as $p(X = \text{heads}) = 0.5$ (if the coin is fair).
- **Conditional probability:** This is the measure of the probability of an event, given that another event has occurred. We denote the probability of event y happening, given that we know event x has already happened, as $p(y|x)$ (read *p of y given x*). For example, the probability of tomorrow being Tuesday if today is Monday is $p(\text{tomorrow will be Tuesday} | \text{today is Monday}) = 1$. Of course, there's also the chance that we won't see a tomorrow, but let's ignore that for now.

- **Probability distribution:** This is a function that tells you the probability of different events happening in an experiment. For discrete random variables, this function is also called the **probability mass function**. For continuous random variables, this function is also called the **probability density function**. For example, the probability distribution of the coin toss X would take the value 0.5 for $X = \text{heads}$, and 0.5 for $X = \text{tails}$. Across all possible outcomes, the distribution must add up to 1.
- **Joint probability distribution:** This is basically the preceding probability function applied to multiple random variables. For example, when flipping two fair coins A and B , the joint probability distribution would enumerate all possible outcomes— $(A = \text{heads}, B = \text{heads})$, $(A = \text{heads}, B = \text{tails})$, $(A = \text{tails}, B = \text{heads})$, and $(A = \text{tails}, B = \text{tails})$ —and tell you the probability of each of these outcomes.

Now here's the trick: If we think of a dataset as a random variable X , then a machine learning model is basically trying to learn the mapping of X onto a set of possible target labels Y . In other words, we are trying to learn the conditional probability $p(Y|X)$, which is the probability that a random sample drawn from X has a target label Y .

There are two different ways to learn $p(Y|X)$, as briefly mentioned earlier:

- **Discriminative models:** This class of models directly learns $p(Y|X)$ from training data without wasting any time trying to understand the underlying probability distributions (such as $p(X)$, $p(Y)$, or even $p(Y|X)$). This approach applies to pretty much every learning algorithm we have encountered so far: linear regression, k -nearest neighbors, decision trees, and so on.
- **Generative models:** This class of model learns everything about the underlying probability distribution and then infers $p(Y|X)$ from the joint probability distribution $p(X, Y)$. Because these models know $p(X, Y)$, they can not only tell us how likely it is for a data point to have a particular target label but also generate entirely new data points. Bayesian models are one example of this class of models.

Cool, so how do Bayesian models actually work? Let's have a look at a specific example.

Understanding Bayes' theorem

There are quite a few scenarios where it would be really good to know how likely it is for our classifier to make a mistake. For example, in [Chapter 5, Using Decision Trees to Make a Medical Diagnosis](#), we trained a decision tree to diagnose women with breast cancer based on some medical tests. You can imagine that in this case, we would want to avoid a misdiagnosis at all costs; diagnosing a healthy woman with breast cancer (a false positive) would be both soul-crushing and lead to unnecessary, expensive medical procedures, whereas missing a woman's breast cancer (a false negative) might eventually cost the woman her life.

Good to know we have Bayesian models to count on. Let's walk through a specific (and quite famous) example from <http://yudkowsky.net/rational/bayes>:

1% of women at age forty who participate in routine screening have breast cancer. 80% of women with breast cancer will get positive mammographies. 9.6% of women without breast cancer will also get positive mammographies. A woman in this age group had a positive mammography in a routine screening. What is the probability that she actually has breast cancer?

What do you think the answer is?

Well, given that her mammography was positive, you might reason that the probability of her having cancer is quite high (somewhere near 80%). It seems much less likely that the woman would belong to the 9.6% with false positives, so the real probability is probably somewhere between 70% and 80%.

I'm afraid that's not correct.

Here's one way to think about this problem. For the sake of simplicity, let's assume we're looking at some concrete number of patients, say 10,000. Before the mammography screening, the 10,000 women can be divided into two groups:

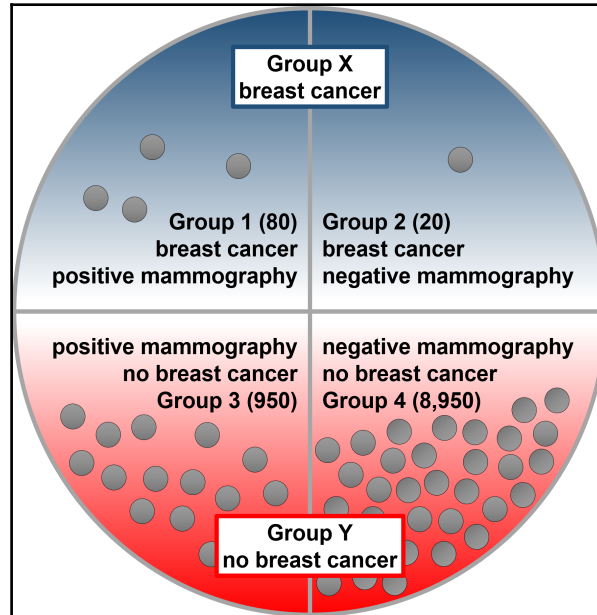
- **Group X:** 100 women *with* breast cancer
- **Group Y:** 9,900 women *without* breast cancer

So far, so good. If we sum up the numbers in the two groups, we get a total of 10,000 patients, confirming that nobody has been lost in the math. After the mammography screening, we can divide the 10,000 women into four groups:

- **Group 1:** 80 women with breast cancer, and a positive mammography
- **Group 2:** 20 women with breast cancer, and a negative mammography

- **Group 3:** 950 women without breast cancer, and a positive mammography
- **Group 4:** 8,950 women without breast cancer, and a negative mammography

As you can see, the sum of all the four groups is still 10,000. The sum of Groups 1 and 2 (the groups with breast cancer) corresponds to Group X, and the sum of Groups 3 and 4 (the groups without breast cancer) corresponds to Group Y. This might become clearer when we draw it out:



Dividing the data points into groups

In this figure, the top half corresponds to Group X, and the bottom half corresponds to Group Y. Analogously, the left half corresponds to all women with positive mammographies, and the right half corresponds to all women with negative mammographies.

Now, it is easier to see that what we are looking for concerns only the left half of the figure. The proportion of cancer patients with positive results within the group of all patients with positive results is the proportion of Group 1 within Groups 1 and 3:

$$80 / (80 + 950) = 80 / 1,030 = 7.8\%$$

In other words, if you administer a mammography to 10,000 patients, then out of the 1,030 with positive mammographies, 80 of those positive mammography patients will have cancer. This is the correct answer, the answer a doctor should give a positive mammography patient if she asks about the chance she has breast cancer; if thirteen patients ask this question, roughly 1 out of those 13 will have cancer.

What we just calculated is called a **conditional probability**: What is our **degree of belief** that a woman has breast cancer **under the condition** (we also say **given**) a positive mammography? As in the last subsection, we denote this with $p(\text{cancer} \mid \text{mammography})$, or $p(C \mid M)$, for short. Using capital letters once again enforces the idea that both health and the mammography can have several outcomes, depending on several underlying (and possibly unknown) causes. Therefore, they are random variables.

Then, we can express $P(C \mid M)$ with the following formula:

$$p(C \mid M) = \frac{p(C, M)}{p(C, M) + p(\sim C, M)}$$

Here, $p(C, M)$ denotes the probability that both C and M are true (meaning the probability that a woman both has cancer and a positive mammography). This is equivalent to the probability of a woman belonging to Group 1 as shown earlier.

The comma (,) means logical *and*, and the tilde ($\sim$) stands for logical *not*. Thus, $p(\sim C, M)$ denotes the probability that C is not true and M is true (meaning the probability that a woman does not have cancer but has a positive mammography). This is equivalent to the probability of a woman belonging to Group 3. So, the denominator basically adds up women in Group 1 ($p(C, M)$) and Group 3 ($p(\sim C, M)$).

But wait! Those two groups together simply denote the probability of a woman having a positive mammography, $p(M)$. Therefore, we can simplify the preceding equation:

$$p(C \mid M) = \frac{p(C, M)}{p(M)}$$

The Bayesian version is to re-interpret what $p(C, M)$ means. We can express $p(C, M)$ as follows:

$$p(C, M) = p(M | C) p(C).$$

Now it gets a bit confusing. Here, $p(C)$ is simply the probability that a woman has cancer (corresponding to the aforementioned Group X). Given that a woman has cancer, what is the probability that her mammography will be positive? From the problem question, we know it is 80%. This is $p(M | C)$, the probability of M , given C .

Replacing $p(C, M)$ in the first equation with this new formula, we get the following equation:

$$p(C | M) = \frac{p(C) p(M | C)}{p(M)}$$

In the Bayesian world, these terms all have their specific names:

- $p(C | M)$ is called the **posterior**, which is always the thing we want to compute. In our example, this corresponds to the degree of belief that a woman has breast cancer, given a positive mammography.
- $p(C)$ is called the **prior** as it corresponds to our initial knowledge about how common breast cancer is. We also call this our initial degree of belief in C .
- $p(M | C)$ is called the **likelihood**.
- $p(M)$ is called the **evidence**.

So, you can rewrite the equation one more time as follows:

$$\text{posterior} = \frac{\text{prior} \times \text{likelihood}}{\text{evidence}}$$

In practice, there is interest only in the numerator of that fraction because the denominator does not depend on C , so the denominator is basically constant and can be neglected.

Understanding the naive Bayes classifier

So far, we have only talked about one piece of evidence. However, in most real-world scenarios, we have to predict an outcome (such as a random variable Y) given multiple pieces of evidence (such as random variables X_1 and X_2). So, instead of calculating $p(Y|X)$, we would often have to calculate $p(Y|X_1, X_2, \dots, X_n)$. Unfortunately, this makes the math very complicated. For two random variables X_1 and X_2 , the joint probability would be computed like this:

$$p(C, X_1, X_2) = p(X_1 | X_2, C) p(X_2 | C) p(C).$$

The ugly part is the term $p(X_1 | X_2, C)$, which says that the conditional probability of X_1 depends on all other variables, including C . This gets even uglier in the case of n variables $X_1, X_2, \dots, X_n$.

Thus, the idea of the naive Bayes classifier is simply to ignore all those mixed terms and instead assume that all the pieces of evidence (X_i) are independent of each other. Since this is rarely the case in the real world, you might call this assumption a bit **naive**. And that's where the naive Bayes classifier got its name from.

The assumption that all the pieces of evidence are independent simplifies the term $p(X_1 | X_2, C)$ to $p(X_1 | C)$. In the case of two pieces of evidence (X_1 and X_2), the last equation simplifies to the following equation:

$$p(C | X_1, X_2) = p(X_1 | C) p(X_2 | C) p(C).$$

This simplifies our lives a lot, since we know how to calculate all the terms in the equation. More generally, for n pieces of evidence, we get the following equation:

$$p(C_1 | X_1, X_2, \dots, X_n) = p(X_1 | C_1) p(X_2 | C_1) \dots p(X_n | C_1) p(C_1).$$

This is the probability that we would predict class C_1 , given $X_1, \dots, X_n$. We could write a second equation that would do the same for another class, C_2 , given $X_1, \dots, X_n$, and for a third and a fourth. This is the secret sauce of the naive Bayes classifier.

The naive Bayes classifier then combines this model with a decision rule. The most common rule is to check all the equations (for all classes $C_1, C_2, \dots, C_m$) and then pick the class that has the highest probability. This is also known as the **maximum a posteriori (MAP)** decision rule.

Implementing your first Bayesian classifier

But enough with the math, let's do some coding!

In the previous chapter, we learned how to generate a number of Gaussian blobs using scikit-learn. Do you remember how that is done?

Creating a toy dataset

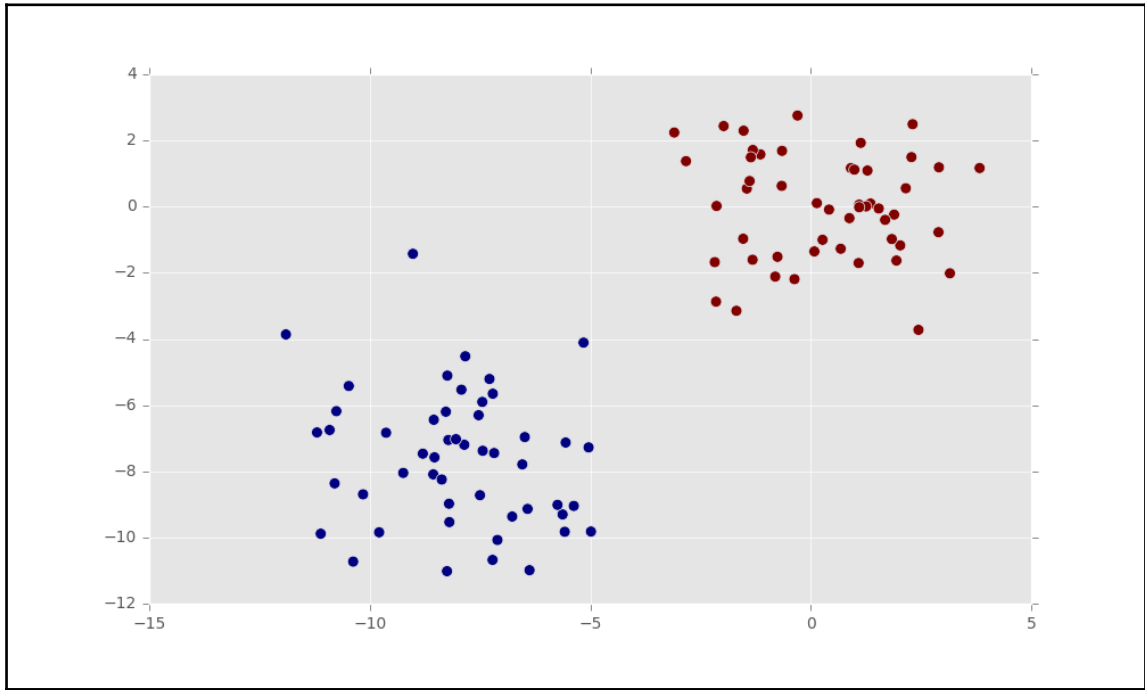
The function I'm referring to resides within scikit-learn's `datasets` module. Let's create 100 data points, each belonging to one of two possible classes, and group them into two Gaussian blobs. To make the experiment reproducible, we specify an integer to pick a seed for the `random_state`. You can again pick whatever number you prefer. Here I went with Thomas Bayes' year of birth (just for kicks):

```
In [1]: from sklearn import datasets
...     X, y = datasets.make_blobs(100, 2, centers=2,
...                               random_state=1701, cluster_std=2)
```

Let's have a look at the dataset we just created using our trusty friend, Matplotlib:

```
In [2]: import matplotlib.pyplot as plt
...     plt.style.use('ggplot')
...     %matplotlib inline
In [3]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50);
```

I'm sure this is getting easier every time. We use `scatter` to create a scatter plot of all x values ($X[:, 0]$) and y values ($X[:, 1]$), which will result in the following output:



Scatter plot of the Thomas Bayes-seeded dataset

In agreement with our specifications, we see two different point clusters. They hardly overlap, so it should be relatively easy to classify them. What do you think—could a linear classifier do the job?

Yes, it could. Recall that a linear classifier would try to draw a straight line through the figure, trying to put all blue dots on one side and all red dots on the other. A diagonal line going from the top-left corner to the bottom-right corner could clearly do the job. So we would expect the classification task to be relatively easy, even for a naive Bayes classifier.

But first, don't forget to split the dataset into training and test sets! Here, I reserve 10% of the data points for testing:

```
In [4]: import numpy as np
...     from sklearn import model_selection as ms
...     X = X.astype(np.float32)
...     X_train, X_test, y_train, y_test = ms.train_test_split(
...         X, y, test_size=0.1
```

```
... )
```

Classifying the data with a normal Bayes classifier

We will then use the same procedure as in earlier chapters to train a **normal Bayes classifier**. Wait, why not a naive Bayes classifier? Well, it turns out OpenCV doesn't really provide a true naive Bayes classifier. Instead, it comes with a Bayesian classifier that doesn't necessarily expect features to be independent, but rather expects the data to be clustered into Gaussian blobs. This is exactly the kind of dataset we created earlier!

We can create a new classifier using the following function:

```
In [5]: import cv2
...     model_norm = cv2.ml.NormalBayesClassifier_create()
```

Then, training is done via the `train` method:

```
In [6]: model_norm.train(X_train, cv2.ml.ROW_SAMPLE, y_train)
Out[6]: True
```

Once the classifier has been trained successfully, it will return `True`. We go through the motions of predicting and scoring the classifier, just like we have done a million times before:

```
In [7]: _, y_pred = model_norm.predict(X_test)
In [8]: from sklearn import metrics
...     metrics.accuracy_score(y_test, y_pred)
Out[8]: 1.0
```

Even better—we can reuse the plotting function from the last chapter to inspect the decision boundary! If you recall, the idea was to create a mesh grid that would encompass all data points and then classify every point on the grid. The mesh grid is created via the NumPy function of the same name:

```
In [9]: def plot_decision_boundary(model, X_test, y_test):
...     # create a mesh to plot in
...     h = 0.02 # step size in mesh
...     x_min, x_max = X_test[:, 0].min() - 1, X_test[:, 0].max() + 1
...     y_min, y_max = X_test[:, 1].min() - 1, X_test[:, 1].max() + 1
...     xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
...                           np.arange(y_min, y_max, h))
```

The `meshgrid` function will return two floating-point matrices `xx` and `yy` that contain the x and y coordinates of every coordinate point on the grid. We can flatten these matrices into column vectors using the `ravel` function, and stack them to form a new matrix `X_hypo`:

```
...     X_hypo = np.column_stack((xx.ravel().astype(np.float32),
...                               yy.ravel().astype(np.float32)))
```

`X_hypo` now contains all x values in `X_hypo[:, 0]` and all y values in `X_hypo[:, 1]`. This is a format that the `predict` function can understand:

```
...     ret = model.predict(X_hypo)
```

However, we want to be able to use models from both OpenCV and scikit-learn. The difference between the two is that OpenCV returns multiple variables (a Boolean flag indicating success/failure and the predicted target labels), whereas scikit-learn returns only the predicted target labels. Hence, we can check whether the `ret` output is a tuple, in which case we know we're dealing with OpenCV. In this case, we store the second element of the tuple (`ret[1]`). Otherwise, we are dealing with scikit-learn and don't need to index into `ret`:

```
...     if isinstance(ret, tuple):
...         zz = ret[1]
...     else:
...         zz = ret
...     zz = zz.reshape(xx.shape)
```

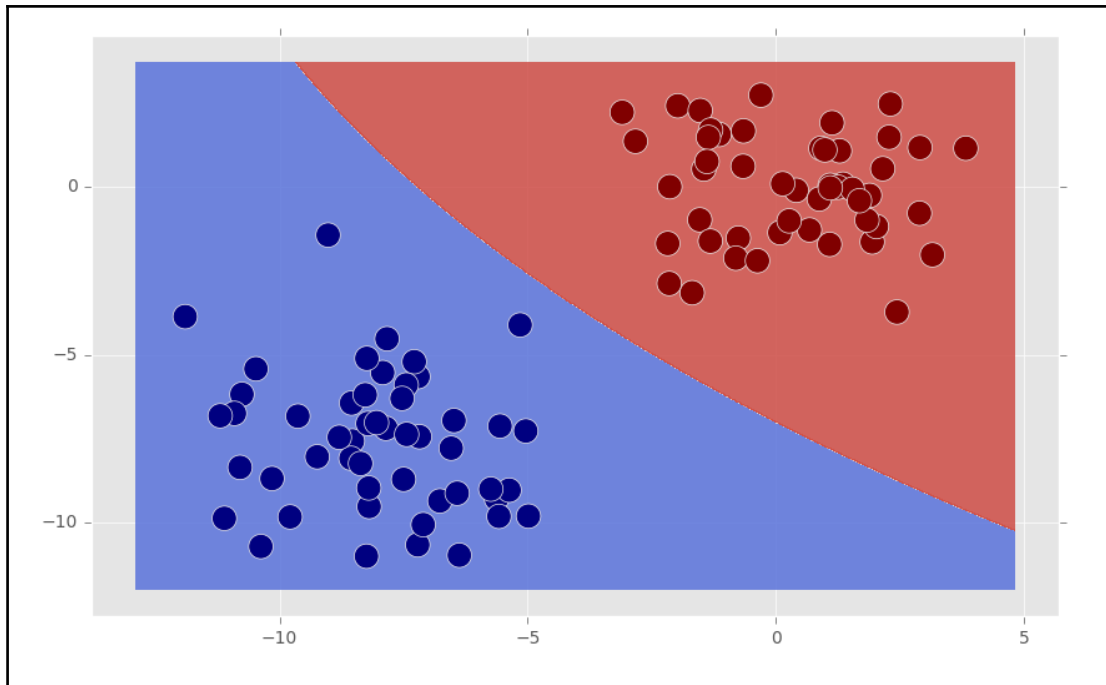
All that's left to do is to create a contour plot where `zz` indicates the color of every point on the grid. On top of that, we plot the data points using our trusty scatter plot:

```
...     plt.contourf(xx, yy, zz, cmap=plt.cm.coolwarm, alpha=0.8)
...     plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=200)
```

We call the function by passing a model (`model_norm`), a feature matrix (X), and a target label vector (y):

```
In [10]: plot_decision_boundary(model_norm, X, y)
```

The output looks like this:



Decision boundary of a normal Bayes classifier

So far, so good. The interesting part is that a Bayesian classifier also returns the probability with which each data point has been classified:

```
In [11]: ret, y_pred, y_proba = model_norm.predictProb(X_test)
```

The function returns a Boolean flag (True for success, False for failure), the predicted target labels (`y_pred`), and the conditional probabilities (`y_proba`). Here, `y_proba` is an $N \times 2$ matrix that indicates, for every one of the N data points, the probability with which it was classified as either class 0 or class 1:

```
In [12]: y_proba.round(2)
Out[12]: array([[ 0.15000001,  0.05      ],
 [ 0.08      ,  0.        ],
 [ 0.        ,  0.27000001],
 [ 0.        ,  0.13      ],
 [ 0.        ,  0.        ],
 [ 0.18000001,  1.88      ],
 [ 0.        ,  0.        ],
 [ 0.        ,  1.88      ],
 [ 0.        ,  0.        ],
 [ 0.        ,  0.        ]], dtype=float32)
```

This means that for the first data point (top row), the probability of it belonging to class 0 (that is, $p(C_0|X)$) is 0.15 (or 15%). Similarly, the probability of belonging to class 1 is $p(C_1|X) = 0.05$.



The reason why some of the rows show values greater than 1 is that OpenCV does not really return probability values. Probability values are always between 0 and 1, and each row in the preceding matrix should add up to 1. Instead, what is being reported is a **likelihood**, which is basically the numerator of the conditional probability equation, $p(C) p(M | C)$. The denominator, $p(M)$, does not need to be computed. All we need to know is that $0.15 > 0.05$ (top row). Hence, the data point most likely belongs to class 0.

Classifying the data with a naive Bayes classifier

We can compare the result to a true naïve Bayes classifier by asking scikit-learn for help:

```
In [13]: from sklearn import naive_bayes
...      model_naive = naive_bayes.GaussianNB()
```

As usual, training the classifier is done via the `fit` method:

```
In [14]: model_naive.fit(X_train, y_train)
Out[14]: GaussianNB(priors=None)
```

Scoring the classifier is built in:

```
In [15]: model_naive.score(X_test, y_test)
Out[15]: 1.0
```

Again a perfect score! However, in contrast to OpenCV, this classifier's `predict_proba` method returns true probability values, because all values are between 0 and 1, and because all rows add up to 1:

```
In [16]: yprob = model_naive.predict_proba(X_test)
...      yprob.round(2)
Out[16]: array([[ 0.,  1.],
                [ 0.,  1.],
                [ 0.,  1.],
                [ 1.,  0.],
                [ 1.,  0.],
                [ 1.,  0.],
                [ 0.,  1.],
                [ 0.,  1.],
                [ 1.,  0.]
```

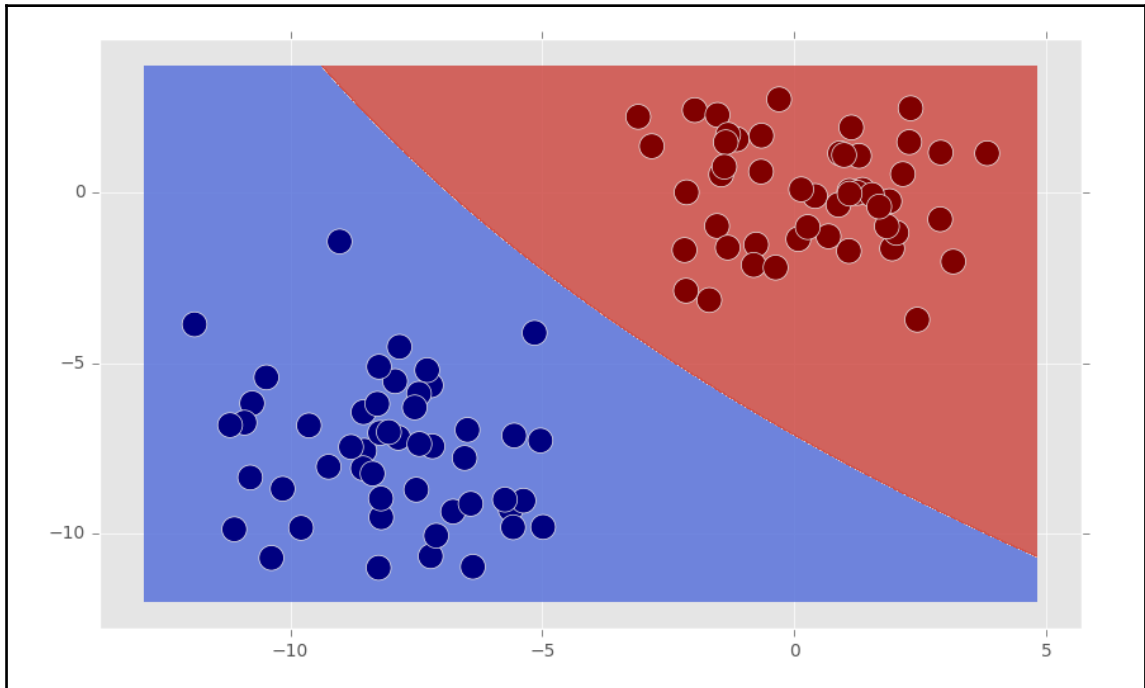
```
[ 1.,  0.]])
```

You might have noticed something else: This classifier has absolutely no doubt about the target label of each and every data point. It's all or nothing.

The decision boundary returned by the naive Bayes classifier looks slightly different, but can be considered identical to the previous command for the purpose of this exercise:

```
In [17]: plot_decision_boundary(model_naive, X, y)
```

The output looks like this:



Decision boundary of a naive Bayes classifier

Visualizing conditional probabilities

Similarly, we can also visualize probabilities. For this, we slightly modify the plot function from the previous example. We start out by creating a mesh grid between $(x_{\min}, x_{\max})$ and $(y_{\min}, y_{\max})$:

```
In [18]: def plot_proba(model, x_test, y_test):
```

```
...         # create a mesh to plot in
...         h = 0.02 # step size in mesh
...         x_min, x_max = X_test[:, 0].min() - 1, X_test[:, 0].max() + 1
...         y_min, y_max = X_test[:, 1].min() - 1, X_test[:, 1].max() + 1
...         xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
...                               np.arange(y_min, y_max, h))
```

Then we flatten `xx` and `yy` and add them column-wise to the feature matrix `X_hypo`:

```
...         X_hypo = np.column_stack((xx.ravel().astype(np.float32),
...                                   yy.ravel().astype(np.float32)))
```

If we want to make this function work with both OpenCV and scikit-learn, we need to implement a switch for `predictProb` (in the case of OpenCV) and `predict_proba` (in the case of scikit-learn). For this, we check whether `model` has a method called `predictProb`. If the method exists, we can call it, otherwise we assume we're dealing with a model from scikit-learn:

```
...         if hasattr(model, 'predictProb'):
...             _, _, y_proba = model.predictProb(X_hypo)
...         else:
...             y_proba = model.predict_proba(X_hypo)
```

Like in In [16] that we saw earlier, `y_proba` will be a 2D matrix containing, for each data point, the probability of the data belonging to class 0 (in `y_proba[:, 0]`) and to class 1 (in `y_proba[:, 1]`). An easy way to convert these two values into a color that the contour function can understand is to simply take the difference of the two probability values:

```
...         zz = y_proba[:, 1] - y_proba[:, 0]
...         zz = zz.reshape(xx.shape)
```

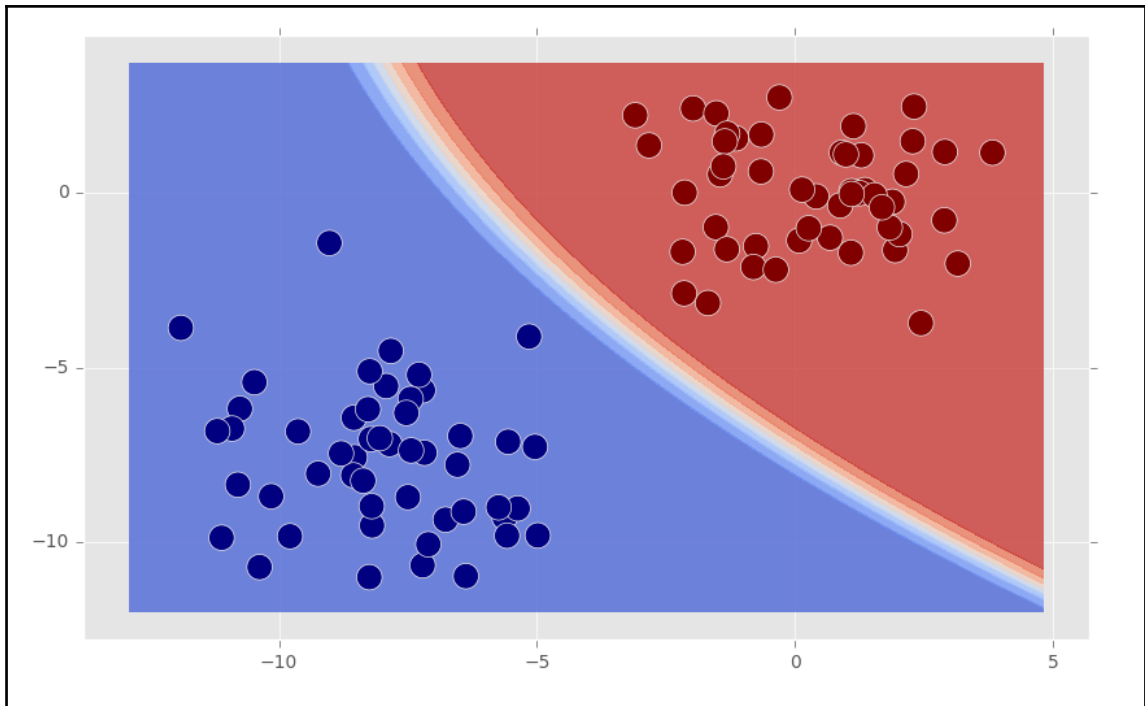
The last step is to plot `X_test` as a scatter plot on top of the colored meshgrid:

```
...         plt.contourf(xx, yy, zz, cmap=plt.cm.coolwarm, alpha=0.8)
...         plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=200)
```

Now we are ready to call the function:

```
In [19]: plot_proba(model_naive, X, y)
```

The result looks like this:



Conditional probabilities of a naive Bayes classifier

Classifying emails using the naive Bayes classifier

The final task of this chapter will be to apply our newly gained skills to a real spam filter!

Naive Bayes classifiers are actually a very popular model for email filtering. Their naivety lends itself nicely to the analysis of text data, where each feature is a word (or a **bag of words**), and it would not be feasible to model the dependence of every word on every other word.

There are a bunch of good email datasets out there, such as the following:

- The Ling-Spam corpus: <http://csmining.org/index.php/ling-spam-datasets.html>
- The Hewlett-Packard spam database: <https://archive.ics.uci.edu/ml/machine-learning-databases/spambase>

- The Enron-Spam dataset: <http://www.aueb.gr/users/ion/data/enron-spam>
- The Apache SpamAssassin public corpus: <http://csmining.org/index.php/spam-assassin-datasets.html>

In this section, we will be using the Enron-Spam dataset, which can be downloaded for free from the given website. However, if you followed the installation instructions at the beginning of this book and have downloaded the latest code from GitHub, you are already good to go!

Loading the dataset

If you downloaded the latest code from GitHub, you will find a number of `.zip` files in the `notebooks/data/chapter7` directory. These files contain raw email data (with fields for To:, Cc:, and text body) that are either classified as spam (with the `SPAM = 1` class label) or not (also known as ham, the `HAM = 0` class label).

We build a variable called `sources`, which contains all the raw data files:

```
In [1]: HAM = 0
...     SPAM = 1
...     datadir = 'data/chapter7'
...     sources = [
...         ('beck-s.tar.gz', HAM),
...         ('farmer-d.tar.gz', HAM),
...         ('kaminski-v.tar.gz', HAM),
...         ('kitchen-l.tar.gz', HAM),
...         ('lokay-m.tar.gz', HAM),
...         ('williams-w3.tar.gz', HAM),
...         ('BG.tar.gz', SPAM),
...         ('GP.tar.gz', SPAM),
...         ('SH.tar.gz', SPAM)
...     ]
```

The first step is to extract these files into subdirectories. For this, we can use the `extract_tar` function we wrote in the previous chapter:

```
In [2]: def extract_tar(datafile, extractdir):
...     try:
...         import tarfile
...     except ImportError:
...         raise ImportError("You do not have tarfile installed. "
...                             "Try unzipping the file outside of "
...                             "Python.")
...     tar = tarfile.open(datafile)
```

```
...         tar.extractall(path=extractdir)
...         tar.close()
...         print("%s successfully extracted to %s" % (datafile,
...                                                     extractdir))
...
```

In order to apply the function to all data files in the sources, we need to run a loop. The `extract_tar` function expects a path to the `.tar.gz` file—which we build from `datadir` and an entry in `sources`—and a directory to extract the files to (`datadir`). This will extract all emails in, for example, `data/chapter7/beck-s.tar.gz` to the `data/chapter7/beck-s/` subdirectory:

```
In [3]: for source, _ in sources:
...     datafile = '%s/%s' % (datadir, source)
...     extract_tar(datafile, datadir)
Out[3]: data/chapter7/beck-s.tar.gz successfully extracted to data/chapter7
data/chapter7/farmer-d.tar.gz successfully extracted to
data/chapter7
data/chapter7/kaminski-v.tar.gz successfully extracted to
data/chapter7
data/chapter7/kitchen-l.tar.gz successfully extracted to
data/chapter7
data/chapter7/lokey-m.tar.gz successfully extracted to
data/chapter7
data/chapter7/williams-w3.tar.gz successfully extracted to
data/chapter7
data/chapter7/BG.tar.gz successfully extracted to data/chapter7
data/chapter7/GP.tar.gz successfully extracted to data/chapter7
data/chapter7/SH.tar.gz successfully extracted to data/chapter7
```

Now here's the tricky bit. Every one of these subdirectories contains a number of other directories, wherein the text files reside. So we need to write two functions:

- `read_single_file(filename)`: This is a function that extracts the relevant content from a single file called `filename`
- `read_files(path)`: This is a function that extracts the relevant content from all files in a particular directory called `path`

In order to extract the relevant content from a single file, we need to be aware of how each file is structured. The only thing we know is that the header section of the email (From:, To:, Cc:) and the main body of text are separated by a newline character `'\n'`. So what we can do is iterate over every line in the text file and keep only those lines that belong to the main text body, which will be stored in the variable `lines`. We also want to keep a Boolean flag `past_header` around, which is initially set to `False` but will be flipped to `True` once we are past the header section. We start out by initializing those two variables:

```
In [4]: import os
...     def read_single_file(filename):
...         past_header, lines = False, []
```

Then we check whether a file with the name `filename` exists. If it does, we start looping over it line by line:

```
...         if os.path.isfile(filename):
...             f = open(filename, encoding="latin-1")
...             for line in f:
```

You may have noticed the `encoding="latin-1"` part. Since some of the emails are not in Unicode, this is an attempt to decode the files correctly.

We do not want to keep the header information, so we keep looping until we encounter the `'\n'` character, at which point we flip `past_header` from `False` to `True`. At this point, the first condition of the following `if-else` clause is met, and we append all remaining lines in the text file to the `lines` variable:

```
...                 if past_header:
...                     lines.append(line)
...                 elif line == '\n':
...                     past_header = True
...             f.close()
```

In the end, we concatenate all lines into a single string, separated by the newline character, and return both the full path to the file and the actual content of the file:

```
...         content = '\n'.join(lines)
...         return filename, content
```

The job of the second function will be to loop over all files in a folder and call `read_single_file` on them:

```
In [5]: def read_files(path):
...     for root, dirnames, filenames in os.walk(path):
...         for filename in filenames:
...             filepath = os.path.join(root, filename)
...             yield read_single_file(filepath)
```

Here, `yield` is a keyword that is similar to `return`. The difference is that `yield` returns a generator instead of the actual values, which is desirable if you expect to have a large number of items to iterate over.

Building a data matrix using Pandas

Now it's time to introduce another essential data science tool that comes preinstalled with Python Anaconda: **Pandas**. Pandas is built on NumPy and provides a number of useful tools and methods to deal with data structures in Python. Just as we generally import NumPy under the alias `np`, it is common to import Pandas under the `pd` alias:

```
In [6]: import pandas as pd
```

Pandas provides a useful data structure called `DataFrame`, which can be understood as a generalization of a 2D NumPy array, as shown here:

```
In [7]: pd.DataFrame({
...     'model': [
...         'Normal Bayes',
...         'Multinomial Bayes',
...         'Bernoulli Bayes'
...     ],
...     'class': [
...         'cv2.ml.NormalBayesClassifier_create()',
...         'sklearn.naive_bayes.MultinomialNB()',
...         'sklearn.naive_bayes.BernoulliNB()'
...     ]
... })
```

The output of the cell will look like this:

Out[7]:

	class	model
0	cv2.ml.NormalBayesClassifier_create()	Normal Bayes
1	sklearn.naive_bayes.MultinomialNB()	Multinomial Bayes
2	sklearn.naive_bayes.BernoulliNB()	Bernoulli Bayes

Example of a Pandas DataFrame

We can combine the preceding functions to build a Pandas DataFrame from the extracted data:

```
In [8]: def build_data_frame(extractdir, classification):
...     rows = []
...     index = []
...     for file_name, text in read_files(extractdir):
```

```
...         rows.append({'text': text, 'class': classification})
...         index.append(file_name)
...
...         data_frame = pd.DataFrame(rows, index=index)
...         return data_frame
```

We then call it with the following command:

```
In [9]: data = pd.DataFrame({'text': [], 'class': []})
...     for source, classification in sources:
...         extractdir = '%s/%s' % (datadir, source[:-7])
...         data = data.append(build_data_frame(extractdir,
...                                             classification))
... 
```

Preprocessing the data

Scikit-learn offers a number of options when it comes to encoding text features, which we discussed in [Chapter 4, Representing Data and Engineering Features](#). One of the simplest methods of encoding text data, we recall, is by **word count**: For each phrase, you count the number of occurrences of each word within it. In scikit-learn, this is easily done using `CountVectorizer`:

```
In [10]: from sklearn import feature_extraction
...       counts = feature_extraction.text.CountVectorizer()
...       X = counts.fit_transform(data['text'].values)
...       X.shape
Out[10]: (52076, 643270)
```

The result is a giant matrix, which tells us that we harvested a total of 52,076 emails that collectively contain 643,270 different words. However, scikit-learn is smart and saved the data in a sparse matrix:

```
In [11]: X
Out[11]: <52076x643270 sparse matrix of type '<class 'numpy.int64'>'
         with 8607632 stored elements in Compressed Sparse Row
         format>
```

In order to build the vector of target labels (y), we need to access data in the Pandas `DataFrame`. This can be done by treating the `DataFrame` like a dictionary, where the `values` attribute will give us access to the underlying NumPy array:

```
In [12]: y = data['class'].values
```

Training a normal Bayes classifier

From here on out, things are (almost) like they always were. We can use scikit-learn to split the data into training and test sets. (Let's reserve 20% of all data points for testing):

```
In [13]: from sklearn import model_selection as ms
...      X_train, X_test, y_train, y_test = ms.train_test_split(
...          X, y, test_size=0.2, random_state=42
...      )
```

We can instantiate a new normal Bayes classifier with OpenCV:

```
In [14]: import cv2
...      model_norm = cv2.ml.NormalBayesClassifier_create()
```

However, OpenCV does not know about sparse matrices (at least its Python interface does not). If we were to pass `X_train` and `y_train` to the `train` function like we did earlier, OpenCV would complain that the data matrix is not a NumPy array. But converting the sparse matrix into a regular NumPy array will likely make you run out of memory. Thus, a possible workaround is to train the OpenCV classifier only on a subset of data points (say 1,000) and features (say 300):

```
In [15]: import numpy as np
...      X_train_small = X_train[:1000, :300].toarray().astype(np.float32)
...      y_train_small = y_train[:1000].astype(np.float32)
```

Then it becomes possible to train the OpenCV classifier (although this might take a while):

```
In [16]: model_norm.train(X_train_small, cv2.ml.ROW_SAMPLE, y_train_small)
```

Training on the full dataset

However, if you want to classify the full dataset, we need a more sophisticated approach. We turn to scikit-learn's naive Bayes classifier, as it understands how to handle sparse matrices. In fact, if you didn't pay attention and treated `X_train` like every NumPy array before, you might not even notice that anything is different:

```
In [17]: from sklearn import naive_bayes
...      model_naive = naive_bayes.MultinomialNB()
...      model_naive.fit(X_train, y_train)
Out[17]: MultinomialNB(alpha=1.0, class_prior=None, fit_prior=True)
```

Here we used `MultinomialNB` from the `naive_bayes` module, which is the version of naive Bayes classifier that is best suited to handle categorical data, such as word counts.

The classifier is trained almost instantly and returns the scores for both the training and the test set:

```
In [18]: model_naive.score(X_train, y_train)
Out[18]: 0.95086413826212191
In [19]: model_naive.score(X_test, y_test)
Out[19]: 0.94422043010752688
```

And there we have it: 94.4% accuracy on the test set! Pretty good for not doing much other than using the default values, isn't it?

However, what if we were super critical of our own work and wanted to improve the result even further? There are a couple of things we could do.

Using n-grams to improve the result

One thing to do is to use **n-gram counts** instead of plain word counts. So far, we have relied on what is known as a **bag of words**: We simply threw every word of an email into a bag and counted the number of its occurrences. However, in real emails, the **order** in which **words appear** can carry a great deal of information!

This is exactly what *n*-gram counts are trying to convey. You can think of an *n*-gram as a phrase that is *n* words long. For example, the phrase *Statistics has its moments* contains the following 1-grams: *Statistics*, *has*, *its*, and *moments*. It also has the following 2-grams: *Statistics has*, *has its*, and *its moments*. It also has two 3-grams (*Statistics has its* and *has its moments*), and only a single 4-gram.

We can tell `CountVectorizer` to include any order of *n*-grams into the feature matrix by specifying a range for *n*:

```
In [20]: counts = feature_extraction.text.CountVectorizer(
...       ngram_range=(1, 2)
...       )
...       X = counts.fit_transform(data['text'].values)
```

We then repeat the entire procedure of splitting the data and training the classifier:

```
In [21]: X_train, X_test, y_train, y_test = ms.train_test_split(
...       X, y, test_size=0.2, random_state=42
...       )
In [22]: model_naive = naive_bayes.MultinomialNB()
...       model_naive.fit(X_train, y_train)
Out[22]: MultinomialNB(alpha=1.0, class_prior=None, fit_prior=True)
```

You might have noticed that the training is taking much longer this time. To our delight, we find that the performance has significantly increased:

```
In [23]: model_naive.score(X_test, y_test)
Out[23]: 0.97062211981566815
```

However, n -gram counts are not perfect. They have the disadvantage of unfairly weighting longer documents (because there are more possible combinations of forming n -grams). To avoid this problem, we can use **relative frequencies** instead of a simple **number of occurrences**. We have already encountered one way to do so, and it had a horribly complicated name. Do you remember what it was called?

Using tf-idf to improve the result

It was called the **term-inverse document frequency** (**tf-idf**), and we encountered it in Chapter 4, *Representing Data and Engineering Features*. If you recall, what tf-idf does is basically weigh the word count by a measure of how often they appear in the entire dataset. A useful side effect of this method is the idf part—the inverse frequency of words. This makes sure that frequent words, such as *and*, *the*, and *but*, carry only a small weight in the classification.

We apply tf-idf to the feature matrix by calling `fit_transform` on our existing feature matrix `X`:

```
In [24]: tfidf = feature_extraction.text.TfidfTransformer()
In [25]: X_new = tfidf.fit_transform(X)
```

Don't forget to split the data:

```
In [26]: X_train, X_test, y_train, y_test = ms.train_test_split(X_new, y,
...                     test_size=0.2, random_state=42)
```

Then, when we train and score the classifier again, we suddenly find a remarkable score of 99% accuracy!

```
In [27]: model_naive = naive_bayes.MultinomialNB()
...      model_naive.fit(X_train, y_train)
...      model_naive.score(X_test, y_test)
Out[27]: 0.99087941628264209
```

To convince us of the classifier's awesomeness, we can inspect the **confusion matrix**. This is a matrix that shows, for every class, how many data samples were misclassified as belonging to a different class. The diagonal elements in the matrix tell us how many samples of the class i were correctly classified as belonging to the class i . The off-diagonal elements represent misclassifications:

```
In [28]: metrics.confusion_matrix(y_test, model_naive.predict(X_test))
Out[28]: array([[3746,   84],
               [  11, 6575]])
```

This tells us we got 3,746 class 0 classifications correct, and 6,575 class 1 classifications correct. We confused 84 samples of class 0 as belonging to class 1 and 11 samples of class 1 as belonging to class 0. If you ask me, that's about as good as it gets.

Summary

In this chapter, we took our first stab at probability theory, learning about random variables and conditional probabilities, which allowed us to get a glimpse of Bayes' theorem—the underpinning of a naive Bayes classifier. We talked about the difference between discrete and continuous random variables, between likelihoods and probabilities, between priors and evidence, and between normal and naive Bayes classifiers.

Finally, our theoretical knowledge would be of no use if we didn't apply it to a practical example. We obtained a dataset of raw email messages, parsed it, and trained Bayesian classifiers on it to classify emails as either ham or spam using a variety of feature extraction approaches.

In the next chapter, we will switch gears and, for once, discuss what to do if we have to deal with unlabeled data.

8

Discovering Hidden Structures with Unsupervised Learning

So far, we have focused our attention exclusively on supervised learning problems, where every data point in the dataset had a known label or target value. However, what do we do when there is no known output, or no teacher to supervise the learning algorithm?

This is what **unsupervised learning** is all about. In unsupervised learning, the learning is shown only in the input data and is asked to extract knowledge from this data without further instruction. We have already talked about one of the many forms that unsupervised learning comes in—**dimensionality reduction**. Another popular domain is **cluster analysis**, which aims to partition data into distinct groups of similar items.

In this chapter, we want to understand how different clustering algorithms can be used to extract hidden structures in simple, unlabeled datasets. These hidden structures have many benefits, whether they are used in feature extraction, image processing, or even as a preprocessing step for supervised learning tasks. As a concrete example, we will learn how to apply clustering to images in order to reduce their color spaces to 16 bits.

More specifically, we want to answer the following questions:

- What are ***k*-means clustering** and **expectation-maximization**, and how do I implement these things in OpenCV?
- How can I arrange clustering algorithms in hierarchical trees, and what are the benefits that come from that?
- How can I use unsupervised learning for preprocessing, image processing, and classification?

Let's get started!

Understanding unsupervised learning

Unsupervised learning might come in many shapes and forms, but the goal is always to convert original data into a richer, more meaningful representation, whether that means making it easier for humans to understand or easier for machine learning algorithms to parse.

Some common applications of unsupervised learning include the following:

- **Dimensionality reduction:** This takes a high-dimensional representation of data consisting of many features and tries to compress the data so that its main characteristics can be explained with a small number of highly informative features. For example, when applied to housing prices in the neighborhoods of Boston, dimensionality reduction might be able to tell us that the indicators we should pay most attention to are the property tax and the neighborhood's crime rate.
- **Factor analysis:** This tries to find the hidden causes or unobserved components that gave rise to the observed data. For example, when applied to all the episodes of the 1970s TV show *Scooby-Doo, Where Are You!*, factor analysis might be able to tell us that (spoiler alert!) every ghost or monster on the show is essentially some disgruntled count playing an elaborate hoax on the town.
- **Cluster analysis:** This tries to partition the data into distinct groups of similar items. This is the type of unsupervised learning we will focus on in this chapter. For example, when applied to all the movies on Netflix, cluster analysis might be able to automatically group them into genres.

To make things more complicated, these analyses have to be performed on unlabeled data, where we do not know beforehand what the right answer should be. Consequently, a major challenge in unsupervised learning is to determine whether an algorithm did well or learned anything useful. Often, the only way to evaluate the result of an unsupervised learning algorithm is to inspect it manually and determine by hand whether the result makes sense.

That being said, unsupervised learning can be immensely helpful, for example, as a preprocessing or feature extraction step. You can think of unsupervised learning as a **data transformation**—a way to transform data from its original representation into a more informative form. Learning a new representation might give us deeper insights about our data, and sometimes it might even improve the accuracy of supervised learning algorithms.

Understanding k-means clustering

The most essential clustering algorithm that OpenCV provides is *k*-means clustering, which searches for a predetermined number of *k* clusters (or groups) within an unlabeled multidimensional dataset.

It does so by using two simple assumptions about what an optimal clustering should look like:

- The center of each cluster is simply the arithmetic mean of all the points belonging to the cluster
- Each point in the cluster is closer to its own center than to other cluster centers

It's the easiest to understand the algorithm by looking at a concrete example.

Implementing our first k-means example

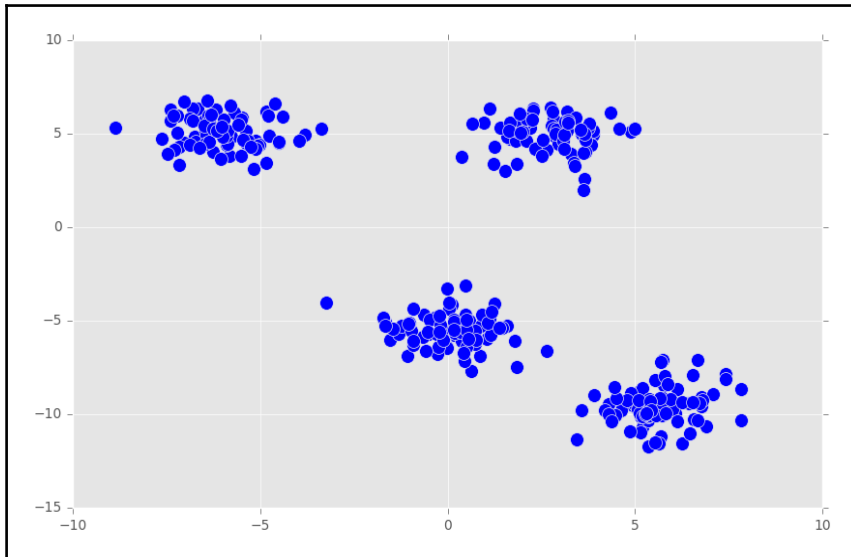
First, let's generate a 2D dataset containing four distinct blobs. To emphasize that this is an unsupervised approach, we will leave the labels out of the visualization. We will continue using `matplotlib` for all our visualization purposes:

```
In [1]: import matplotlib.pyplot as plt
...      %matplotlib inline
...      plt.style.use('ggplot')
```

Following the same recipe from earlier chapters, we will create a total of 300 blobs (`n_samples=300`) belonging to four distinct clusters (`centers=4`):

```
In [2]: from sklearn.datasets.samples_generator import make_blobs
...      X, y_true = make_blobs(n_samples=300, centers=4,
...                              cluster_std=1.0, random_state=10)
...      plt.scatter(X[:, 0], X[:, 1], s=100);
```

This will generate the following figure:



Example dataset of 300 unlabeled points organized into four distinct clusters

Even without assigning target labels to the data, it is straightforward to pick out the four clusters by eye. The k -means algorithm can do this, too, without having any information about target labels or underlying data distributions.

Although k -means is, of course, a statistical model, in OpenCV, it does not come via the `ml` module and the common `train` and `predict` API calls. Instead, it is directly available as `cv2.kmeans`. In order to use the model, we have to specify some arguments, such as the termination criteria and some initialization flags. Here, we will tell the algorithm to terminate whenever the error is smaller than 1.0 (`cv2.TERM_CRITERIA_EPS`) or when ten iterations have been executed (`cv2.TERM_CRITERIA_MAX_ITER`):

```
In [3]: import cv2
...     criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
...                 10, 1.0)
...     flags = cv2.KMEANS_RANDOM_CENTERS
```

Then we can pass the preceding data matrix (`x`) to `cv2.kmeans`. We also specify the number of clusters (4) and the number of attempts the algorithm should make with different random initial guesses (10), as shown in the following snippet:

```
In [4]: import numpy as np
...     compactness, labels, centers = cv2.kmeans(X.astype(np.float32),
...                                              4, None, criteria,
...                                              10, flags)
```

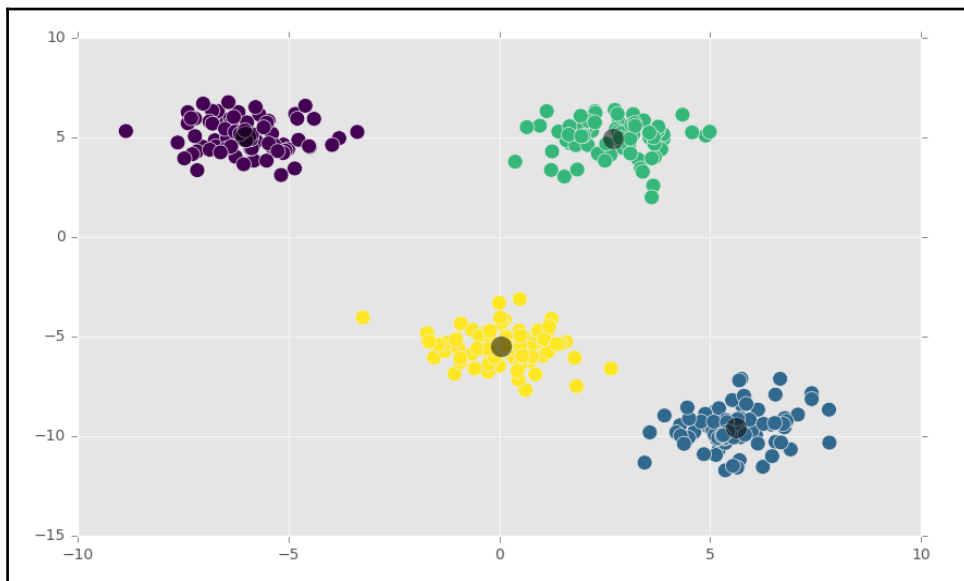
Three different variables are returned. The first one, `compactness`, returns the sum of squared distances from each point to their corresponding cluster centers. A high compactness score indicates that all points are close to their cluster centers, whereas a low compactness score indicates that the different clusters might not be well separated:

```
In [5]: compactness
Out[5]: 527.01581170992
```

Of course, this number strongly depends on the actual values in `x`. If the distances between points were large to begin with, we cannot expect an arbitrarily small compactness score. Hence, it is more informative to plot the data points, colored to their assigned cluster labels:

```
In [6]: plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
...     plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200,
...                 alpha=0.5);
```

This produces a scatter plot of all the data points colored according to which-ever cluster they belong to, with the corresponding cluster centers indicated with a blob of darker shade in the center of every cluster:



Result of *k*-means clustering for *k* = 4

The good news here is that the k -means algorithm (at least in this simple case) assigns the points to clusters very similarly to how we might have, had we done the job by eye. But how did the algorithm find these different clusters so quickly? After all, the number of possible combinations of cluster assignments is exponential to the number of data points! By hand, trying all possible combinations would have certainly taken forever.

Fortunately, an exhaustive search is not necessary. Instead, the typical approach that k -means takes is to use an iterative algorithm, also known as **expectation-maximization**.

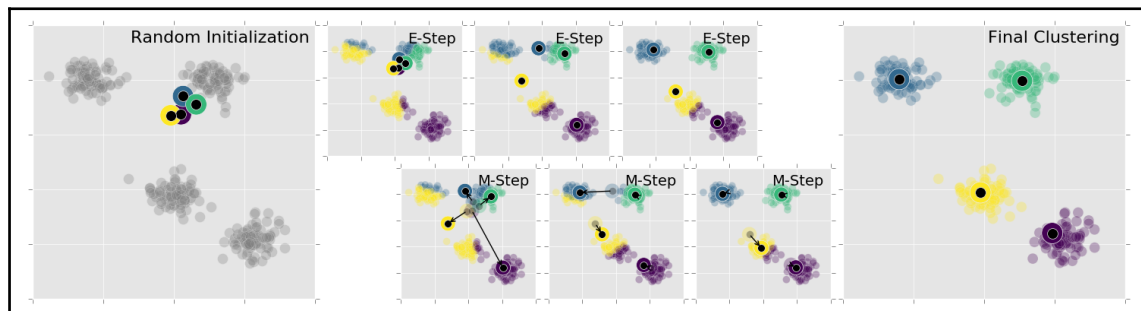
Understanding expectation-maximization

K -means clustering is but one concrete application of a more general algorithm known as expectation-maximization. In short, the algorithm works as follows:

1. Start with some random cluster centers.
2. Repeat until convergence:
 - **Expectation step**: Assign all data points to their nearest cluster center.
 - **Maximization step**: Update the cluster centers by taking the mean of all the points in the cluster.

Here, the expectation step is so named because it involves updating our expectation of which cluster each point in the dataset belongs to. The maximization step is so named because it involves maximizing a fitness function that defines the location of the cluster centers. In the case of k -means, maximization is performed by taking the arithmetic mean of all the data points in a cluster.

This should become clearer with the following figure:



Expectation-maximization step by step

In the preceding image, the algorithm works from left to right. Initially, all data points are grayed out (meaning, we don't know yet which cluster they belong to), and the cluster centers are chosen at random. In each expectation step, the data points are colored according to whichever cluster center they are closest to. In each maximization step, the cluster centers are updated by taking the arithmetic mean of all points belonging to the cluster; the resulting displacement of the cluster centers is indicated with arrows. These two steps are repeated until the **algorithm converges**, meaning until the maximization step does not further improve the clustering outcome.



Another popular application of expectation-maximization is the so-called **Gaussian mixture models (GMMs)**, where the clusters are not spherical but multivariate Gaussians. You can find more information about them at: <http://scikit-learn.org/stable/modules/mixture.html>.

Implementing our own expectation-maximization solution

The expectation-maximization algorithm is simple enough for us to code it up ourselves. To do so, we will define a function `find_clusters(X, n_clusters, rseed=5)` that takes as input a data matrix (`X`), the number of clusters we want to discover (`n_clusters`), and a random seed (optional, `rseed`). As will become clear in a second, scikit-learn's `pairwise_distances_argmin` function will come in handy:

```
In [7]: from sklearn.metrics import pairwise_distances_argmin
...     def find_clusters(X, n_clusters, rseed=5):
```

We can implement expectation-maximization for *k*-means in five essential steps:

1. **Initialization:** Randomly choose a number `n_clusters` of cluster centers. We don't just pick any random number but instead pick actual data points to be the cluster centers. We do this by permuting `X` along its first axis and picking the first `n_clusters` points in this random permutation:

```
...         rng = np.random.RandomState(rseed)
...         i = rng.permutation(X.shape[0])[:n_clusters]
...         centers = X[i]
```

2. **While looping forever:** Assign labels based on the closest cluster centers. Here, scikit-learn's `pairwise_distance_argmin` function does exactly what we want: It computes, for each data point in `X`, the index of the closest cluster center in `centers`:

```
...         while True:
...             labels = pairwise_distances_argmin(X, centers)
```

3. **Find the new cluster centers:** In this step, we have to take the arithmetic mean of all data points in `X` that belong to a specific cluster (`X[labels == i]`):

```
...         new_centers = np.array([X[labels ==
...                                 i].mean(axis=0)
```

4. **Check for convergence, and break the while loop if necessary:** This is the last step to make sure that we stop the execution of the algorithm once the job is done. We determine whether the job is done by checking whether all the new cluster centers are equal to the old cluster centers. If this is true, we exit the loop, otherwise we keep looping:

```
...         for i in range(n_clusters)]
...         if np.all(centers == new_centers):
...             break
...         centers = new_centers
```

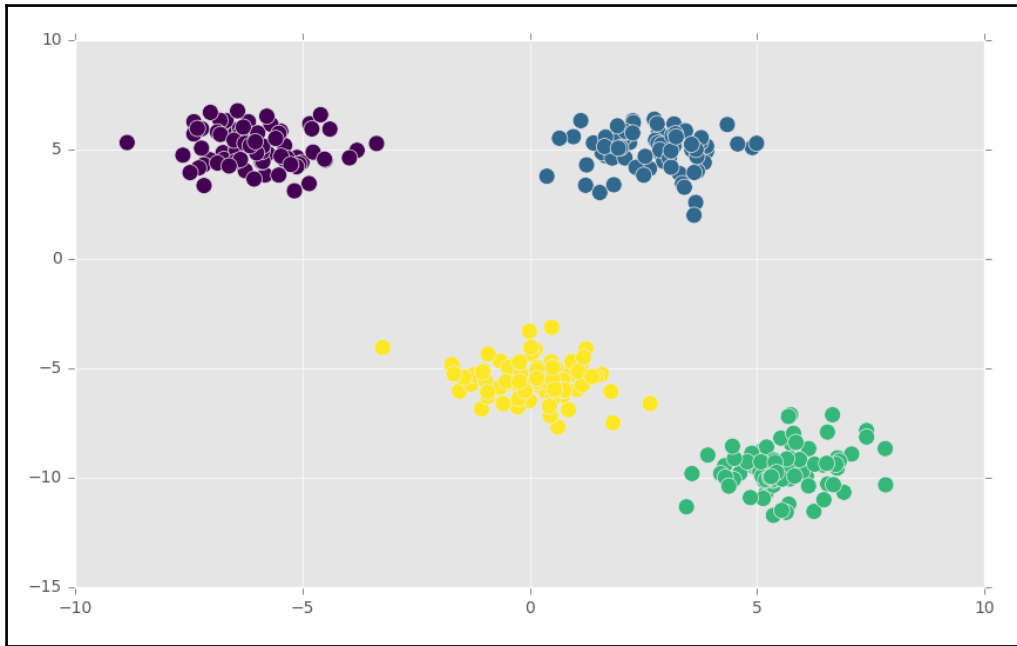
5. Exit the function, and return the result:

```
...         return centers, labels
```

We can apply our function to the preceding data matrix `X` we created. Since we know what the data looks like, we know that we are looking for four clusters:

```
In [8]: centers, labels = find_clusters(X, 4)
...     plt.scatter(X[:, 0], X[:, 1], c=labels, s=100, cmap='viridis');
```

This will generate the following plot:



Outcome of our home-made k -means using expectation-maximization

As we can see, our home-made algorithm got the job done! Granted, this particular clustering example was fairly easy, and most real-life implementations of k -means clustering will do a bit more under the hood. But for now, we are happy.

Knowing the limitations of expectation-maximization

For its simplicity, expectation-maximization performs incredibly well in a range of scenarios. That being said, there are a number of potential limitations that we need to be aware of:

- Expectation-maximization does not guarantee that we will find the globally best solution.
- We must know the number of desired clusters beforehand.
- The decision boundaries of the algorithms are linear.
- The algorithm is slow for large datasets.

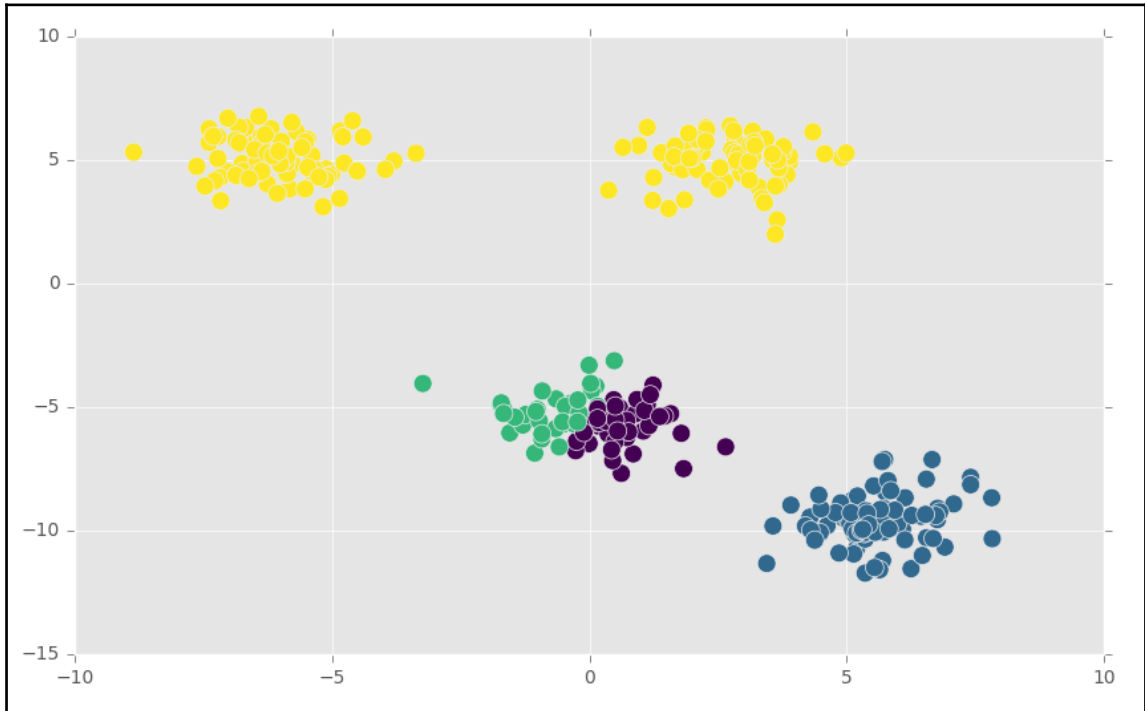
Let's quickly discuss these potential caveats in a little more detail.

First caveat: No guarantee of finding the global optimum

Although mathematicians have proved that the expectation-maximization step improves the result in each step, there is still no guarantee that, in the end, we will find the global best solution. For example, if we use a different random seed in our simple example (such as using seed 10 instead of 5), we suddenly get very poor results:

```
In [9]: centers, labels = find_clusters(X, 4, rseed=10)
...     plt.scatter(X[:, 0], X[:, 1], c=labels, s=100, cmap='viridis');
```

This will generate the following plot:



Example of *k*-means missing the global optimum

What happened?

The short answer is that the random initialization of cluster centers was unfortunate. It led to the center of the yellow cluster migrating in—between the two top blobs, essentially combining them into one. As a result, the other clusters got confused because they suddenly had to split two visually distinct blobs into three clusters.

For this reason, it is common for the algorithm to be run for multiple initial states. Indeed, OpenCV does this by default (set by the optional `attempts` parameter).

Second caveat: We must select the number of clusters beforehand

Another potential limitation is that k -means cannot learn the number of clusters from the data. Instead, we must tell it how many clusters we expect beforehand. You can see how this could be problematic for complicated real-world data that you don't fully understand yet.

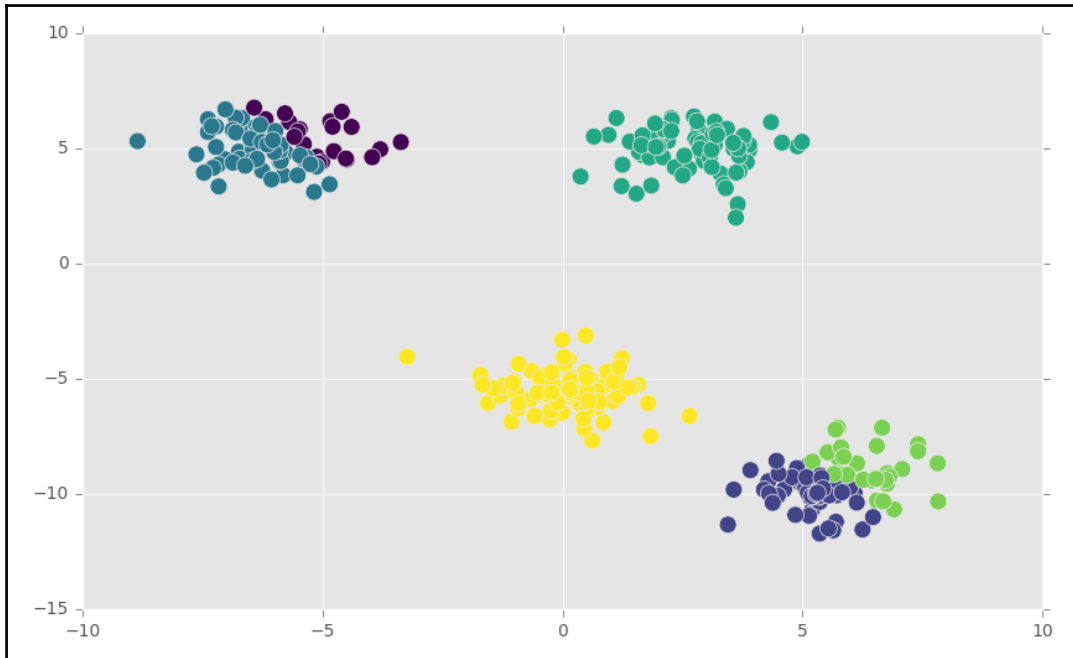
From the viewpoint of k -means, there is no wrong or nonsensical number of clusters. For example, if we ask the algorithm to identify six clusters in the dataset generated in the preceding section, it will happily proceed and find the best six clusters:

```
In [10]: criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
...               10, 1.0)
...       flags = cv2.KMEANS_RANDOM_CENTERS
...       compactness, labels, centers = cv2.kmeans(X.astype(np.float32),
...                                               6, None, criteria,
...                                               10, flags)
```

Here, we used the exact same preceding code and only changed the number of clusters from four to six. We can plot the data points again and color them according to the target labels:

```
In [11]: plt.scatter(X[:, 0], X[:, 1], c=labels, s=100, cmap='viridis');
```

The output looks like this:



Example of *k*-means discovering more clusters than there actually are

Since we generated the data ourselves, we know that each data point comes from a total of four distinct clusters. For more complicated data, where we are uncertain about the right number of clusters, there are a few things we could try.

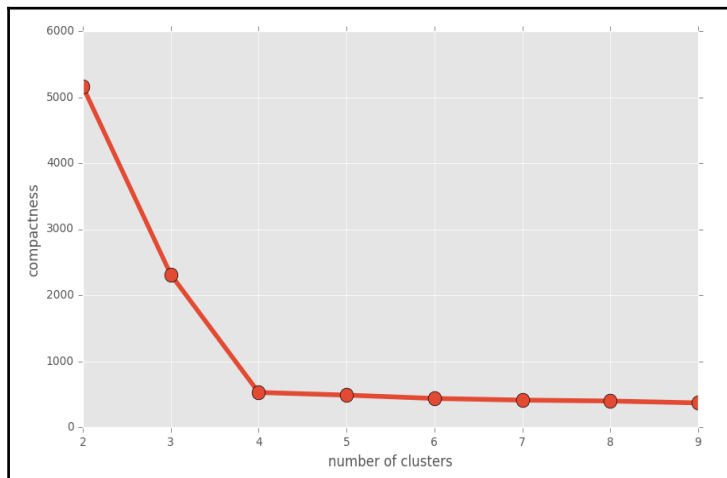
First and foremost, there is the **elbow method**, which asks us to repeat the clustering for a whole range of *k* values and record the compactness value:

```
In [12]: kvals = np.arange(2, 10)
...      compactness = []
...      for k in kvals:
...          c, _, _ = cv2.kmeans(X.astype(np.float32), k, None,
...                               criteria, 10, flags)
...          compactness.append(c)
```

Then we will plot the compactness as a function of *k*:

```
In [13]: plt.plot(kvals, compactness, 'o-', linewidth=4,
...               markersize=12)
...      plt.xlabel('number of clusters')
...      plt.ylabel('compactness')
```

This will result in a plot that looks like an *arm*. The point that lies at the *elbow* is the number of clusters to pick:



Elbow method

In our case, we would pick the number four. The reasoning behind this is that moving from left to right, four is the smallest number of clusters that gives a very compact representation. Three clusters give us a representation that is only half as compact as four clusters. Choosing five clusters or more does not further improve the compactness. Thus, four should be our best guess as to the number of clusters.

If our results are more complicated than that, we might want to consider a more sophisticated method. Some of the most common ones include the following:

- **Silhouette analysis** allows us to study the separation between the resulting clusters. If a lot of the points in one cluster are closer to a neighboring cluster than their own, we might be better off putting them all in a single cluster.
- **Gaussian mixture models** use something called the Bayesian information criterion to assess the right number of clusters for us.
- **Density-based spatial clustering of applications with noise (DBSCAN)** and **affinity propagation** are two more sophisticated clustering algorithms that can choose a suitable number of clusters for us.

In addition, we can also study the separation distance between the resulting clusters using what is known as a **silhouette plot**. This plot displays a measure of how close each point in one cluster is to points in the neighboring clusters, and thus provides a way to assess parameters, such as the number of clusters, visually. This measure has a range of $[-1, 1]$.

Third caveat: Cluster boundaries are linear

The k -means algorithm is based on a simple assumption, which is that points will be closer to their own cluster center than to others. Consequently, k -means always assumes linear boundaries between clusters, meaning that it will fail whenever the geometry of the clusters is more complicated than that.

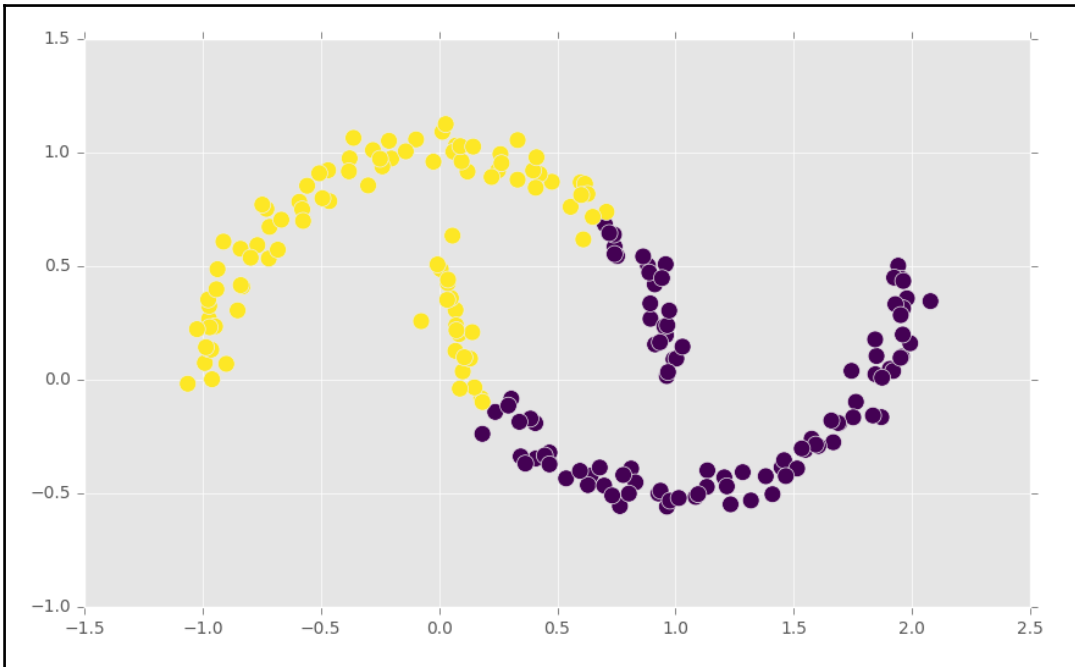
We see this limitation for ourselves by generating a slightly more complicated dataset. Instead of generating data points from Gaussian blobs, we want to organize the data into two overlapping half circles. We can do this using scikit-learn's `make_moons`. Here, we choose 200 data points belonging to two half circles, in combination with some Gaussian noise:

```
In [14]: from sklearn.datasets import make_moons
...      X, y = make_moons(200, noise=.05, random_state=12)
```

This time, we tell k -means to look for two clusters:

```
In [15]: criteria = (cv2.TERM_CRITERIA_EPS +
...                  cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
...      flags = cv2.KMEANS_RANDOM_CENTERS
...      compactness, labels, centers = cv2.kmeans(X.astype(np.float32),
...                                              2, None, criteria,
...                                              10, flags)
...      plt.scatter(X[:, 0], X[:, 1], c=labels, s=100, cmap='viridis');
```

The resulting scatter plot looks like this:



Example of k -means finding linear boundaries in nonlinear data

As is evident from the plot, k -means failed to identify the two half circles and instead split the data with what looks like a diagonal straight line (from bottom left to top right).

This scenario should ring a bell. We had the exact same problem when we talked about linear SVMs in Chapter 6, *Detecting Pedestrians with Support Vector Machines*. The idea there was to use the kernel trick in order to transform the data into a higher-dimensional feature space. Can we do the same here?

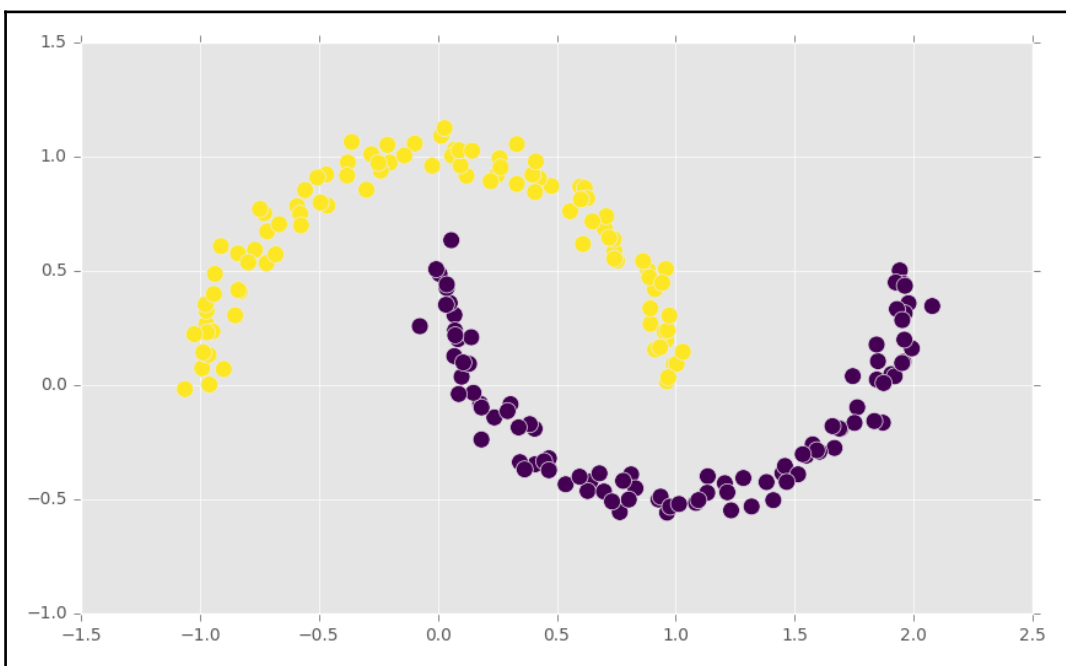
We most certainly can. There is a form of kernelized k -means that works akin to the kernel trick for SVMs, called **spectral clustering**. Unfortunately, OpenCV does not provide an implementation of spectral clustering. Fortunately, scikit-learn does:

```
In [16]: from sklearn.cluster import SpectralClustering
```

The algorithm uses the same API as all other statistical models: We set optional arguments in the constructor and then call `fit_predict` on the data. Here, we want to use the graph of the nearest neighbors to compute a higher-dimensional representation of the data and then assign labels using *k*-means:

```
In [17]: model = SpectralClustering(n_clusters=2,  
...                               affinity='nearest_neighbors',  
...                               assign_labels='kmeans')  
...     labels = model.fit_predict(X)  
...     plt.scatter(X[:, 0], X[:, 1], c=labels, s=100, cmap='viridis');
```

The output of spectral clustering looks like this:



Spectral clustering is able to find nonlinear cluster boundaries

We see that spectral clustering gets the job done. Alternatively, we could have transformed the data into a more suitable representation ourselves and then applied OpenCV's linear *k*-means to it. The lesson of all this is that perhaps, again, feature engineering saves the day.

Fourth caveat: k-means is slow for a large number of samples

The final limitation of *k*-means is that it is relatively slow for large datasets. You can imagine that quite a lot of algorithms might suffer from this problem. However, *k*-means is affected especially badly: each iteration of *k*-means must access every single data point in the dataset and compare it to all the cluster centers.

You might wonder whether the requirement to access all data points during each iteration is really necessary. For example, you might just use a subset of the data to update the cluster centers at each step. Indeed, this is the exact idea that underlies a variation of the algorithm called **batch-based *k*-means**. Unfortunately, this algorithm is not implemented in OpenCV.



K-means is provided by scikit-learn as part of their clustering module:
`sklearn.cluster.MiniBatchKMeans`.

Despite the limitations discussed earlier, *k*-means has a number of interesting applications, especially in computer vision.

Compressing color spaces using k-means

One exciting application of *k*-means is the compression of image color spaces. For example, a typical **true-color image** comes with a 24-bit color depth, allowing for a total of 16,777,216 color variations. However, in most images, a large number of the colors will be unused, and many of the pixels in the image will have similar or identical colors.

Alternatively, we can use *k*-means to reduce the color palette to, for example, 16 color variations. The trick here is to think of the cluster centers as the reduced color palette. Then *k*-means will automatically organize the millions of colors in the original image into the appropriate number of colors!

Visualizing the true-color palette

Let's have a look at a particular image:

```
In [1]: import cv2
...     import numpy as np
...     lena = cv2.imread('data/lena.jpg', cv2.IMREAD_COLOR)
```

By now, we know how to start matplotlib in our sleep:

```
In [2]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     plt.style.use('ggplot')
```

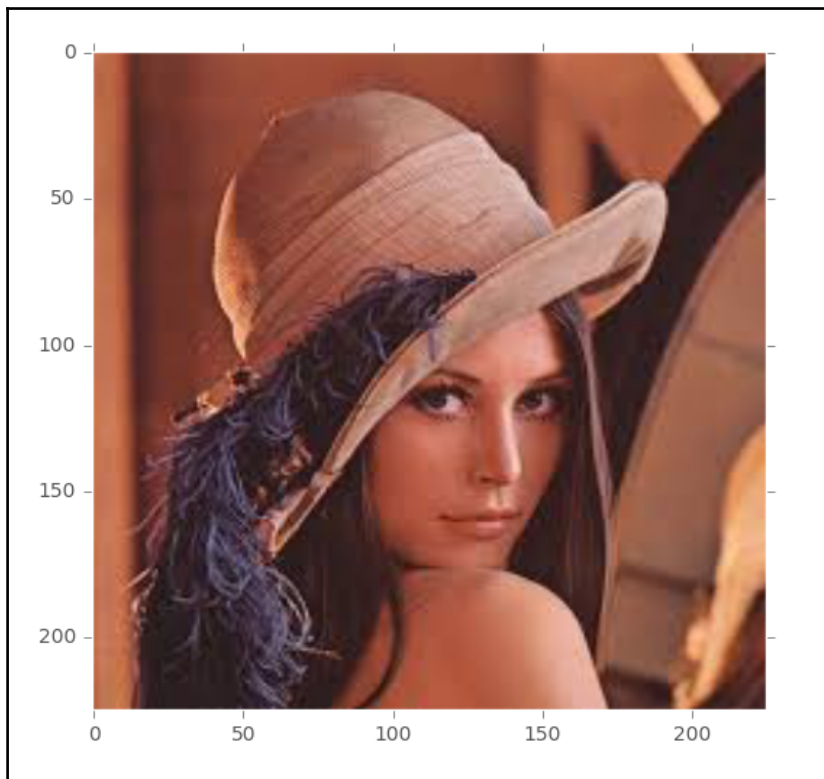
However, this time we want to disable the grid lines that the `ggplot` option typically displays over images:

```
In [3]: plt.rc('axes', **{'grid': False})
```

Then we can visualize Lena with the following command (don't forget to switch the BGR ordering of the color channels to RGB):

```
In [4]: plt.imshow(cv2.cvtColor(lena, cv2.COLOR_BGR2RGB))
```

This will produce the following output:



A picture of Lena (<http://sipi.usc.edu/database/database.php?volume=misc&image=12>).

The image itself is stored in a 3D array of size (height, width, depth) containing blue/green/red contributions as integers from 0 to 255:

```
In [5]: lena.shape
Out[5]: (225, 225, 3)
```

Because every color channel has 256 possible values, the number of possible colors is $256 \times 256 \times 256$, or 16,777,216, as mentioned prior to this. One way to visualize the sheer amount of different colors in the image is to reshape the data to a cloud of points in a 3D color space. We also scale the colors to lie between 0 and 1:

```
In [6]: img_data = lena / 255.0
...     img_data = img_data.reshape((-1, 3))
...     img_data.shape
Out[6]: (50625, 3)
```

In this 3D color space, every row of `data` is a data point. In order to visualize this data, we will write a function called `plot_pixels`, which takes as input the data matrix and a figure title. Optionally, we also let the user specify the colors to use. For the sake of efficiency, we can also limit the analysis to a subset of N pixels:

```
In [7]: def plot_pixels(data, title, colors=None, N=10000):
...     if colors is None:
...         colors = data
... 
```

If a number N is specified, we will randomly choose N data points from the list of all possible ones:

```
...     rng = np.random.RandomState(0)
...     i = rng.permutation(data.shape[0])[:N]
...     colors = colors[i]
```

Because each data point has three dimensions, plotting is a bit tricky. We could either create a 3D plot using Matplotlib's `mplot3d` toolkit, or we could produce two plots that visualize a subspace of the 3D point cloud. In this example, we will opt for the latter and generate two plots, one where we plot red versus green and another where we plot red versus blue. For this, we need to access the *R*, *G*, and *B* values of each data point as column vectors:

```
...     pixel = data[i].T
...     R, G, B = pixel[0], pixel[1], pixel[2]
```

The first plot then shows red on the *x*-axis and green on the *y*-axis:

```
...     fig, ax = plt.subplots(1, 2, figsize=(16, 6))
...     ax[0].scatter(R, G, color=colors, marker='.')
...     ax[0].set(xlabel='Red', ylabel='Green', xlim=(0, 1),
```

```
...                ylim=(0, 1))
```

Analogously, the second figure is a scatter plot of red on the x -axis and blue on the y -axis:

```
...                ax[1].scatter(R, B, color=colors, marker='.')
...                ax[1].set(xlabel='Red', ylabel='Blue', xlim=(0, 1),
...                ylim=(0, 1))
```

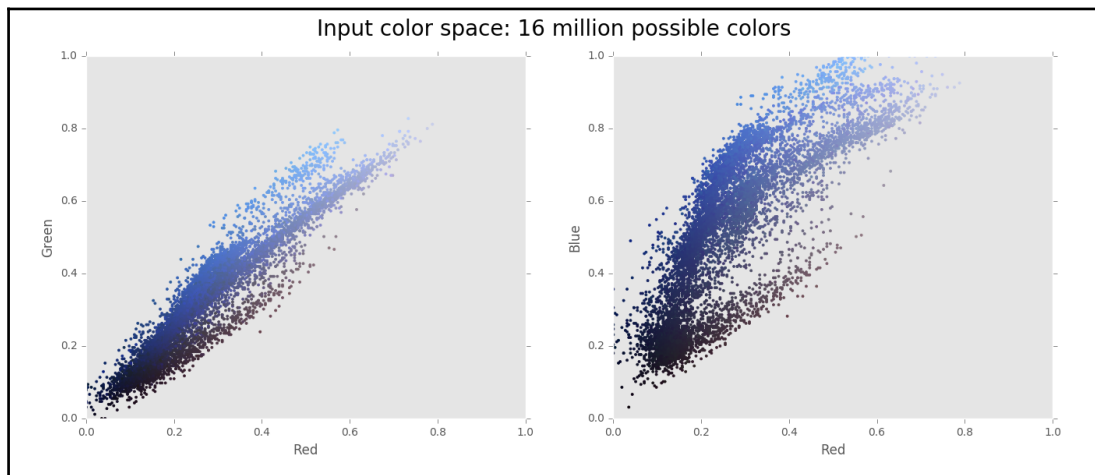
As a last step, we will display the specified title centered over the two subplots:

```
...                fig.suptitle(title, size=20);
```

We can then call the function with our data matrix (`data`) and an appropriate title:

```
In [8]: plot_pixels(img_data, title='Input color space: 16 million '
...                'possible colors')
```

This will produce the following output:



Input color space

Let's see what more we can do with this algorithm.

Reducing the color palette using k-means

Now let's reduce these 16 million colors to just 16 colors by telling *k*-means to cluster all color variations into 16 distinct clusters. We use the before mentioned procedure, but now specify 16 as the number of clusters:

```
In [9]: criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
...               10, 1.0)
...     flags = cv2.KMEANS_RANDOM_CENTERS
...     img_data = img_data.astype(np.float32)
...     compactness, labels, centers = cv2.kmeans(img_data,
...                                             16, None, criteria,
...                                             10, flags)
```

The resulting cluster corresponds to the 16 colors of our reduced color palette. Visual inspection of the `centers` array reveals that all colors have three entries—B, G, and R—with values between 0 and 1:

```
In [10]: centers
Out[10]: array([[ 0.29973754,  0.31500012,  0.48251548],
 [ 0.27192295,  0.35615689,  0.64276862],
 [ 0.17865284,  0.20933454,  0.41286203],
 [ 0.39422086,  0.62827665,  0.94220853],
 [ 0.34117648,  0.58823532,  0.90196079],
 [ 0.42996961,  0.62061119,  0.91163337],
 [ 0.06039202,  0.07102439,  0.1840712 ],
 [ 0.5589878 ,  0.6313886 ,  0.83993536],
 [ 0.37320262,  0.54575169,  0.88888896],
 [ 0.35686275,  0.57385623,  0.88954246],
 [ 0.47058824,  0.48235294,  0.59215689],
 [ 0.34346411,  0.57483661,  0.88627452],
 [ 0.13815609,  0.12984112,  0.21053818],
 [ 0.3752504 ,  0.47029912,  0.75687987],
 [ 0.31909946,  0.54829341,  0.87378371],
 [ 0.40409693,  0.58062142,  0.8547557 ]], dtype=float32)
```

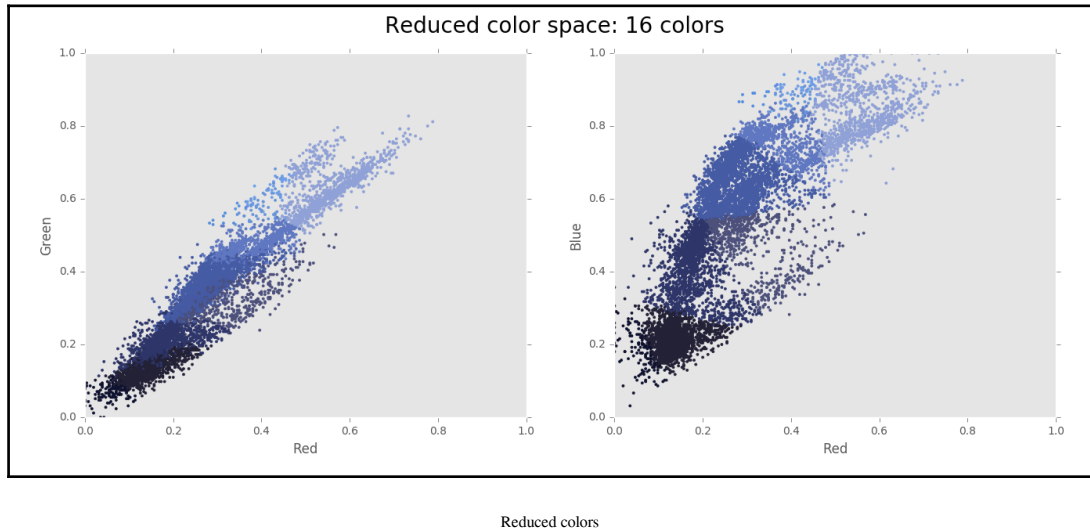
These 16 colors correspond to the 16 cluster labels contained in the `labels` vector. So we want all data points with label 0 to be colored according to row 0 in the `centers` array; all data points with label 1 to be colored according to row 1 in the `centers` array; and so on. In other words, we want to use `labels` as an index into the `centers` array—these are our new colors:

```
In [11]: new_colors = centers[labels].reshape((-1, 3))
```

We can plot the data again, but this time, we will use `new_colors` to color the data points accordingly:

```
In [12]: plot_pixels(img_data, colors=new_colors,
...                 title="Reduce color space: 16 colors")
```

The result is the recoloring of the original pixels, where each pixel is assigned the color of its closest cluster center:



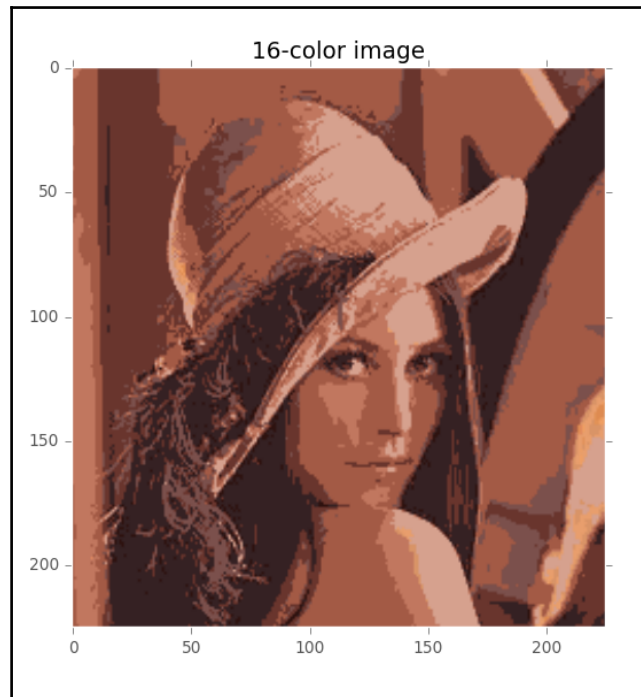
In order to see the effect of this recoloring, we have to plot `new_colors` as an image. In order to get from the image to the data matrix, we flattened the earlier image. Now we need to do the inverse to get back to the image, which is to reshape `new_colors` according to the shape of the Lena image:

```
In [13]: lena_recolored = new_colors.reshape(lena.shape)
```

Then we can visualize the recolored Lena image like any other image:

```
In [14]: plt.figure(figsize=(10, 6))
...       plt.imshow(cv2.cvtColor(lena_recolored, cv2.COLOR_BGR2RGB));
...       plt.title('16-color image')
```

The result looks like this:



16-color version of Lena

Pretty cool, right?

Although some detail is arguably lost, overall, the preceding image is still clearly recognizable. This is pretty remarkable, considering that we just compressed the image by a factor of around 1 million!

You can repeat this procedure for any desired number of colors.



Another way to reduce the color palette of images involves the use of **bilateral filters**. The resulting images often look like cartoon versions of the original image. You can find an example of this in the book *OpenCV with Python Blueprints*, M. Beyeler Packt Publishing.

Another potential application of k -means is something you might not expect: to use it for image classification.

Classifying handwritten digits using k -means

Although the last application was a pretty creative use of k -means, we can do better still. We have previously discussed k -means in the context of unsupervised learning, where we tried to discover some hidden structure in the data.

However, doesn't the same concept apply to most classification tasks? Let's say our task was to classify handwritten digits. Don't most zeros look similar, if not the same? And don't all zeros look categorically different from all possible ones? Isn't this exactly the kind of "hidden structure" we set out to discover with unsupervised learning? Doesn't this mean we could use clustering for classification as well?

Let's find out together. In this section, we will attempt to use k -means to try and classify handwritten digits. In other words, we will try to identify similar digits without using the original label information.

Loading the dataset

From the earlier chapters, you might recall that scikit-learn provides a whole range of handwritten digits via its `load_digits` utility function. The dataset consists of 1,797 samples with 64 features each, where each of the features has the brightness of one pixel in an 8×8 image:

```
In [1]: from sklearn.datasets import load_digits
...     digits = load_digits()
...     digits.data.shape
Out[1]: (1797, 64)
```

Running k -means

Setting up k -means works exactly the same as in the previous examples. We tell the algorithm to perform at most 10 iterations and stop the process if our prediction of the cluster centers does not improve within a distance of 1.0:

```
In [2]: import cv2
...     criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
...                 10, 1.0)
...     flags = cv2.KMEANS_RANDOM_CENTERS
```

Then we apply k -means to the data as we did before. Since there are 10 different digits (0-9), we tell the algorithm to look for 10 distinct clusters:

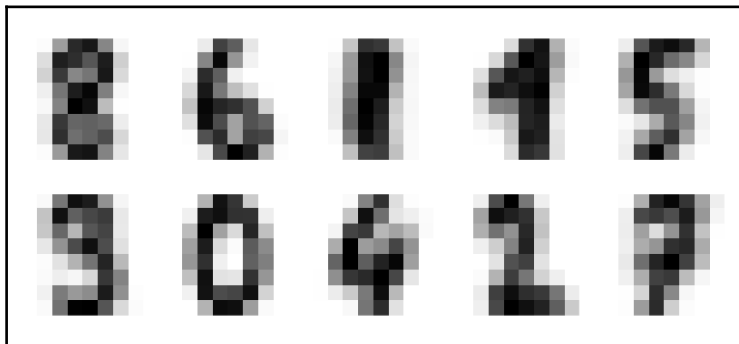
```
In [3]: import numpy as np
...     digits.data = digits.data.astype(np.float32)
...     compactness, clusters, centers = cv2.kmeans(digits.data, 10, None,
...                                                criteria, 10, flags)
```

And done!

Similar to the $N \times 3$ matrix that represented different RGB colors, this time, the centers array consists of $N \times 8 \times 8$ center images, where N is the number of clusters. Therefore, if we want to plot the centers, we have to reshape the `centers` matrix back into 8×8 images:

```
In [4]: import matplotlib.pyplot as plt
...     plt.style.use('ggplot')
...     %matplotlib inline
...     fig, ax = plt.subplots(2, 5, figsize=(8, 3))
...     centers = centers.reshape(10, 8, 8)
...     for axi, center in zip(ax.flat, centers):
...         axi.set(xticks=[], yticks=[])
...         axi.imshow(center, interpolation='nearest', cmap=plt.cm.binary)
```

The output looks like this:



Cluster centers recovered by k -means

Look familiar?

Remarkably, k -means was able to partition the digit images not just into any 10 random clusters, but into the digits 0-9! In order to find out which images were grouped into which clusters, we need to generate a `labels` vector as we know it from supervised learning problems:

```
In [5]: from scipy.stats import mode
...     labels = np.zeros_like(clusters.ravel())
...     for i in range(10):
...         mask = (clusters.ravel() == i)
...         labels[mask] = mode(digits.target[mask])[0]
```

Then we can calculate the performance of the algorithm using scikit-learn's `accuracy_score` metric:

```
In [6]: from sklearn.metrics import accuracy_score
...     accuracy_score(digits.target, labels)
Out[6]: 0.78464106844741233
```

Remarkably, *k*-means achieved 78.4% accuracy without knowing the first thing about the labels of the original images!

We can gain more insights about what went wrong and how by looking at the **confusion matrix**. The confusion matrix is a 2D matrix C , where every element $C_{i,j}$ is equal to the number of observations known to be in group (or cluster) i , but predicted to be in group j . Thus, all elements on the diagonal of the matrix represent data points that have been correctly classified (that is, known to be in group i and predicted to be in group i). Off-diagonal elements show misclassifications.

In scikit-learn, creating a confusion matrix is essentially a one-liner:

```
In [7]: from sklearn.metrics import confusion_matrix
...     confusion_matrix(digits.target, labels)
Out[7]: array([[177,  0,  0,  0,  1,  0,  0,  0,  0,  0],
               [ 0, 154, 25,  0,  0,  1,  2,  0,  0,  0],
               [ 1,  3, 147, 11,  0,  0,  0,  3, 12,  0],
               [ 0,  1,  2, 159,  0,  2,  0,  9, 10,  0],
               [ 0, 12,  0,  0, 162,  0,  0,  5,  2,  0],
               [ 0,  0,  0, 40,  2, 138,  2,  0,  0,  0],
               [ 1,  2,  0,  0,  0,  0, 177,  0,  1,  0],
               [ 0, 14,  0,  0,  0,  0,  0, 164,  1,  0],
               [ 0, 23,  3,  8,  0,  5,  1,  2, 132,  0],
               [ 0, 21,  0, 145,  0,  5,  0,  8,  1,  0]])
```

The confusion matrix tells us that *k*-means did a pretty good job at classifying data points from the first nine classes; however, it confused all nines to be (mostly) threes. Still, this result is pretty solid, given that the algorithm had no target labels to be trained on.

Organizing clusters as a hierarchical tree

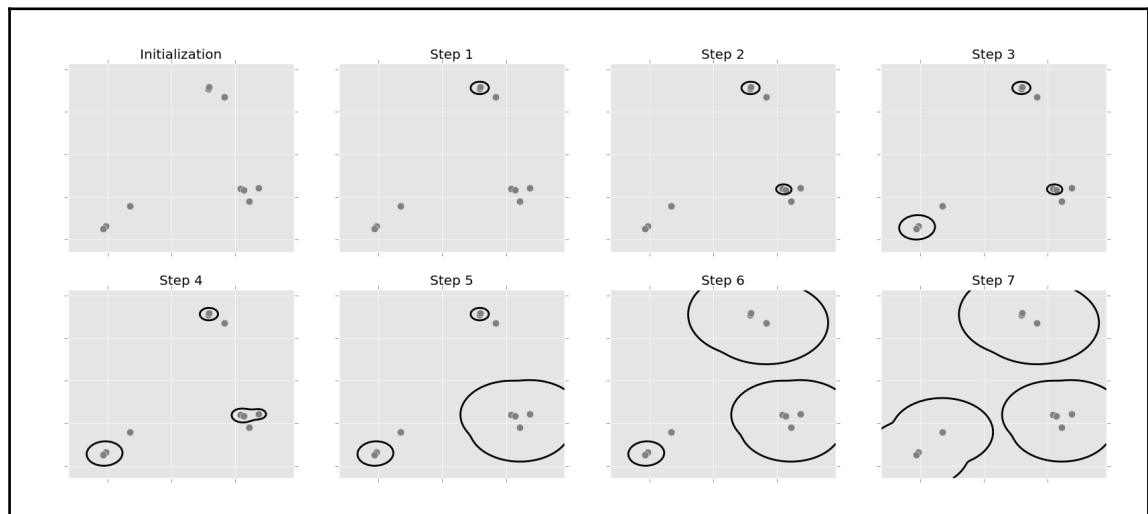
An alternative to k -means is **hierarchical clustering**. One advantage of hierarchical clustering is that it allows us to organize the different clusters in a hierarchy (also known as a **dendrogram**), which can make it easier to interpret the results. Another useful advantage is that we do not need to specify the number of clusters upfront.

Understanding hierarchical clustering

There are two approaches to hierarchical clustering:

- In **agglomerative hierarchical clustering**, we start with each data point potentially being its own cluster, and we subsequently merge the closest pair of clusters until only one cluster remains.
- In **divisive hierarchical clustering**, it's the other way around: We start by assigning all the data points to one and the same cluster, and we subsequently split the cluster into smaller clusters until each cluster only contains one sample.

Of course, we can specify the number of desired clusters if we wish to. In the following figure, we asked the algorithm to find a total of three clusters:



Step-by-step example of agglomerative hierarchical clustering

In step 1, the algorithm put the two closest points into their own cluster (middle, top). In step 2, two points on the lower right turned out to be the closest pair across all possible pairs in the dataset, so they were merged into their own cluster. This process continued until all data points were assigned to one of the three clusters (step 7), at which point the algorithm terminated.



If you are using OpenCV's C++ API, you might also want to check out **Fast Approximate Nearest Neighbor (FLANN)**-based hierarchical clustering using *k*-means in `cv::flann::hierarchicalClustering`.

Let's dig deeper into implementing the agglomerative hierarchical clustering.

Implementing agglomerative hierarchical clustering

Although OpenCV does not provide an implementation of agglomerative hierarchical clustering, it is a popular algorithm that should, by all means, belong to our machine learning repertoire.

We start out by generating 10 random data points, just like in the previous figure:

```
In [1]: from sklearn.datasets import make_blobs
...     X, y = make_blobs(random_state=100, n_samples=10)
```

Using the familiar statistical modeling API, we import the `AgglomerativeClustering` algorithm and specify the desired number of clusters:

```
In [2]: from sklearn import cluster
...     agg = cluster.AgglomerativeClustering(n_clusters=3)
```

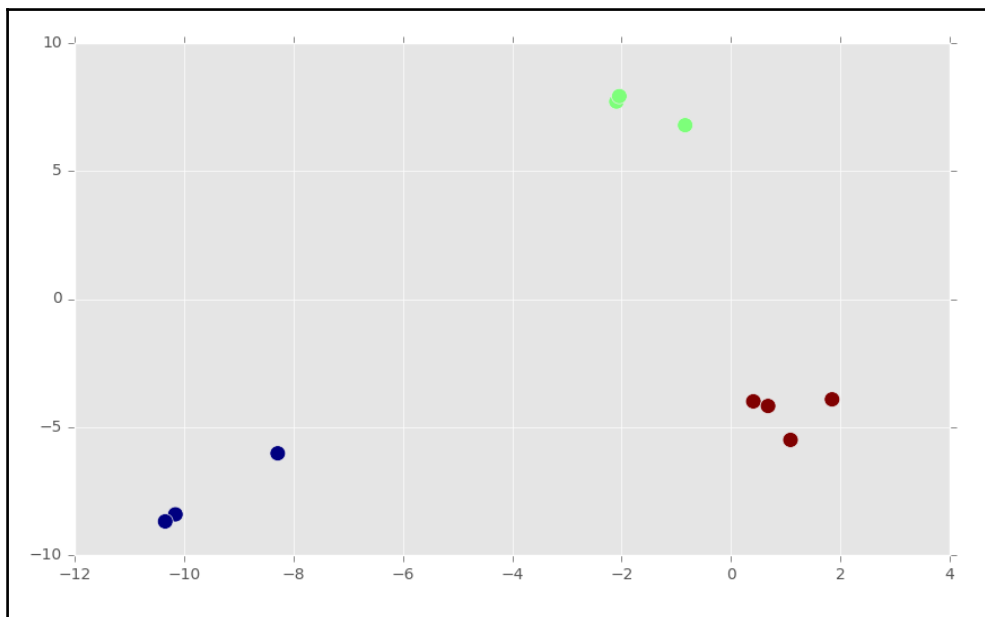
Fitting the model to the data works, as usual, via the `fit_predict` method:

```
In [3]: labels = agg.fit_predict(X)
```

We can generate a scatter plot where every data point is colored according to the predicted label:

```
In [4]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     plt.style.use('ggplot')
...     plt.scatter(X[:, 0], X[:, 1], c=labels, s=100)
```

The resulting clustering is equivalent to the following figure:



Clustering result of agglomerative hierarchical clustering

That's it! This marks the end of another wonderful adventure.

Summary

In this chapter, we talked about a number of unsupervised learning algorithms, including k -means, spherical clustering, and agglomerative hierarchical clustering. We saw that k -means is just a specific application of the more general expectation-maximization algorithm, and we discussed its potential limitations.

Furthermore, we applied k -means to two specific applications, which were to reduce the color palette of images and to classify handwritten digits.

In the next chapter, we will move back into the world of supervised learning and talk about some of the most powerful current machine learning algorithms: **neural networks** and **deep learning**.

9

Using Deep Learning to Classify Handwritten Digits

Let's now return to supervised learning and discuss a family of algorithms known as **artificial neural networks**. Early studies of neural networks go back to the 1940s when Warren McCulloch and Walter Pitts first described how biological nerve cells (or **neurons**) in the brain might work. More recently, artificial neural networks have seen a revival under the buzzword **deep learning**, which powers state-of-the-art technologies, such as Google's DeepMind and Facebook's DeepFace algorithms.

In this chapter, we want to wrap our heads around some simple versions of artificial neural nets, such as the **McCulloch-Pitts neuron**, the **perceptron**, and the **multilayer perceptron**. Once we have familiarized ourselves with the basics, we will be ready to implement a more sophisticated deep neural net in order to classify handwritten digits from the popular **MNIST database** (short for **Mixed National Institute of Standards and Technology database**). For this, we will be making use of **Keras**, a high-level neural network library, which is also frequently used by researchers and tech companies.

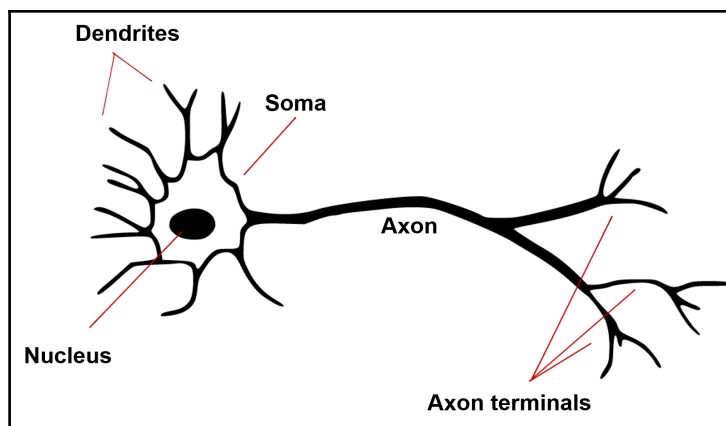
Along the way, we want to get answers to the following questions:

- How do I implement perceptrons and multilayer perceptrons in OpenCV?
- What is the difference between stochastic and batch gradient descent, and how does it fit in with backpropagation?
- How do I know what size my neural net should be?
- How can I use Keras to build sophisticated deep neural networks?

Excited? Then let's go!

Understanding the McCulloch-Pitts neuron

In 1943, Warren McCulloch and Walter Pitts published a mathematical description of neurons as they were believed to operate in the brain. A neuron receives input from other neurons through connections on its **dendritic tree**, which are integrated to produce an output at the cell body (or **soma**). The output is then communicated to other neurons via a long wire (or **axon**), which eventually branches out to make one or more connections (at **axon terminals**) on the dendritic tree of other neurons. An example neuron is shown in the following figure:



Schematic of a neuron (nerve cell). Adapted from a figure by Looxix at French Wikipedia (CC BY-SA 3.0)

McCulloch and Pitts described the inner workings of such a neuron as a simple logic gate that would be either on or off, depending on the input it receives on its dendritic tree. Specifically, the neuron would sum up all of its inputs, and if the sum exceeded a certain threshold, an output signal would be generated and passed on by the axon.



However, today we know that real neurons are much more complicated than that. Biological neurons perform intricate nonlinear mathematical operations on thousands of inputs and can change their responsiveness dynamically depending on the context, importance, or novelty of the input signal. You can think of real neurons being as complex as computers and of the human brain being as complex as the internet.

Let's consider a simple artificial neuron that receives exactly two inputs, x_0 and x_1 . The job of the artificial neuron is to calculate a sum of the two inputs (usually in the form of a **weighted sum**), and if this sum exceeds a certain threshold (often zero), the neuron will be considered active and output a one; else it will be considered silent and output a minus one (or zero). In more mathematical terms, the output y of this **McCulloch-Pitts neuron** can be described as follows:

$$y = \begin{cases} 1 & \text{if } x_0w_0 + x_1w_1 \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

In the preceding equation, w_0 and w_1 are **weight coefficients**, which, together with x_0 and x_1 , make up the weighted sum. In textbooks, the two different scenarios where the output y is either +1 and -1 would often be masked by an **activation function** ϕ , which could take on two different values:

$$y = \phi(x_0w_0 + x_1w_1) = \phi(z)$$

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

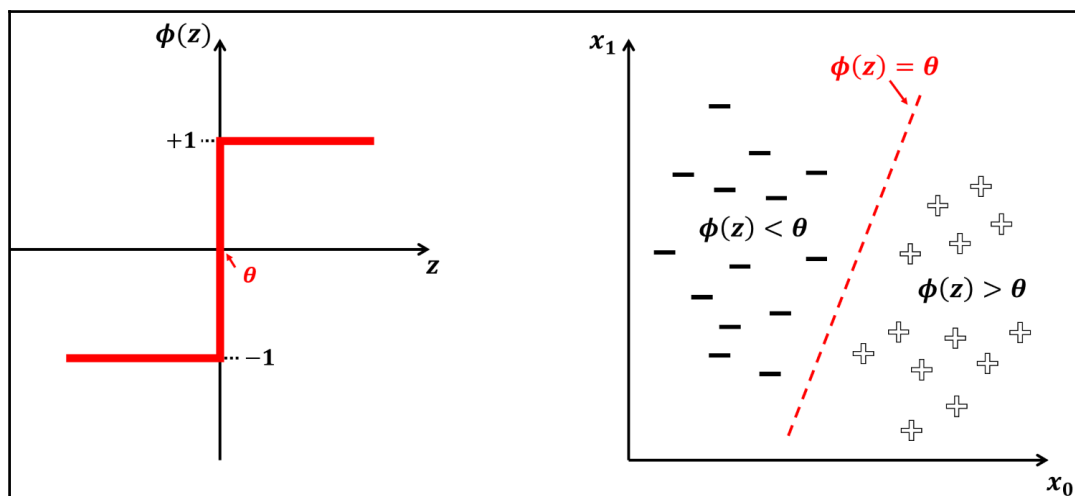
Here, we introduce a new variable z (the so-called **net input**), which is equivalent to the weighted sum: $z = w_0x_0 + w_1x_1$. The weighted sum is then compared to a threshold θ to determine the value of ϕ and subsequently the value of y . Apart from that, these two equations say exactly the same thing as the preceding one.

If these equations look strangely familiar, you might be reminded of Chapter 3, *A Taste of Machine Learning*, when we were talking about linear classifiers.

And you are right, a McCulloch-Pitts neuron is essentially a linear, binary classifier!

You can think of it this way: x_0 and x_1 are the input features, w_0 and w_1 are weights to be learned, and the classification is performed by the activation function ϕ . If we do a good job of learning the weights, which we would do with the help of a suitable training set, we could classify data as positive or negative samples. In this scenario, $\phi(z)=\theta$ would act as the decision boundary.

This might all make more sense with the help of the following figure:



An artificial neuron's activation function (left) used to classify some example data (right)

On the left, you can see the neuron's activation function ϕ plotted against z . Remember that z is nothing more than the weighted sum of the two inputs x_0 and x_1 . The rule is that as long as the weighted sum is below some threshold θ , the output of the neuron is -1; above θ , the output is +1. On the right, you can see the decision boundary denoted by $\phi(z) = \theta$, which splits the data into two regimes, $\phi(z) < \theta$ (where all data points are predicted to be negative samples) and $\phi(z) > \theta$ (where all data points are predicted to be positive samples).



The decision boundary does not need to be vertical or horizontal, it can be tilted as shown in the preceding figure. But in the case of a single McCulloch-Pitts neuron, the decision boundary will always be a straight line.

Of course, the magic lies with learning the weight coefficients w_0 and w_1 such that the decision boundary comes to lie right between all positive and all negative data points. To train a neural network, we generally need three things:

- **Training data:** It is no surprise to learn that we need some data samples with which the effectiveness of our classifier can be verified.
- **Cost function** (also known as **loss function**): A cost function provides a measure of how good the current weight coefficients are. There is a wide range of cost functions available, which we will talk about towards the end of this chapter. One solution is to count the number of misclassifications. Another one is to calculate the **sum of squared errors**.

- **Learning rule:** A learning rule specifies mathematically how we have to update the weight coefficients from one iteration to the next. This learning rule usually depends on the error (measured by the cost function) we observed on the training data.

This is where the work of renowned researcher Frank Rosenblatt comes in.

Understanding the perceptron

In the 1950s, American psychologist and artificial intelligence researcher Frank Rosenblatt invented an algorithm that would automatically learn the optimal weight coefficients w_0 and w_1 needed to perform an accurate binary classification: the **perceptron learning rule**.

Rosenblatt's original perceptron algorithm can be summed up as follows:

1. Initialize the weights to zero or some small random numbers.
2. For each training sample s_i , perform the following steps:
 1. Compute the predicted target value $\hat{y}_i$.
 2. Compare $\hat{y}_i$ to the ground truth y_i , and update the weights accordingly:
 - If the two are the same (correct prediction), skip ahead.
 - If the two are different (wrong prediction), push the weight coefficients w_0 and w_1 towards the positive or negative target class respectively.

Let's have a closer look at the last step, which is the **weight update rule**. The goal of the weight coefficients is to take on values that allow for successful classification of the data. Since we initialize the weights either to zero or small random numbers, chances that we get 100% accuracy from the outset are incredibly low. We, therefore, need to make small changes to the weights such that our classification accuracy improves.

In the two-input case, this means that we have to update w_0 by some small change Δw_0 and w_1 by some small change Δw_1 :

$$\begin{aligned}w_0 &= w_0 + \Delta w_0 \\w_1 &= w_1 + \Delta w_1\end{aligned}$$

Rosenblatt reasoned that we should update the weights in a way that makes classification more accurate. We thus need to compare the perceptron's predicted output $\hat{y}_i$ for every data point i to the ground truth y_i . If the two values are the same, the prediction was correct, meaning the perceptron is already doing a great job and we shouldn't be changing anything. However, if the two values are different, we should push the weights closer to the positive or negative class respectively. This can be summed up with the following equations:

$$\begin{aligned}\Delta w_0 &= \eta (y_i - \hat{y}_i) x_0 \\ \Delta w_1 &= \eta (y_i - \hat{y}_i) x_1\end{aligned}$$

Here, the parameter η denotes the learning rate (usually a constant between 0 and 1). Usually, η is chosen sufficiently small so that with every data sample s_i , we make only a **small step** toward the desired solution.

To get a better intuition of this update rule, let's assume that for a particular sample s_i , we predicted $\hat{y}_i = -1$, but the ground truth would have been $y_i = +1$. If we set $\eta = 1$, then we have $\Delta w_0 = (1 - (-1)) x_0 = 2 x_0$. In other words, the update rule would want to increase the weight value w_0 by $2 x_0$. A stronger weight value would make it more likely for the weighted sum to be above the threshold next time, thus hopefully classifying the sample as $\hat{y}_i = -1$ next time around.

On the other hand, if $\hat{y}_i$ is the same as y_i , then $(\hat{y}_i - y_i)$ would cancel out, and $\Delta w_0 = 0$.



The perceptron learning rule is **guaranteed to converge** on the optimal solution only if the data is linearly separable and the learning rate is sufficiently small.

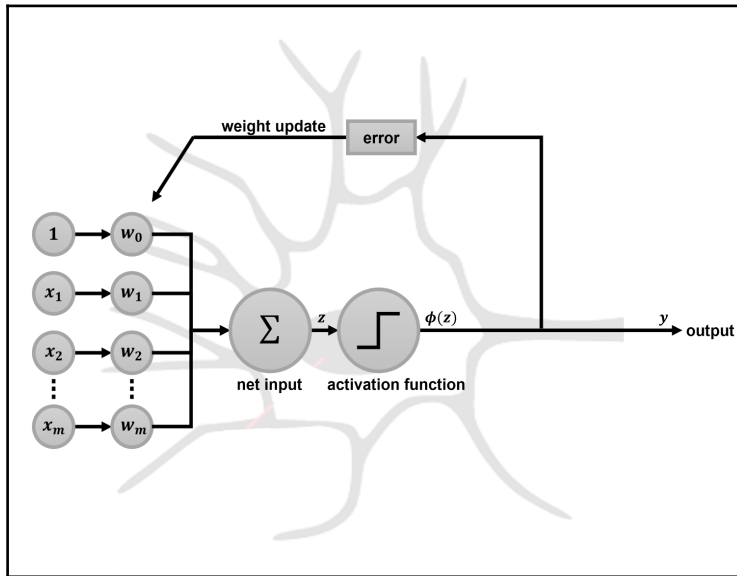
It is straightforward—in a mathematical sense—to extend the perceptron learning rule to more than two inputs x_0 and x_1 by extending the number of terms in the weighted sum:

$$z = x_0 w_0 + x_1 w_1 + \cdots + x_m w_m = \sum_{i=1}^m x_i w_i$$

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

It is customary to make one of the weight coefficients not depend on any input features (usually w_0 , so $x_0 = 1$) so that it can act as a scalar (or **bias term**) in the weighted sum.

To make the perceptron algorithm look more like an artificial neuron, we can illustrate it as follows:



Illustrating the perceptron algorithm as an artificial neuron

Here, the input features and weight coefficients are interpreted as part of the dendritic tree, where the neuron gets all its inputs from. These inputs are then summed up and passed through the activation function, which happens at the cell body. The output is then sent along the axon to the next cell. In the perceptron learning rule, we also made use of the neuron's output to calculate the error, which, in turn, was used to update the weight coefficients.



One distinction between the 1950s version of the perceptron and modern perceptrons is the use of a more sophisticated activation function ϕ ; for example, the hyperbolic tangent or the sigmoid function.

Now, let's try it on some sample data!

Implementing your first perceptron

Perceptrons are easy enough to be implemented from scratch. We can mimic the typical OpenCV or scikit-learn implementation of a classifier by creating a `Perceptron` object. This will allow us to initialize new perceptron objects that can learn from data via a `fit` method and make predictions via a separate `predict` method.

When we initialize a new perceptron object, we want to pass a learning rate (`lr`, or η in the previous section) and the number of iterations after which the algorithm should terminate (`n_iter`):

```
In [1]: import numpy as np
In [2]: class Perceptron(object):
...     def __init__(self, lr=0.01, n_iter=10):
...         self.lr = lr
...         self.n_iter = n_iter
...
```

The `fit` method is where most of the work is done. This method should take as input some data samples (`x`) and their associated target labels (`y`). We will then create an array of weights (`self.weights`), one for each feature (`X.shape[1]`), initialized to zero. For convenience, we will keep the bias term (`self.bias`) separate from the weight vector and initialize it to zero as well:

```
...     def fit(self, X, y):
...         self.weights = np.zeros(X.shape[1])
...         self.bias = 0.0
```

The `predict` method should take in a number of data samples (`x`) and for each of them, return a target label, either +1 or -1. In order to perform this classification, we need to implement $\phi(z) > \theta$. Here we will choose $\theta = 0$, and the weighted sum can be computed with NumPy's dot product:

```
...     def predict(self, X):
...         return np.where(np.dot(X, self.weights) + self.bias >= 0.0,
...                          1, -1)
```

Then we will calculate the Δw terms for every data sample (x_i, y_i) in the dataset and repeat this step for a number of iterations (`self.n_iter`). For this, we need to compare the ground-truth label (y_i) to the predicted label (aforementioned `self.predict(x_i)`). The resulting delta term will be used to update both the weights and the bias term:

```
...         for _ in range(self.n_iter):
...             for xi, yi in zip(X, y):
...                 delta = self.lr * (yi - self.predict(xi))
...                 self.weights += delta * xi
...                 self.bias += delta
```

That's it!

Generating a toy dataset

To test our perceptron classifier, we need to create some mock data. Let's keep things simple for now and generate 100 data samples (`n_samples`) belonging to one of two blobs (`centers`), again relying on scikit-learn's `make_blobs` function:

```
In [3]: from sklearn.datasets.samples_generator import make_blobs
...      X, y = make_blobs(n_samples=100, centers=2,
...                        cluster_std=2.2, random_state=42)
```

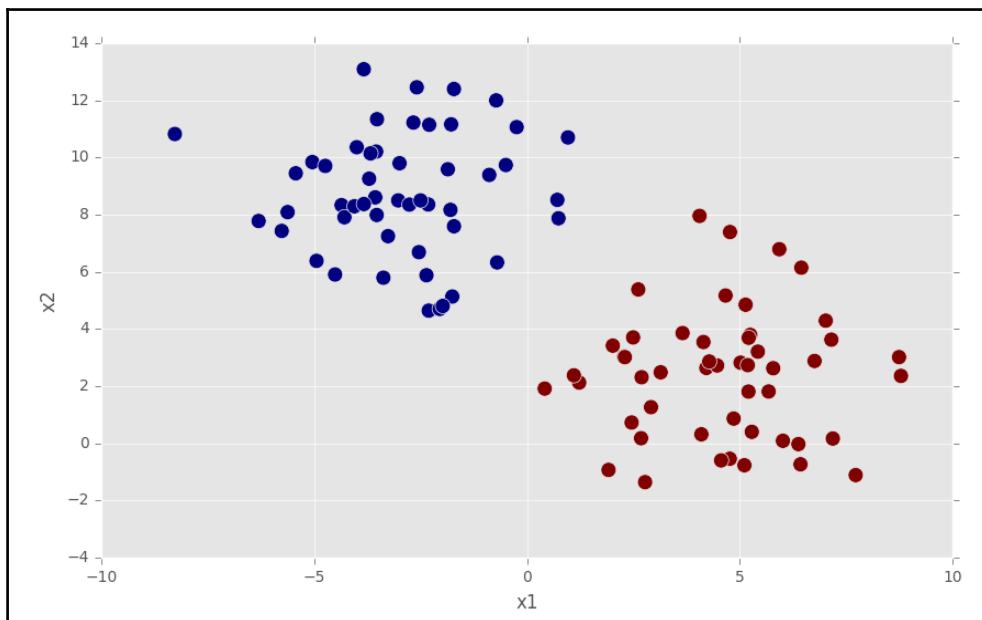
One thing to keep in mind is that our perceptron classifier expects target labels to be either +1 or -1, whereas `make_blobs` returns 0 and 1. An easy way to adjust the labels is with the following equation:

```
In [4]: y = 2 * y - 1
```

Let's have a look at the data:

```
In [5]: import matplotlib.pyplot as plt
...      plt.style.use('ggplot')
...      %matplotlib inline
...      plt.scatter(X[:, 0], X[:, 1], s=100, c=y);
...      plt.xlabel('x1')
...      plt.ylabel('x2')
```

This will produce the following plot:



Example dataset for the perceptron classifier

What do you think, will our perceptron classifier have an easy time finding a decision boundary that separates these two blobs?

Chances are it will. We mentioned earlier that a perceptron is a linear classifier. This means that as long as you can draw a straight line in the preceding plot to separate the two blobs, there exists a linear decision boundary that the perceptron should be able to find, that is, if we implemented everything correctly. Let's find out.

Fitting the perceptron to data

We can instantiate our perceptron object similar to other classifiers we encountered with OpenCV:

```
In [6]: p = Perceptron(lr=0.1, n_iter=10)
```

Here, we chose a learning rate of 0.1 and told the perceptron to terminate after 10 iterations. These values are chosen rather arbitrarily at this point, although we will come back to them in a little while.



Choosing an appropriate learning rate is critical, but it's not always clear what the most appropriate choice is. The learning rate determines how fast or slow we move towards the optimal weight coefficients. If the learning rate is too large, we might accidentally skip the optimal solution. If it is too small, we will need a large number of iterations to converge to the best values.

Once the perceptron is set up, we can call the `fit` method to optimize the weight coefficients:

```
In [7]: p.fit(X, y)
```

Did it work? Let's have a look at the learned weight values:

```
In [8]: p.weights
Out[8]: array([ 2.20091094, -0.4798926 ])
```

And don't forget to have a peek at the bias term:

```
In [9]: p.bias
Out[9]: 0.20000000000000001
```

If we plug these values into our equation for ϕ , it becomes clear that the perceptron learned a decision boundary of the form $2.2 x_1 - 0.48 x_2 + 0.2 \geq 0$.

Evaluating the perceptron classifier

In order to find out how good our perceptron performs, we can calculate the accuracy score on all data samples:

```
In [10]: from sklearn.metrics import accuracy_score
...       accuracy_score(p.predict(X), y)
Out[10]: 1.0
```

Perfect score!

Let's have a look at the decision landscape by bringing back our `plot_decision_boundary` from the earlier chapters:

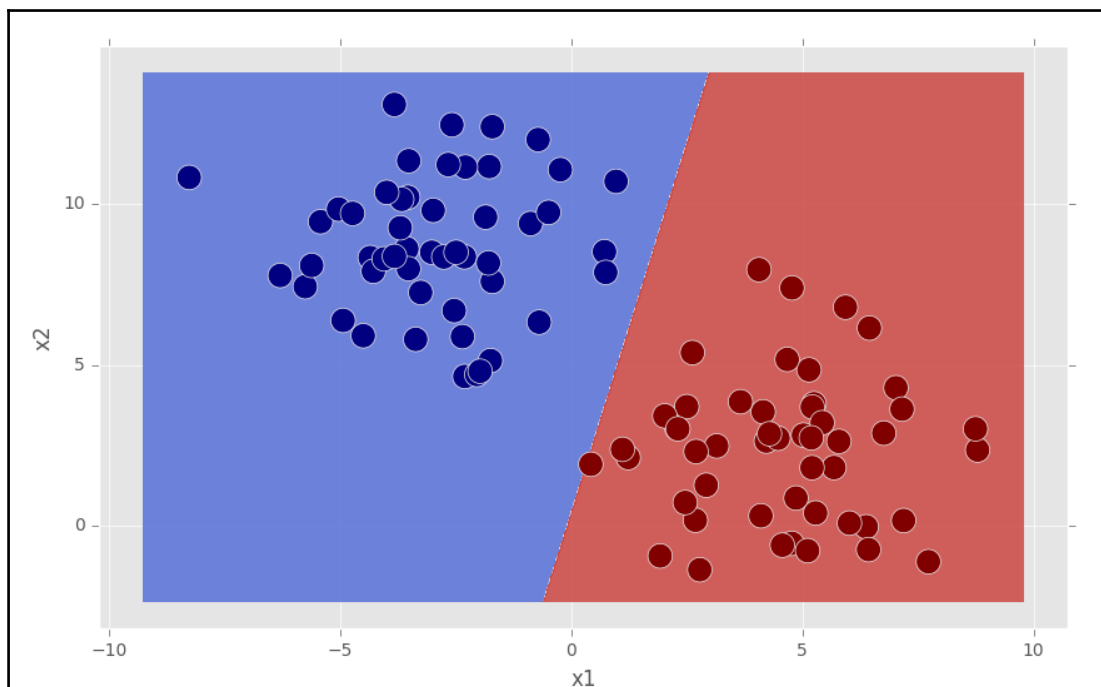
```
In [10]: def plot_decision_boundary(classifier, X_test, y_test):
...     # create a mesh to plot in
...     h = 0.02 # step size in mesh
...     x_min, x_max = X_test[:, 0].min() - 1, X_test[:, 0].max() + 1
...     y_min, y_max = X_test[:, 1].min() - 1, X_test[:, 1].max() + 1
...     xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
```

```
...             np.arange(y_min, y_max, h))
...
...     X_hypo = np.c_[xx.ravel().astype(np.float32),
...                   yy.ravel().astype(np.float32)]
...     zz = classifier.predict(X_hypo)
...     zz = zz.reshape(xx.shape)
...
...     plt.contourf(xx, yy, zz, cmap=plt.cm.coolwarm, alpha=0.8)
...     plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=200)
```

We can plot the decision landscape by passing the perceptron object (`p`), the data (`x`), and the corresponding target labels (`y`):

```
In [11]: plot_decision_boundary(p, X, y)
```

This will produce the following plot:



Linear decision boundary that separates the two classes

Of course, this problem was rather simple, even for a simple linear classifier. Even more so, you might have noticed that we didn't split the data into training and test sets. If we had data that wasn't linearly separable, the story might be a little different.

Applying the perceptron to data that is not linearly separable

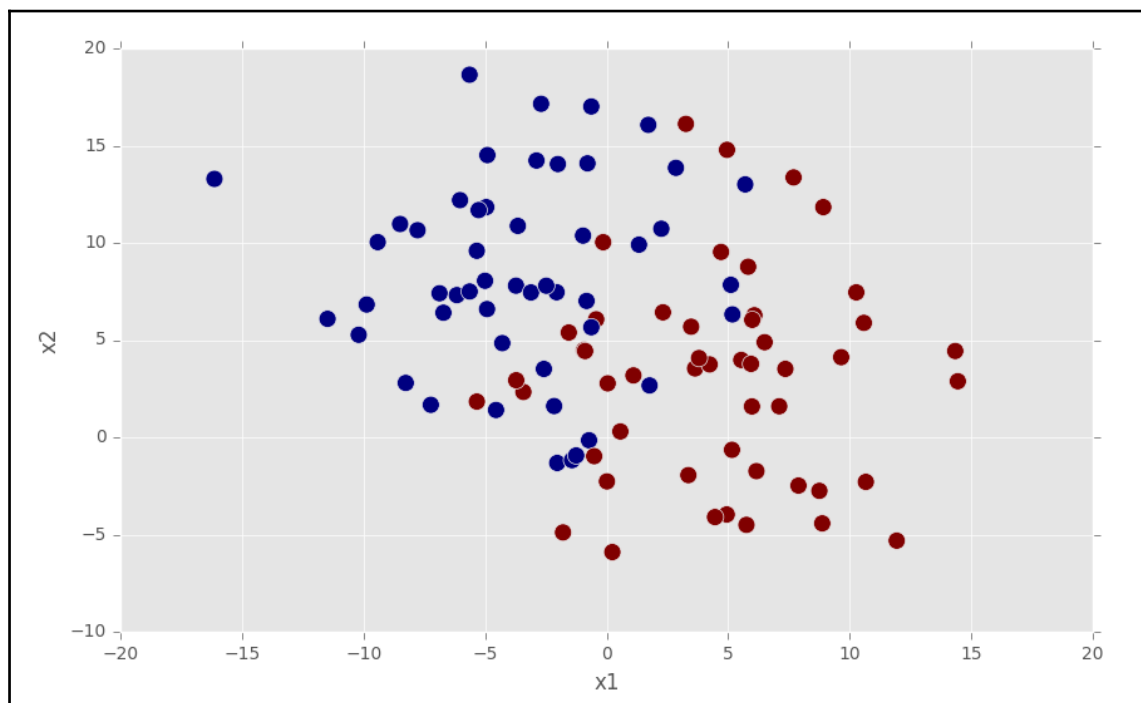
Since the perceptron is a linear classifier, you can imagine that it would have trouble trying to classify data that is not linearly separable. We can test this by increasing the spread (`cluster_std`) of the two blobs in our toy dataset so that the two blobs start overlapping:

```
In [12]: X, y = make_blobs(n_samples=100, centers=2,  
...     cluster_std=5.2, random_state=42)  
...     y = 2 * y - 1
```

We can plot the dataset again using Matplotlib's `scatter` function:

```
In [13]: plt.scatter(X[:, 0], X[:, 1], s=100, c=y);  
...     plt.xlabel('x1')  
...     plt.ylabel('x2')
```

As is evident in the following figure, this data is no longer linearly separable because there is no straight line that perfectly separates the two blobs:



Example data that is not linearly separable

So what would happen if we applied the perceptron classifier to this dataset?

We can find an answer to this question by repeating the preceding steps:

```
In [14]: p = Perceptron(lr=0.1, n_iter=10)
...      p.fit(X, y)
```

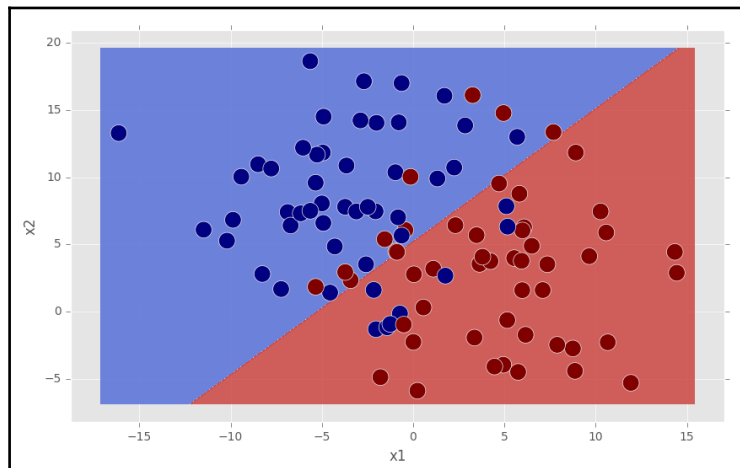
Then we find an accuracy score of 81%:

```
In [15]: accuracy_score(p.predict(X), y)
Out[15]: 0.81000000000000005
```

In order to find out which data points were misclassified, we can again visualize the decision landscape using our helper function:

```
In [16]: plot_decision_boundary(p, X, y)
...      plt.xlabel('x1')
...      plt.ylabel('x2')
```

The following figure makes the limitations of the perceptron classifier evident. Being a linear classifier, it tried to separate the data using a straight line but ultimately failed:

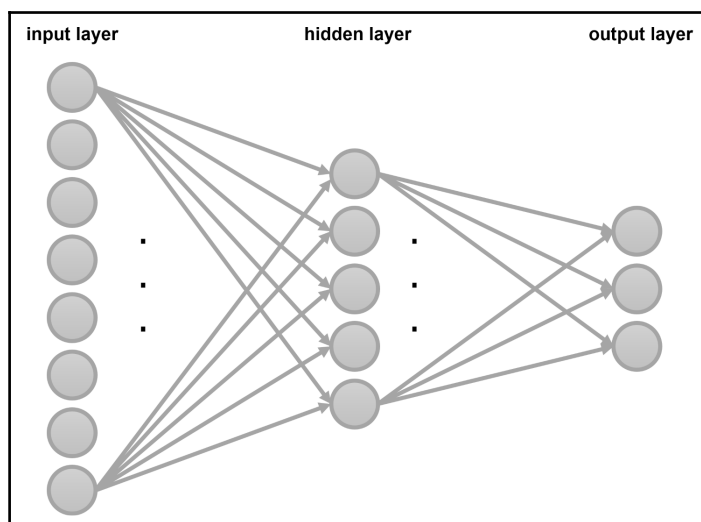


The perceptron's linear decision boundary can no longer separate the data

Fortunately, there are ways to make the perceptron more powerful and ultimately create nonlinear decision boundaries.

Understanding multilayer perceptrons

In order to create nonlinear decision boundaries, we can combine multiple perceptrons to form a larger network. This is also known as a **multilayer perceptron (MLP)**. MLPs usually consist of at least three layers, where the first layer has a node (or neuron) for every input feature of the dataset, and the last layer has a node for every class label. The layer in between is called the **hidden layer**. An example of this **feedforward neural network** architecture is shown in the following figure:



Example of a three-layer perceptron

In this network, every circle is an artificial neuron (or essentially a perceptron), and the output of one artificial neuron might serve as input to the next artificial neuron, much like how real biological neurons are wired up in the brain. By placing perceptrons side by side, we get a single one-layer neural network. Analogously, by stacking one one-layer neural network upon the other, we get a multilayer neural network—an MLP.



A neural network that contains loops and/or backward connections is called a **recurrent neural network**.

One remarkable property of MLPs is that they can represent any mathematical function if you make the network big enough. This is also known as the **universal approximation property**. For example, a one hidden-layer MLP (such as the one shown in the preceding figure) is able to exactly represent:

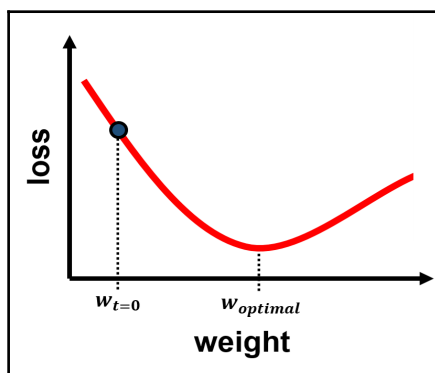
- Any boolean function (such as AND, OR, NOT, NAND, and so on)
- Any bounded continuous function (such as the sine or cosine functions)

Even better, if you add another layer, an MLP can approximate any arbitrary function. You can see why these neural networks are so powerful—if you just give them enough neurons and enough layers, they can basically learn any input-output function!

Understanding gradient descent

When we talked about the perceptron earlier in this chapter, we identified three essential ingredients needed for training: training data, a cost function, and a learning rule. Whereas the learning rule worked great for a single perceptron, unfortunately, it did not generalize to MLPs, so people had to come up with a more general rule.

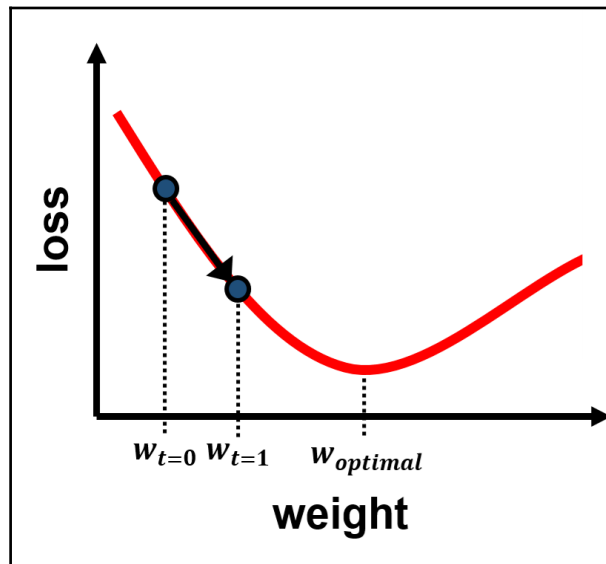
If you think about how we measure the success of a classifier, we usually do so with the help of a cost function. A typical example is the number of misclassifications of the network or the mean squared error. This function (also known as a **loss function**) usually depends on the parameters we are trying to tweak. In neural networks, these parameters are the weight coefficients. Let's assume a simple neural net has a single weight to tweak, w . Then we can visualize the cost as a function of the weight:



Example loss function

At the beginning of training, at time zero, we may start out way on the left of this graph ($w_{t=0}$). But from the graph, we know that there would be a better value for w , namely $w_{optimal}$, which would minimize the cost function. Smallest cost means lowest error, so it should be our highest goal to reach $w_{optimal}$ through learning.

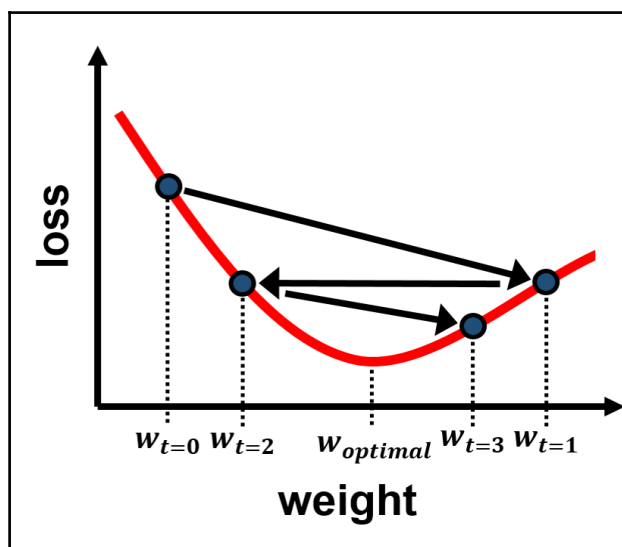
This is exactly what **gradient descent** does. You can think of the gradient as a vector that points up the hill. In gradient descent, we are trying to walk opposite of the gradient, effectively walking down the hill, from the peaks down to the valley:



Gradient descent

Once you reach the valley, the gradient goes to zero, and that completes the training.

There are several ways to reach the valley — we could approach from the left, or we could approach from the right. The starting point of our descent is determined by the **initial weight values**. Furthermore, we have to be careful not to take too large a step, otherwise we might miss the valley:



Consequences of choosing too large a step size in gradient descent

Hence, in **stochastic gradient descent** (sometimes also called **iterative** or **on-line gradient descent**), the goal is to take small steps but to take them as often as possible. The effective step size is determined by the **learning rate** of the algorithm. Specifically, we would perform the following procedure over and over:

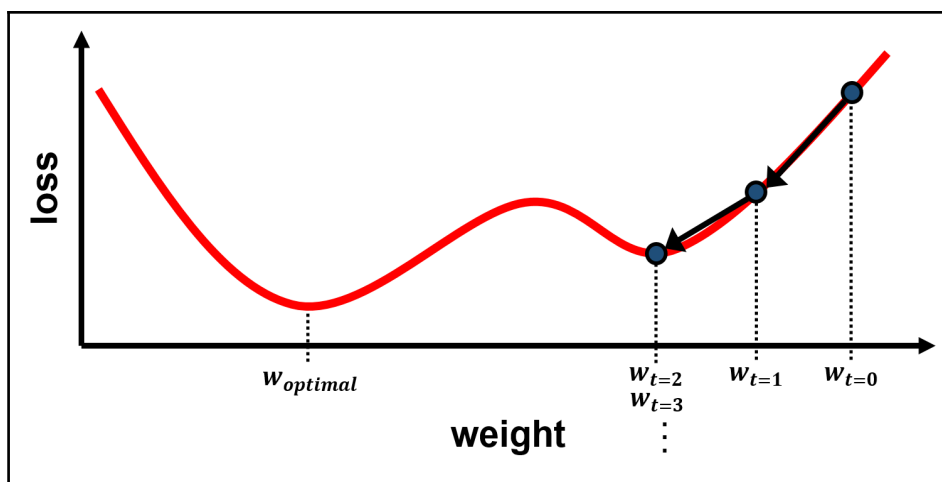
1. Present a small number of training samples to the network (called the **batch size**).
2. On this small batch of data, calculate the gradient of the cost function.
3. Update the weight coefficients by taking a small step in the opposite direction of the gradient, towards the valley.
4. Repeat steps 1-3 until the weight cost no longer goes down. This is an indication that we have reached the valley.



A compromise between batch gradient descent and stochastic gradient descent is the so-called **mini-batch learning**. Mini-batch learning can be understood as applying batch gradient descent to smaller subsets of the training data, for example, 50 samples at a time.

Can you think of an example where this procedure might fail?

One scenario that comes to mind is where the cost function has multiple valleys, some deeper than others, as shown in the following figure:



Getting stuck in a local minimum

If we start on the left, we should arrive at the same valley as before—no problem. But, if our starting point is all the way to the right, we might encounter another valley on the way. Gradient descent will lead us straight down to the valley, but it will not have any means to climb out of it.



This is also known as **getting stuck in a local minimum**. Researchers have come up with different ways to try and avoid this issue, one of them being to add noise to the process.

There is one piece left in the puzzle. Given our current weight coefficients, how do we know the slope of the cost function?

Training multi-layer perceptrons with backpropagation

This is where **backpropagation** comes in, which is an algorithm for estimating the gradient of the cost function in neural networks. Some might say that it is basically a fancy word for the chain rule, which is a means to calculate the **partial derivative** of functions that depend on more than one variable. Nonetheless, it is a method that helped bring the field of artificial neural networks back to life, so we should be thankful for that.

Understanding backpropagation involves quite a bit of calculus, so I will only give you an intuition here.

Let's remind ourselves that the cost function, and therefore its gradient, depends on the difference between the true output (y_i) and the current output ($\hat{y}_i$) for every data sample i . If we choose to define the cost function as the mean squared error, the corresponding equation would look like this:

$$E = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2$$

Here, the sum is over all data samples i . Of course, the output of a neuron ($\hat{y}$) depends on its input (z), which, in turn, depends on input features (x_j) and weights (w_j):

$$\hat{y} = \phi(z)$$

$$z = \sum_j x_j w_j$$

Therefore, in order to calculate the slope of the cost function as shown in the preceding figures, we have to calculate the derivative of E with respect to some weight, w_j . This calculation can involve quite a bit of mathematics because E depends on $\hat{y}$, which depends on z , which depends on w_j . However, thanks to the chain rule, we know that we can compartmentalize this calculation in the following way:

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w_j}$$

Here, the term of the left-hand side is the partial derivative of E with respect to w_j , which is what we are trying to calculate. If we had more neurons in the network, the equation would not end at w_j but might include some additional terms describing how the error flows through different hidden layers. This is also where backpropagation gets its name from; similar to the activity flowing from the input layer via the hidden layers to the output layer, the error gradient flows backwards through the network. In other words, the error gradient in the hidden layer depends on the error gradient in the output layer, and so on. You can see it is as if the error **back-propagated** through the network.

Well, *this is still kind of complicated*, you might say. The good news is, however, that the only term to depend on w_j in the preceding equation is the rightmost term, making the math at least a little easier. But you are right, this is still kind of complicated, and there's no reason for us to dig deeper here.



For more information on backpropagation, I can warmly recommend the following book: Ian Goodfellow, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning (Adaptive Computation and Machine Learning series)*. MIT Press, ISBN 978-026203561-3.

After all, what we are really interested in is how all this stuff works in practice.

Implementing a multilayer perceptron in OpenCV

Implementing an MLP in OpenCV uses the same syntax that we have seen at least a dozen times before. In order to see how an MLP compares to a single perceptron, we will operate on the same toy data as before:

```
In [1]: from sklearn.datasets.samples_generator import make_blobs
...     X_raw, y_raw = make_blobs(n_samples=100, centers=2,
...                               cluster_std=5.2, random_state=42)
```

Preprocessing the data

However, since we are working with OpenCV, this time, we want to make sure the input matrix is made up of 32-bit floating point numbers, otherwise the code will break:

```
In [2]: import numpy as np
...      X = X_raw.astype(np.float32)
```

Furthermore, we need to think back to [Chapter 4, Representing Data and Engineering and Features](#), and remember how to represent categorical variables. We need to find a way to represent target labels, not as integers but with a one-hot encoding. The easiest way to achieve this is by using scikit-learn's preprocessing module:

```
In [3]: from sklearn.preprocessing import OneHotEncoder
...      enc = OneHotEncoder(sparse=False, dtype=np.float32)
...      y = enc.fit_transform(y_raw.reshape(-1, 1))
```

Creating an MLP classifier in OpenCV

The syntax to create an MLP in OpenCV is the same as for all the other classifiers:

```
In [4]: import cv2
...      mlp = cv2.ml.ANN_MLP_create()
```

However, now we need to specify how many layers we want in the network and how many neurons there are per layer. We do this with a list of integers, which specify the number of neurons in each layer. Since the data matrix `X` has two features, the first layer should also have two neurons in it (`n_input`). Since the output has two different values, the last layer should also have two neurons in it (`n_output`). In between these two layers, we can put as many hidden layers with as many neurons as we want. Let's choose a single hidden layer with an arbitrary number of eight neurons in it (`n_hidden`):

```
In [5]: n_input = 2
...      n_hidden = 8
...      n_output = 2
...      mlp.setLayerSizes(np.array([n_input, n_hidden, n_output]))
```

Customizing the MLP classifier

Before we move on to training the classifier, we can customize the MLP classifier via a number of optional settings:

- `mlp.setActivationFunction`: This defines the activation function to be used for every neuron in the network
- `mlp.setTrainMethod`: This defines a suitable training method
- `mlp.setTermCriteria`: This sets the termination criteria of the training phase

Whereas our home-brewed perceptron classifier used a linear activation function, OpenCV provides two additional options:

- `cv2.ml.ANN_MLP_IDENTITY`: This is the linear activation function, $f(x) = x$.
- `cv2.ml.ANN_MLP_SIGMOID_SYM`: This is the symmetrical sigmoid function (also known as **hyperbolic tangent**), $f(x) = \beta (1 - \exp(-\alpha x)) / (1 + \exp(-\alpha x))$. Whereas α controls the slope of the function, β defines the upper and lower bounds of the output.
- `cv2.ml.ANN_GAUSSIAN`: This is the Gaussian function (also known as the **bell curve**), $f(x) = \beta \exp(-\alpha x^2)$. Whereas α controls the slope of the function, β defines the upper bound of the output.



Careful, OpenCV is being really confusing here! What they call a symmetrical sigmoid function is in reality the hyperbolic tangent. In other software, a sigmoid function usually has responses in the range $[0, 1]$, whereas the hyperbolic tangent has responses in the range $[-1, 1]$. Even worse, if you use the symmetrical sigmoid function with its default parameters, the output will range in $[-1.7159, 1.7159]$!

In this example, we will use a proper sigmoid function that squashes the input values into the range $[0, 1]$. We do this by choosing $\alpha = 2.5$ and $\beta = 1.0$:

```
In [6]: mlp.setActivationFunction(cv2.ml.ANN_MLP_SIGMOID_SYM, 2.5, 1.0)
```

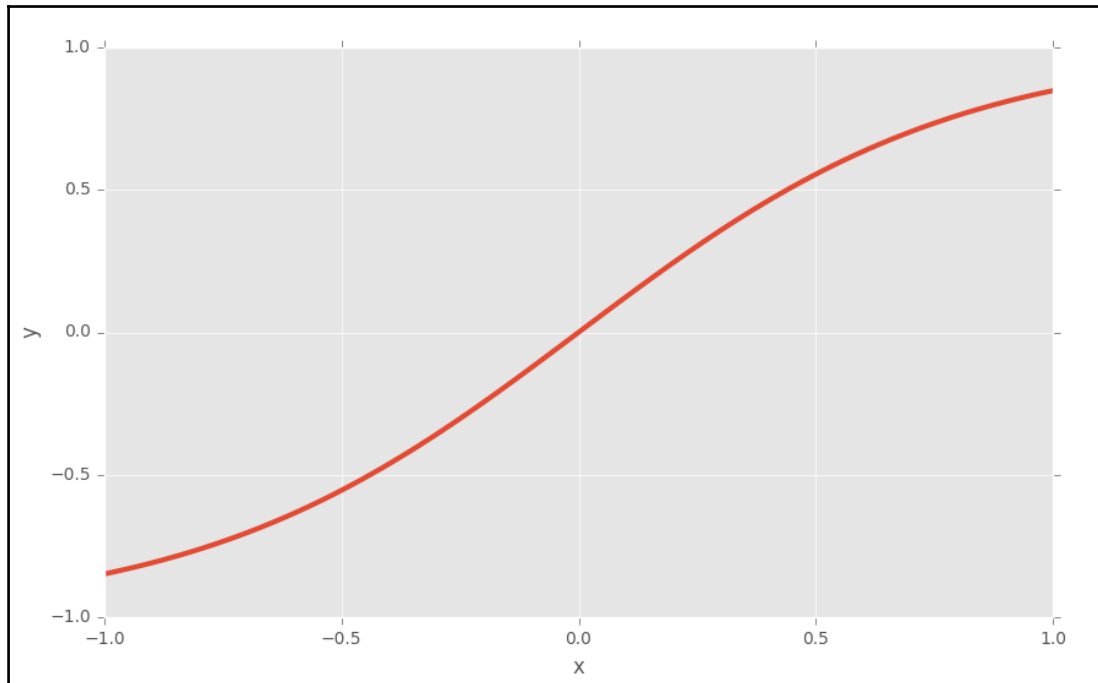
If you are curious what this activation function looks like, we can take a short excursion with Matplotlib:

```
In [7]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     plt.style.use('ggplot')
```

In order to see what the activation function looks like, we can create a NumPy array that densely samples x values in the range $[-1, 1]$, and then calculate the corresponding y values using the preceding mathematical expression:

```
In [8]: alpha = 2.5
...     beta = 1.0
...     x_sig = np.linspace(-1.0, 1.0, 100)
...     y_sig = beta * (1.0 - np.exp(-alpha * x_sig))
...     y_sig /= (1 + np.exp(-alpha * x_sig))
...     plt.plot(x_sig, y_sig, linewidth=3);
...     plt.xlabel('x')
...     plt.ylabel('y')
```

As you can see in the following figure, this will create a nice squashing function whose output values will lie in the range $[-1, 1]$:



Example of the symmetrical sigmoid function for $\alpha = 2.5$ and $\beta = 1.0$

As mentioned in the preceding part, a training method can be set via `mlp.setTrainMethod`. The following methods are available:

- `cv2.ml.ANN_MLP_BACKPROP`: This is the backpropagation algorithm we talked about previously. You can set additional scaling factors via `mlp.setBackpropMomentumScale` and `mlp.setBackpropWeightScale`.
- `cv2.ml.ANN_MLP_RPROP`: This is the Rprop algorithm, which is short for resilient backpropagation. We won't have time to discuss this algorithm, but you can set additional parameters of this algorithm via `mlp.setRpropDW0`, `mlp.setRpropDWMMax`, `mlp.setRpropDWMin`, `mlp.setRpropDWMinus`, and `mlp.setRpropDWPlus`.

In this example, we will choose backpropagation:

```
In [9]: mlp.setTrainMethod(cv2.ml.ANN_MLP_BACKPROP)
```

Lastly, we can specify the criteria that must be met for training to end via `mlp.setTermCriteria`. This works the same for every classifier in OpenCV and is closely tied to the underlying C++ functionality. We first tell OpenCV which criteria we are going to specify (for example, the maximum number of iterations). Then we specify the value for this criterion. All values must be delivered in a tuple.

Hence, in order to run our MLP classifier until we either reach 300 iterations or the error does not increase anymore beyond some small range of values, we would write the following:

```
In [10]: term_mode = cv2.TERM_CRITERIA_MAX_ITER + cv2.TERM_CRITERIA_EPS
...      term_max_iter = 300
...      term_eps = 0.01
...      mlp.setTermCriteria((term_mode, term_max_iter, term_eps))
```

Then we are ready to train the classifier!

Training and testing the MLP classifier

This is the easy part. Training the MLP classifier is the same as with all other classifiers:

```
In [11]: mlp.train(X, cv2.ml.ROW_SAMPLE, y)
Out[11]: True
```

The same goes for predicting target labels:

```
In [12]: _, y_hat = mlp.predict(X)
```

The easiest way to measure accuracy is by using scikit-learn's helper function:

```
In [13]: from sklearn.metrics import accuracy_score
...       accuracy_score(y_hat.round(), y)
Out[13]: 0.8399999999999997
```

It looks like we were able to increase our performance from 81% with a single perceptron to 84% with an MLP consisting of ten hidden-layer neurons and two output neurons. In order to see what changed, we can look at the decision boundary one more time:

```
In [14]: def plot_decision_boundary(classifier, X_test, y_test):
...       # create a mesh to plot in
...       h = 0.02 # step size in mesh
...       x_min, x_max = X_test[:, 0].min() - 1, X_test[:, 0].max() + 1
...       y_min, y_max = X_test[:, 1].min() - 1, X_test[:, 1].max() + 1
...       xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
...                               np.arange(y_min, y_max, h))
...
...       X_hypo = np.c_[xx.ravel().astype(np.float32),
...                       yy.ravel().astype(np.float32)]
...       _, zz = classifier.predict(X_hypo)
```

However, there is a problem right here, in that `zz` is now a one-hot encoded matrix. In order to transform the one-hot encoding into a number that corresponds to the class label (zero or one), we can use NumPy's `argmax` function:

```
...       zz = np.argmax(zz, axis=1)
```

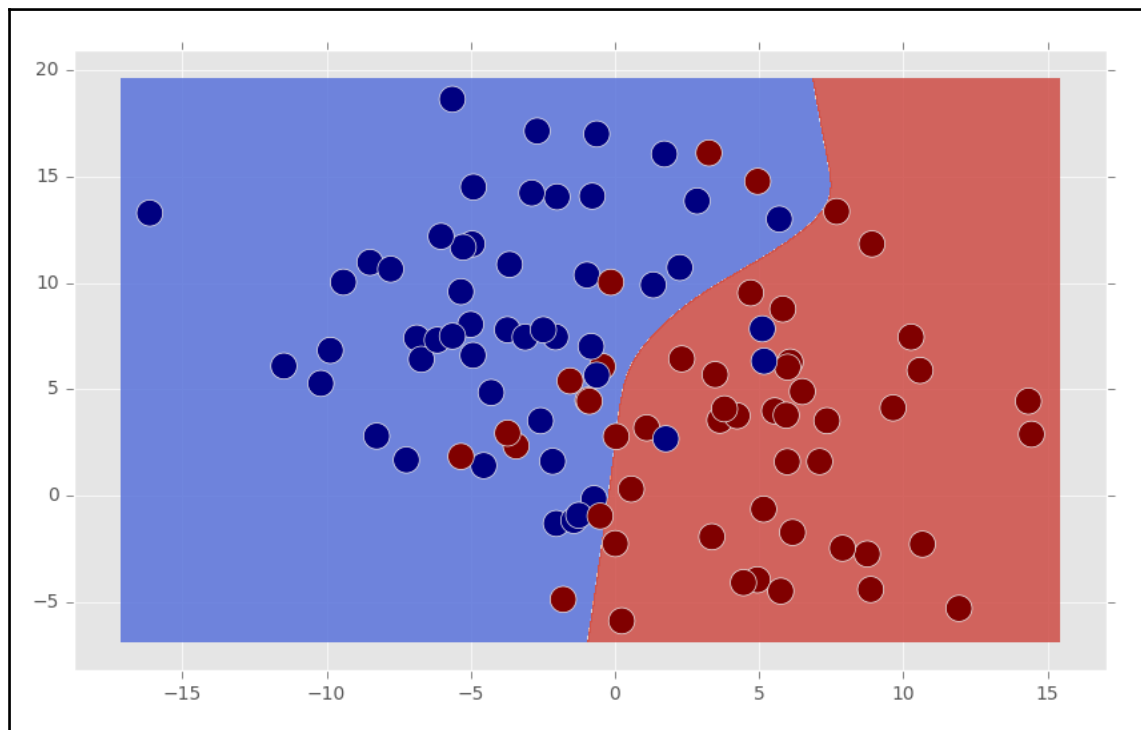
Then the rest stays the same:

```
...       zz = zz.reshape(xx.shape)
...       plt.contourf(xx, yy, zz, cmap=plt.cm.coolwarm, alpha=0.8)
...       plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=200)
```

Then we can call the function like this:

```
In [15]: plot_decision_boundary(mlp, X, y_raw)
```

The output looks like this:



Decision boundary of an MLP with one hidden layer

And voila! The decision boundary is no longer a straight line. That being said, you might have expected a more drastic performance increase. But nobody said we have to stop here! There are at least two different things we can try from here on out:

- We can add more neurons to the hidden layer. You can do this by replacing `n_hidden` on line 6 with a larger value and running the code again. Generally speaking, the more neurons you put in the network, the more powerful the MLP will be.
- We can add more hidden layers. It turns out that this is where neural nets really get their power from.

Hence, this is where I should tell you about deep learning.

Getting acquainted with deep learning

Back when deep learning didn't have a fancy name yet, it was called artificial neural networks. So you already know a great deal about it! This was a respected field in itself, but after the days of Rosenblatt's perceptron, many researchers and machine learning practitioners slowly began to lose interest in the field since no one had a good solution for training a neural network with multiple layers.

Eventually, interest in neural networks was rekindled in 1986 when *David Rumelhart*, *Geoffrey Hinton*, and *Ronald Williams* were involved in the (re)discovery and popularization of the aforementioned backpropagation algorithm. However, it was not until recently that computers became powerful enough so they could actually execute the backpropagation algorithm on large-scale networks, leading to a surge in deep learning research.



You can find more information on the history and origin of deep learning in the following scientific article: Wang & Raj (2017), *On the Origin of Deep Learning*, arXiv:1702.07800.

With the current popularity of deep learning in both industry and academia, we are fortunate enough to have a whole range of open-source deep learning frameworks at our disposal:

- Google Brain's **TensorFlow** (<http://www.tensorflow.org>): This is a machine learning library that describes computations as dataflow graphs. To date, this is one of the most commonly used deep learning libraries. Hence, it is also evolving quickly, so you might have to check back often for software updates. TensorFlow provides a whole range of user interfaces, including Python, C++, and Java interface.
- Microsoft Research's **Cognitive Toolkit** (CNTK, <https://www.microsoft.com/en-us/research/product/cognitive-toolkit>): This is a deep learning framework that describes neural networks as a series of computational steps via a directed graph.
- UC Berkeley's **Caffe** (<http://caffe.berkeleyvision.org>): This is a pure deep learning framework written in C++, with an additional Python interface.

- University of Montreal's **Theano** (<http://deeplearning.net/software/theano>): This is a numerical computation library compiled to run efficiently on CPU and GPU architectures. Theano is more than a machine learning library; it can express any computation using a specialized computer algebra system. Hence, it is best suited for people who wish to write their machine learning algorithms from scratch.
- **Torch** (<http://www.torch.ch>): This is a scientific computing framework based on the Lua programming language. Like Theano, Torch is more than a machine learning library, but it is heavily used for deep learning by companies such as Facebook, IBM, and Yandex.

Finally, there is also **Keras**, which we will be using in the following sections. In contrast to the preceding frameworks, Keras understands itself as an interface rather than an end-to-end deep learning framework. It allows you to specify deep neural nets using an easy-to-understand API, which can then be run on backends, such as TensorFlow, CNTK, or Theano.

Getting acquainted with Keras

The core data structure of Keras is a model, which is similar to OpenCV's classifier object, except it focuses on neural networks only. The simplest type of model is the `Sequential` model, which arranges the different layers of the neural net in a linear stack, just like we did for the MLP in OpenCV:

```
In [1]: from keras.models import Sequential
...     model = Sequential()
Out[1]: Using TensorFlow backend.
```

Then different layers can be added to the model one by one. In Keras, layers do not just contain neurons, they also perform a function. Some core layer types include the following:

- **Dense**: This is a densely connected layer. This is exactly what we used when we designed our MLP: a layer of neurons that is connected to every neuron in the previous layer.
- **Activation**: This applies an activation function to an output. Keras provides a whole range of activation functions, including OpenCV's identify function (`linear`), the hyperbolic tangent (`tanh`), a sigmoidal squashing function (`sigmoid`), a softmax function (`softmax`), and many more.
- **Reshape**: This reshapes an output to a certain shape.

There are other layers that calculate arithmetic or geometric operations on their inputs:

- **Convolutional layers:** These layers allow you to specify a kernel with which the input layer is convolved. This allows you to perform operations such as a Sobel filter or apply a Gaussian kernel in 1D, 2D, or even 3D.
- **Pooling layers:** These layers perform a max pooling operation on their input, where the output neuron's activity is given by the maximally active input neuron.

Some other layers that are popular in deep learning are as follows:

- **Dropout:** This layer randomly sets a fraction of input units to zero at each update. This is a way to inject noise into the training process, making it more robust.
- **Embedding:** This layer encodes categorical data, similar to some functions from scikit-learn's preprocessing module.
- **GaussianNoise:** This layer applies additive zero-centered Gaussian noise. This is another way of injecting noise into the training process, making it more robust.

A perceptron similar to the preceding one could thus be implemented using a Dense layer that has two inputs and one output. Staying true to our earlier example, we will initialize the weights to zero and use the hyperbolic tangent as an activation function:

```
In [2]: from keras.layers import Dense
...      model.add(Dense(1, activation='tanh',
input_dim=2,
...                               kernel_initializer='zeros'))
```

Finally, we want to specify the training method. Keras provides a number of optimizers, including the following:

- **stochastic gradient descent ('sgd'):** This is what we have discussed before
- **root mean square propagation ('RMSprop'):** This is a method in which the learning rate is adapted for each of the parameters
- **adaptive moment estimation ('Adam'):** This is an update to root mean square propagation and many more

In addition, Keras also provides a number of different loss functions:

- **mean squared error** ('mean_squared_error'): This is what was discussed before
- **hinge loss** ('hinge'): This is a maximum-margin classifier often used with SVM, as discussed in Chapter 6, *Detecting Pedestrians with Support Vector Machines*, and many more

You can see that there's a whole plethora of parameters to be specified and methods to choose from. To stay true to our aforementioned perceptron implementation, we will choose stochastic gradient descent as an optimizer, the mean squared error as a cost function, and accuracy as a scoring function:

```
In [3]: model.compile(optimizer='sgd',  
...                   loss='mean_squared_error',  
...                   metrics=['accuracy'])
```

In order to compare the performance of the Keras implementation to our home-brewed version, we will apply the classifier to the same dataset:

```
In [4]: from sklearn.datasets.samples_generator import make_blobs  
...     X, y = make_blobs(n_samples=100, centers=2,  
...     cluster_std=2.2, random_state=42)
```

Finally, a Keras model is fit to the data with a very familiar syntax. Here, we can also choose how many iterations to train for (*epochs*), how many samples to present before we calculate the error gradient (*batch_size*), whether to shuffle the dataset (*shuffle*), and whether to output progress updates (*verbose*):

```
In [5]: model.fit(X, y, epochs=400, batch_size=100, shuffle=False,  
...             verbose=0)
```

After the training completes, we can evaluate the classifier as follows:

```
In [6]: model.evaluate(X, y)  
Out[6]: 32/100 [=====>.....] - ETA: 0s  
[0.040941802412271501, 1.0]
```

Here, the first reported value is the mean squared error, whereas the second value denotes accuracy. This means that the final mean squared error was 0.04, and we had 100% accuracy. Way better than our own implementation!



You can find more information on Keras, source code documentation, and a number of tutorials at <http://keras.io>.

With these tools in hand, we are now ready to approach a real-world dataset!

Classifying handwritten digits

In the previous section, we covered a lot of the theory around neural networks, which can be a little overwhelming if you are new to this topic. In this section, we will classify handwritten digits from the popular **MNIST dataset**, which has been constructed by Yann LeCun and colleagues and serves as a popular benchmark dataset for machine learning algorithms.

We will train two different networks on it:

- a multilayer perceptron using OpenCV
- a deep neural net using Keras

Loading the MNIST dataset

The easiest way to obtain the MNIST dataset is using Keras:

```
In [1]: from keras.datasets import mnist
...     (X_train, y_train), (X_test, y_test) = mnist.load_data()
Out[1]: Using TensorFlow backend.
        Downloading data from
        https://s3.amazonaws.com/img-datasets/mnist.npz
```

This will download the data from the Amazon Cloud (might take a while depending on your internet connection) and automatically split the data into training and test sets.



MNIST provides its own predefined train-test split. This way, it is easier to compare performance of different classifiers because they will all use the same data for training and the same data for testing.

This data comes in a format that we are already familiar with:

```
In [2]: X_train.shape, y_train.shape
Out[2]: ((60000, 28, 28), (60000,))
```

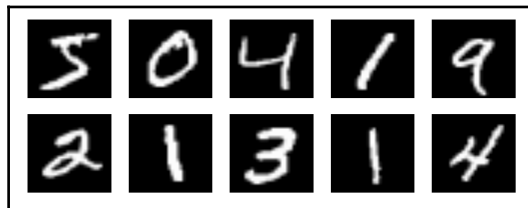
We should take note that the labels come as integer values between zero and nine (corresponding to the digits 0-9):

```
In [3]: import numpy as np
...      np.unique(y_train)
Out[3]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], dtype=uint8)
```

We can have a look at some example digits:

```
In [4]: import matplotlib.pyplot as plt
...      %matplotlib inline
In [5]: for i in range(10):
...       plt.subplot(2, 5, i + 1)
...       plt.imshow(X_train[i, :, :], cmap='gray')
...       plt.axis('off')
```

The digits look like this:



Example digits from the MNIST database

In fact, the MNIST dataset is the successor to the NIST digits dataset provided by scikit-learn that we used before (`sklearn.datasets.load_digits` (refer to Chapter 2, *Working with Data in OpenCV and Python*). Some notable differences are as follows:

- MNIST images are significantly larger (28x28 pixels) than NIST images (8x8 pixels), thus paying more attention to fine details, such as distortions and individual differences between images of the same digit
- The MNIST dataset is much larger than the NIST dataset, providing 60,000 training and 10,000 test samples (as compared to a total of 5,620 NIST images)

Preprocessing the MNIST dataset

As we learned in Chapter 4, *Representing Data and Engineering Features*, there are a number of preprocessing steps we might like to apply here:

- **Centering:** It is important that all the digits are centered in the image. For example, take a look at all the example images of the digit 1 in the preceding figure, which are all made of an almost-vertical strike. If the images were misaligned, the strike could lie anywhere in the image, making it hard for the neural network to find commonalities in the training samples. Fortunately, images in MNIST are already centered.
- **Scaling:** The same is true for scaling the digits so that they all have the same size. This way, the location of strikes, curves, and loops are important. Otherwise, the neural network might easily confuse eights and zeros because they are all made of one or two closed loops. Fortunately, images in MNIST are already scaled.
- **Representing categorical features:** It is important that target labels be one-hot encoded so that we can have ten neurons in the output layer corresponding to the ten different classes 0-9. This step we still have to perform ourselves.

The easiest way to transform `y_train` and `y_test` is by the one-hot encoder from scikit-learn:

```
In [6]: from sklearn.preprocessing import OneHotEncoder
...      enc = OneHotEncoder(sparse=False, dtype=np.float32)
...      y_train_pre = enc.fit_transform(y_train.reshape(-1, 1))
```

This will transform the labels of the training set from a `<n_samples x 1>` vector with integers 0-9 into a `<n_samples x 10>` matrix with floating point numbers 0.0 or 1.0. Analogously, we can transform `y_test` using the same procedure:

```
In [7]: y_test_pre = enc.fit_transform(y_test.reshape(-1, 1))
```

In addition, we need to preprocess `X_train` and `X_test` for the purpose of working with OpenCV. Currently, `X_train` and `X_test` are 3-D matrices `<n_samples x 28 x 28>` with integer values between 0 and 255. Preferably, we want a 2-D matrix `<n_samples x n_features>` with floating point numbers, where `n_features` is 784:

```
In [8]: X_train_pre = X_train_pre.reshape((X_train.shape[0], -1))
...      X_train_pre = X_train_pre.astype(np.float32) / 255.0
...      X_test_pre = X_test_pre.reshape((X_test.shape[0], -1))
...      X_test_pre = X_test_pre.astype(np.float32) / 255.0
```

Then we are ready to train the network.

Training an MLP using OpenCV

We can set up and train an MLP in OpenCV with the following recipe:

1. Instantiate a new MLP object:

```
In [9]: import cv2
...      mlp = cv2.ml.ANN_MLP_create()
```

2. Specify the size of every layer in the network. We are free to add as many layers as we want, but we need to make sure that the first layer has the same number of neurons as input features (784 in our case), and that the last layer has the same number of neurons as class labels (10 in our case):

```
In [10]: mlp.setLayerSizes(np.array([784, 512, 512, 10]))
```

3. Specify an activation function. Here we use the sigmoidal activation function from before:

```
In [11]: mlp.setActivationFunction(cv2.ml.ANN_MLP_SIGMOID_SYM,
...                                2.5, 1.0)
```

4. Specify the training method. Here we use the backpropagation algorithm described above. We also need to make sure that we choose a small enough learning rate. Since we have on the order of 10^5 training samples, it is a good idea to set the learning rate to at most 10^{-5} :

```
In [12]: mlp.setTrainMethod(cv2.ml.ANN_MLP_BACKPROP)
...      mlp.setBackpropWeightScale(0.00001)
```

5. Specify the termination criteria. Here we use the same criteria as above: to run training for ten iterations (`term_max_iter`) or until the error does no longer increase significantly (`term_eps`):

```
In [13]: term_mode = (cv2.TERM_CRITERIA_MAX_ITER +
...                    cv2.TERM_CRITERIA_EPS)
...      term_max_iter = 10
...      term_eps = 0.01
...      mlp.setTermCriteria((term_mode, term_max_iter,
...                             term_eps))
```

5. Train the network on the training set (`X_train_pre`):

```
In [14]: mlp.train(X_train_pre, cv2.ml.ROW_SAMPLE, y_train_pre)
Out[14]: True
```



Before you call `mlp.train`, here is a word of caution: this might take several hours to run, depending on your computer setup! For comparison, it took just under an hour on my own laptop. We are now dealing with a real-world dataset of 60,000 samples: if we run 100 training epochs, we have to compute 6 million gradients! So beware.

When the training completes, we can calculate the accuracy score on the training set to see how far we got:

```
In [15]: _, y_hat_train = mlp.predict(X_train_pre)
In [16]: from sklearn.metrics import accuracy_score
...       accuracy_score(y_hat_train.round(), y_train_pre)
Out[16]: 0.92976666666666663
```

But, of course, what really counts is the accuracy score we get on the held-out test data:

```
In [17]: _, y_hat_test = mlp.predict(X_test_pre)
...       accuracy_score(y_hat_test.round(), y_test_pre)
Out[17]: 0.91690000000000005
```

91.7% accuracy is not bad at all if you ask me! The first thing you should try is to change the layer sizes in `In [10]` above, and see how the test score changes. As you add more neurons to the network, you should see the training score increase—and with it, hopefully, the test score. However, having N neurons in a single layer is not the same as having them spread out over several layers! Can you confirm this observation?

Training a deep neural net using Keras

Although we achieved a formidable score with the MLP above, our result does not hold up to state-of-the-art results. Currently the best result has close to 99.8% accuracy—better than human performance! This is why nowadays, the task of classifying handwritten digits is largely regarded as solved.

To get closer to the state-of-the-art results, we need to use state-of-the-art techniques. Thus, we return to Keras.

Preprocessing the MNIST dataset

To make sure we get the same result every time we run the experiment, we will pick a random seed for NumPy's random number generator. This way, shuffling the training samples from the MNIST dataset will always result in the same order:

```
In [1]: import numpy as np
...      np.random.seed(1337)
```

Keras provides a loading function similar to `train_test_split` from scikit-learn's `model_selection` module. Its syntax might look strangely familiar to you:

```
In [2]: from keras.datasets import mnist
...      (X_train, y_train), (X_test, y_test) = mnist.load_data()
```



In contrast to other datasets we have encountered so far, MNIST comes with a pre-defined train-test split. This allows the dataset to be used as a **benchmark**, as the test score reported by different algorithms will always apply to the same test samples.

The neural nets in Keras act on the feature matrix slightly differently than the standard OpenCV and scikit-learn estimators. Whereas the rows of a feature matrix in Keras still correspond to the number of samples (`X_train.shape[0]` in the code below), we can preserve the two-dimensional nature of the input images by adding more dimensions to the feature matrix:

```
In [3]: img_rows, img_cols = 28, 28
...      X_train = X_train.reshape(X_train.shape[0], img_rows, img_cols, 1)
...      X_test = X_test.reshape(X_test.shape[0], img_rows, img_cols, 1)
...      input_shape = (img_rows, img_cols, 1)
```

Here we have reshaped the feature matrix into a four-dimensional matrix with dimensions `n_features x 28 x 28 x 1`. We also need to make sure we operate on 32-bit floating point numbers between `[0, 1]`, rather than unsigned integers in `[0, 255]`:

```
...      X_train = X_train.astype('float32') / 255.0
...      X_test = X_test.astype('float32') / 255.0
```

Then, we can one-hot encode the training labels like we did before. This will make sure each category of target labels can be assigned to a neuron in the output layer. We could do this with scikit-learn's `preprocessing`, but in this case it is easier to use Keras' own utility function:

```
In [4]: from keras.utils import np_utils
...      n_classes = 10
...      Y_train = np_utils.to_categorical(y_train, n_classes)
```

```
...     Y_test = np_utils.to_categorical(y_test, n_classes)
```

Creating a convolutional neural network

Once we have preprocessed the data, it is time to define the actual model. Here, we will once again rely on the `Sequential` model to define a feedforward neural network:

```
In [5]: from keras.model import Sequential
...     model = Sequential()
```

However, this time, we will be smarter about the individual layers. We will design our neural network around a **convolutional layer**, where the kernel is a 3 x 3 pixel two-dimensional convolution:

```
In [6]: from keras.layers import Convolution2D
...     n_filters = 32
...     kernel_size = (3, 3)
...     model.add(Convolution2D(n_filters, kernel_size[0], kernel_size[1],
...                             border_mode='valid',
...                             input_shape=input_shape))
```



A two-dimensional convolutional layer operates akin to image filtering in OpenCV, where each image in the input data is convolved with a small two-dimensional kernel. In Keras, we can specify the **kernel size** and the **stride**. For more information on convolutional layers, I can recommend the following book: John Hearty (2016). *Advanced Machine Learning with Python*. Packt Publishing, ISBN 978-1-78439-863-7.

After that, we will use a linear rectified unit as an activation function:

```
In [7]: from keras.layers import Activation
...     model.add(Activation('relu'))
```

In a deep convolutional neural net, we can have as many layers as we want. A popular version of this structure applied to MNIST involves performing the convolution and rectification twice:

```
In [8]: model.add(Convolution2D(n_filters, kernel_size[0], kernel_size[1]))
...     model.add(Activation('relu'))
```

Finally, we will pool the activations and add a Dropout layer:

```
In [9]: from keras.layers import MaxPooling2D, Dropout
...     pool_size = (2, 2)
...     model.add(MaxPooling2D(pool_size=pool_size))
...     model.add(Dropout(0.25))
```

Then we will flatten the model and finally pass it through a `softmax` function to arrive at the output layer:

```
In [10]: from keras.layers import Flatten, Dense
...       model.add(Flatten())
...       model.add(Dense(128))
...       model.add(Activation('relu'))
...       model.add(Dropout(0.5))
...       model.add(Dense(n_classes))
...       model.add(Activation('softmax'))
```

Here, we will use the cross-entropy loss and the **Adadelta** algorithm:

```
In [11]: model.compile(loss='categorical_crossentropy',
...                    optimizer='adadelta',
...                    metrics=['accuracy'])
```

Fitting the model

We fit the model like we do with all other classifiers (caution, this might take a while):

```
In [12]: model.fit(X_train, Y_train, batch_size=128, nb_epoch=12,
...               verbose=1, validation_data=(X_test, Y_test))
```

After training completes, we can evaluate the classifier:

```
In [13]: model.evaluate(X_test, Y_test, verbose=0)
Out[13]: 0.9925
```

And we achieve 99.25% accuracy! Worlds apart from the MLP classifier we implemented before. And this is just one way to do things. As you can see, neural networks provide a plethora of tuning parameters, and it is not at all clear which ones will lead to the best performance.

Summary

In this chapter, we added a whole bunch of skills to our list as a machine learning practitioner. Not only did we cover the basics of artificial neural networks, including the perceptron and multilayer perceptrons, we also got our hands on some advanced deep learning software. We learned how to build a simple perceptron from scratch and how to build state-of-the-art networks using Keras. Furthermore, we learnt about all the details of neural nets: activation functions, loss functions, layer types, and training methods. All in all, this was probably the densest chapter yet.

Now that you know about most of the essential supervised learners, it is time to talk about how to combine different algorithms into a more powerful one. Thus, in the next chapter, we will talk about how to build ensemble classifiers.

10

Combining Different Algorithms into an Ensemble

So far, we have looked at a number of interesting machine learning algorithms, from classic methods such as linear regression to more advanced techniques such as deep neural networks. At various points, we pointed out that every algorithm has its own strengths and weaknesses—and we took note of how to spot and overcome these weaknesses.

However, wouldn't it be great if we could simply stack together a bunch of average classifiers to form a much stronger **ensemble** of classifiers?

In this chapter, we will do just that. Ensemble methods are techniques that bind multiple different models together in order to solve a shared problem. Their use has become a common practice in competitive machine learning—making use of an ensemble typically improves an individual classifier's performance by a small percentage.

Among these techniques are the so-called **bagging methods**, where the vote of multiple classifiers is averaged to make a final decision, and **boosting methods**, where one classifier is trying to rectify the errors made by another. One of these methods is known as **random forest**, which is a combination of multiple decision trees. In addition, you might already be familiar with **Adaptive Boosting** (also known as **AdaBoost**), a powerful boosting technique and popular feature of OpenCV.

As we progress through the chapter, we want to find answers to the following questions:

- How can we combine multiple models to form an ensemble classifier?
- What are random forests and what do they have to do with decision trees?
- What's the different between bagging and boosting?

Let's jump right in!

Understanding ensemble methods

The goal of ensemble methods is to combine the predictions of several individual estimators built with a given learning algorithm in order to solve a shared problem. Typically, an ensemble consists of two major components:

- a set of models
- a set of **decision rules** that govern how the results of these models are combined into a single output



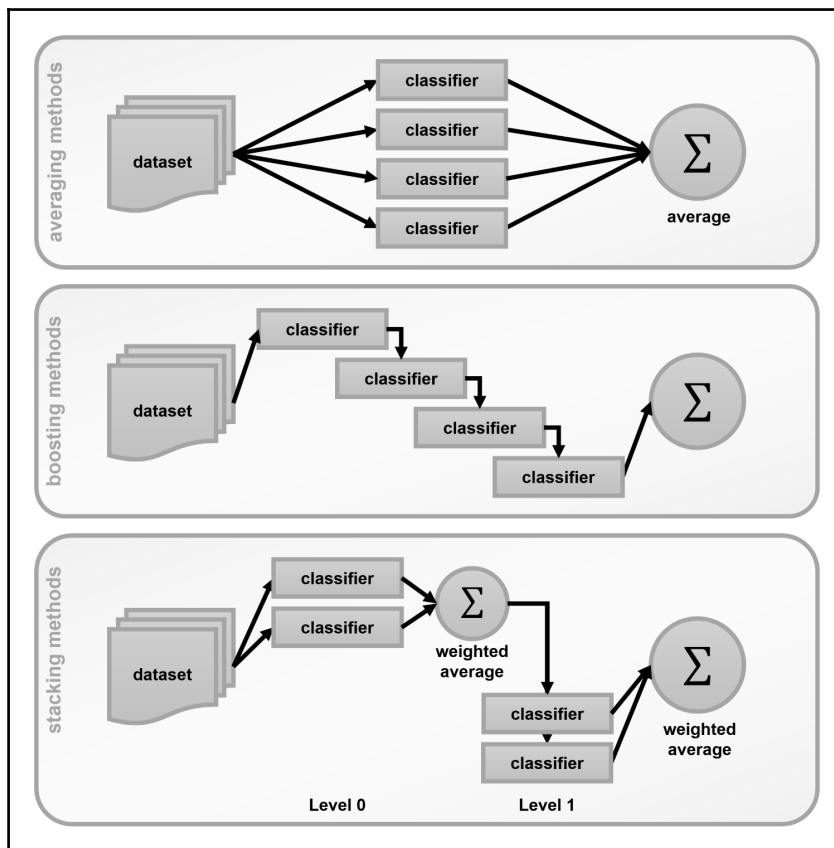
The idea behind ensemble methods has much to do with the **wisdom of the crowd** concept. Rather than the opinion of a single expert, we consider the collective opinion of a group of individuals. In the context of machine learning, these individuals would be **classifiers** or **regressors**. The idea is that if we just ask a large enough number of classifiers, one of them ought to get it right.

A consequence of this procedure is that we get a multitude of opinions about any given problem. So how do we know which classifier is right?

This is why we need a decision rule. Perhaps we consider everybody's opinion of equal importance, or perhaps we would want to weight somebody's opinion based on their expert status. Depending on the nature of our decision rule, ensemble methods can be categorized as follows:

- **Averaging methods:** They develop models in parallel and then use averaging or voting techniques to come up with a combined estimator. This is as close to democracy as ensemble methods can get.
- **Boosting methods:** They involve building models in sequence, where each added model aims to improve the score of the combined estimator. This is akin to debugging the code of your intern or reading the report of your undergraduate student: they are all bound to make errors, and the job of every subsequent expert laying eyes on the topic is to figure out the special cases where the preceding expert got it wrong.
- **Stacking methods:** Also known as **blending methods**, they use the weighted output of multiple classifiers as inputs to the next layer in the model. This is akin to having expert groups who pass on their decision to the next expert group.

The same three categories of ensemble methods are illustrated in the following figure:



Ensemble methods

Ensemble classifiers afford us the opportunity to choose the best elements of each of the proposed solutions in order to come up with a single, final result. As a result, we typically get much more accurate and robust results.



When building an ensemble classifier, we should be mindful that our goal is to tune the performance of the ensemble rather than of the models that comprise it. Individual model performance is thus less important than the overall performance of the ensemble.

Let's briefly discuss the three different types of ensemble methods and talk about how they can be used to classify data.

Understanding averaging ensembles

Averaging methods have a long history in machine learning and are commonly applied in fields such as molecular dynamics and audio signal processing. Such ensembles are typically seen as almost exact replicates of a given system.

An **averaging ensemble** is essentially a collection of models that train on the same dataset. Their results are then aggregated in a number of ways.

One common method involves creating multiple model configurations that take different parameter subsets as input. Techniques that take this approach are referred to collectively as bagging methods.

Bagging methods come in many different flavors. However, they typically only differ by the way they draw random subsets of the training set:

- **Pasting** methods draw random subsets of the samples without replacement
- **Bagging** methods draw random subsets of the samples with replacement
- **Random subspace** methods draw random subsets of the features but train on all data samples
- **Random patches** methods draw random subsets of both samples and features



Averaging ensembles can be used to reduce the variability of a model's performance.

In scikit-learn, bagging methods can be realized using the **meta-estimators** `BaggingClassifier` and `BaggingRegressor`. These are meta-estimators because they allow us to build an ensemble from any other base estimator.

Implementing a bagging classifier

We can, for instance, build an ensemble from a collection of 10 k -NN classifiers as follows:

```
In [1]: from sklearn.ensemble import BaggingClassifier
...      from sklearn.neighbors import KNeighborsClassifier
...      bag_knn = BaggingClassifier(KNeighborsClassifier(),
...                                  n_estimators=10)
```

The `BaggingClassifier` class provides a number of options to customize the ensemble:

- `n_estimators`: As shown in the preceding code, this specifies the number of base estimators in the ensemble.
- `max_samples`: This denotes the number (or fraction) of samples to draw from the dataset to train each base estimator. We can set `bootstrap=True` to sample with replacement (effectively implementing bagging), or we can set `bootstrap=False` to implement pasting.
- `max_features`: This denotes the number (or fraction) of features to draw from the feature matrix to train each base estimator. We can set `max_samples=1.0` and `max_features<1.0` to implement the random subspace method. Alternatively, we can set both `max_samples<1.0` and `max_features<1.0` to implement the random patches method.

This gives us the ultimate freedom to implement any kind of averaging ensemble. An ensemble can then be fit to data like any other estimator using the ensemble's `fit` method.

For example, if we wanted to implement bagging with 10 k -NN classifiers with $k=5$, where every k -NN classifier is trained on 50% of the samples in the dataset, we would modify the preceding command as follows:

```
In [2]: bag_knn = BaggingClassifier(KNeighborsClassifier(n_neighbors=5),
...                               n_estimators=10, max_samples=0.5,
...                               bootstrap=True, random_state=3)
```

In order to observe a performance boost, we have to apply the ensemble to some dataset, such as the breast cancer dataset from Chapter 5, *Using Decision Trees to Make a Medical Diagnosis*:

```
In [3]: from sklearn.datasets import load_breast_cancer
...     dataset = load_breast_cancer()
...     X = dataset.data
...     y = dataset.target
```

As usual, we should follow the best practice of splitting the data into training and test sets:

```
In [4]: from sklearn.model_selection import train_test_split
...     X_train, X_test, y_train, y_test = train_test_split(
...         X, y, random_state=3
...     )
```

Then we can train the ensemble using the `fit` method and evaluate its generalization performance using the `score` method:

```
In [5]: bag_knn.fit(X_train, y_train)
...     bag_knn.score(X_test, y_test)
Out[5]: 0.93706293706293708
```

The performance boost will become evident once we also train a single k -NN classifier on the data:

```
In [6]: knn = KNeighborsClassifier(n_neighbors=5)
...     knn.fit(X_train, y_train)
...     knn.score(X_test, y_test)
Out[6]: 0.91608391608391604
```

Without changing the underlying algorithm, we were able to improve our test score from 91.6% to 93.7% by simply letting 10 k -NN classifiers do the job instead of a single one.

You're welcome to experiment with other bagging ensembles. For example, how would you adjust the preceding code snippets to implement the random patches method?



You can find the answer in the Jupyter Notebook `notebooks/10.00-Combining-Different-Algorithms-Into-an-Ensemble.ipynb` on GitHub.

Implementing a bagging regressor

Similarly, we can use the `BaggingRegressor` class to form an ensemble of regressors.

For example, we could build an ensemble of decision trees to predict the housing prices from the Boston dataset of Chapter 3, *First Steps in Supervised Learning*. The syntax is almost identical to setting up a bagging classifier:

```
In [7]: from sklearn.ensemble import BaggingRegressor
...     from sklearn.tree import DecisionTreeRegressor
...     bag_tree = BaggingRegressor(DecisionTreeRegressor(),
...                                 max_features=0.5, n_estimators=10,
...                                 random_state=3)
```

Of course, we need to load and split the dataset like we did for the breast cancer dataset:

```
In [8]: from sklearn.datasets import load_boston
...     dataset = load_boston()
...     X = dataset.data
...     y = dataset.target
```

```
In [9]: from sklearn.model_selection import train_test_split
...     X_train, X_test, y_train, y_test = train_test_split(
...         X, y, random_state=3
...     )
```

Then we can fit the bagging regressor on `X_train` and score it on `X_test`:

```
In [10]: bag_tree.fit(X_train, y_train)
...      bag_tree.score(X_test, y_test)
Out[10]: 0.82704756225081688
```

As in the preceding example, we find a performance boost of roughly 5%, from 77.3% accuracy for a single decision tree to 82.7% accuracy.

Of course, we wouldn't just stop here. Nobody said the ensemble needs to consist of 10 individual estimators, so we are free to explore different-sized ensembles. On top of that, the `max_samples` and `max_features` parameters allow for a great deal of customization.



A more sophisticated version of bagged decision trees is called random forests, which we will talk about later in this chapter.

Understanding boosting ensembles

Another approach to building ensembles is through boosting. Boosting models use multiple individual learners in sequence to iteratively **boost** the performance of the ensemble.

Typically, the learners used in boosting are relatively simple. A good example is a decision tree with only a single node—a **decision stump**. Another example could be a simple linear regression model. The idea is not to have the strongest individual learners, quite the opposite—we want the individuals to be **weak learners**, so that we get a superior performance only when we consider a large number of individuals.

At each iteration of the procedure, the training set is adjusted so that the next classifier is applied to the data points that the preceding classifier got wrong. Over multiple iterations, the ensemble is extended with a new tree (whichever tree optimized the ensemble performance score) at each iteration.



In scikit-learn, boosting methods can be realized using the `GradientBoostingClassifier` and `GradientBoostingRegressor` objects.

Implementing a boosting classifier

For example, we can build a boosting classifier from a collection of 10 decision trees as follows:

```
In [11]: from sklearn.ensemble import GradientBoostingClassifier
...      boost_class = GradientBoostingClassifier(n_estimators=10,
...                                              random_state=3)
```

These classifiers support both binary and multiclass classification.

Similar to the `BaggingClassifier` class, the `GradientBoostingClassifier` class provides a number of options to customize the ensemble:

- `n_estimators`: This denotes the number of base estimators in the ensemble. A large number of estimators typically results in better performance.
- `loss`: This denotes the loss function (or cost function) to be optimized. Setting `loss='deviance'` implements logistic regression for classification with probabilistic outputs. Setting `loss='exponential'` actually results in AdaBoost, which we will talk about in a little bit.
- `learning_rate`: This denotes the fraction by which to shrink the contribution of each tree. There is a trade-off between `learning_rate` and `n_estimators`.
- `max_depth`: This denotes the maximum depth of the individual trees in the ensemble.
- `criterion`: This denotes the function to measure the quality of a node split.
- `min_samples_split`: This denotes the number of samples required to split an internal node.
- `max_leaf_nodes`: This denotes the maximum number of leaf nodes allowed in each individual tree and so on.

We can apply the boosted classifier to the preceding breast cancer dataset to get an idea of how this ensemble compares to a bagged classifier. But first, we need to reload the dataset:

```
In [12]: dataset = load_breast_cancer()
...      X = dataset.data
...      y = dataset.target
In [13]: X_train, X_test, y_train, y_test = train_test_split(
...      X, y, random_state=3
...      )
```

Then we find that the boosted classifier achieves 94.4% accuracy on the test set—a little under 1% better than the preceding bagged classifier:

```
In [14]: boost_class.fit(X_train, y_train)
...      boost_class.score(X_test, y_test)
Out[14]: 0.94405594405594406
```

We would expect an even better score if we increased the number of base estimators from 10 to 100. In addition, we might want to play around with the learning rate and the depths of the trees.

Implementing a boosting regressor

Implementing a boosted regressor follows the same syntax as the boosted classifier:

```
In [15]: from sklearn.ensemble import GradientBoostingRegressor
...      boost_reg = GradientBoostingRegressor(n_estimators=10,
...                                             random_state=3)
```

We have seen earlier that a single decision tree can achieve 79.3% accuracy on the Boston dataset. A bagged decision tree classifier made of 10 individual regression trees achieved 82.7% accuracy. But how does a boosted regressor compare?

Let's reload the Boston dataset and split it into training and test sets. We want to make sure we use the same value for `random_state` so that we end up training and testing on the same subsets of the data:

```
In [16]: dataset = load_boston()
...      X = dataset.data
...      y = dataset.target
In [17]: X_train, X_test, y_train, y_test = train_test_split(
...      X, y, random_state=3
...      )
```

As it turns out, the boosted decision tree ensemble actually performs worse than the previous code:

```
In [18]: boost_reg.fit(X_train, y_train)
...      boost_reg.score(X_test, y_test)
Out[18]: 0.71991199075668488
```

This result might be confusing at first. After all, we used 10 times more classifiers than we did for the single decision tree. Why would our numbers get worse?

You can see, this is a good example of an expert classifier being smarter than a group of weak learners. One possible solution is to make the ensemble larger. In fact, it is customary to use on the order of 100 weak learners in a boosted ensemble:

```
In [19]: boost_reg = GradientBoostingRegressor(n_estimators=100)
```

Then, when we retrain the ensemble on the Boston dataset, we get a test score of 89.8%:

```
In [20]: boost_reg.fit(X_train, y_train)
...      boost_reg.score(X_test, y_test)
Out[20]: 0.89984081091774459
```

What happens when you increase the number to `n_estimators=500`? There's a lot more we could do by playing with the optional parameters.

As you can see, boosting is a powerful procedure that allows you to get massive performance improvements by combining a large number of relatively simple learners.



A specific implementation of boosted decision trees is the AdaBoost algorithm, which we will talk about later in this chapter.

Understanding stacking ensembles

All the ensemble methods we have seen so far share a common design philosophy: to fit multiple individual classifiers to the data and incorporate their predictions with the help of some simple decision rule (such as averaging or boosting) into a final prediction.

Stacking ensembles, on the other hand, build ensembles with hierarchies. Here, individual learners are organized into multiple layers where the output of one layer of learners is used as training data for a model at the next layer. This way, it is possible to successfully blend hundreds of different models.

Unfortunately, we won't have the time to discuss stacking ensembles in detail.

However, these models can be very powerful, as seen, for example, in the Netflix Prize competition. The most successful models in the competition used to include stacking ensembles that not just combined different layers but extracted from each layer the parameters that were most effective and used them as meta-features for training the next layer.



You can learn more about the Netflix Prize at <http://www.netflixprize.com>. One of the stacking models that was used to achieve the second prize in the competition is also described in the following scientific article: Joseph Sill, Gabor Takacs, Lester Mackey, David Lin (2009). Feature-Weighted Linear Stacking, <https://arxiv.org/abs/0911.0460>.

Let's now move on to discuss some popular bagging and boosting techniques, such as random forests and Adaptive Boosting.

Combining decision trees into a random forest

A popular variation of bagged decision trees are the so-called random forests. These are essentially a collection of decision trees, where each tree is slightly different from the others. In contrast to bagged decision trees, each tree in a random forest is trained on a slightly different subset of data features.

Although a single tree of unlimited depth might do a relatively good job of predicting the data, it is also prone to overfitting. The idea behind random forests is to build a large number of trees, each of them trained on a random subset of data samples and features. Because of the randomness of the procedure, each tree in the forest will overfit the data in a slightly different way. The effect of overfitting can then be reduced by averaging the predictions of the individual trees.

Understanding the shortcomings of decision trees

The effect of overfitting the dataset, which a decision tree often falls victim of is best demonstrated through a simple example.

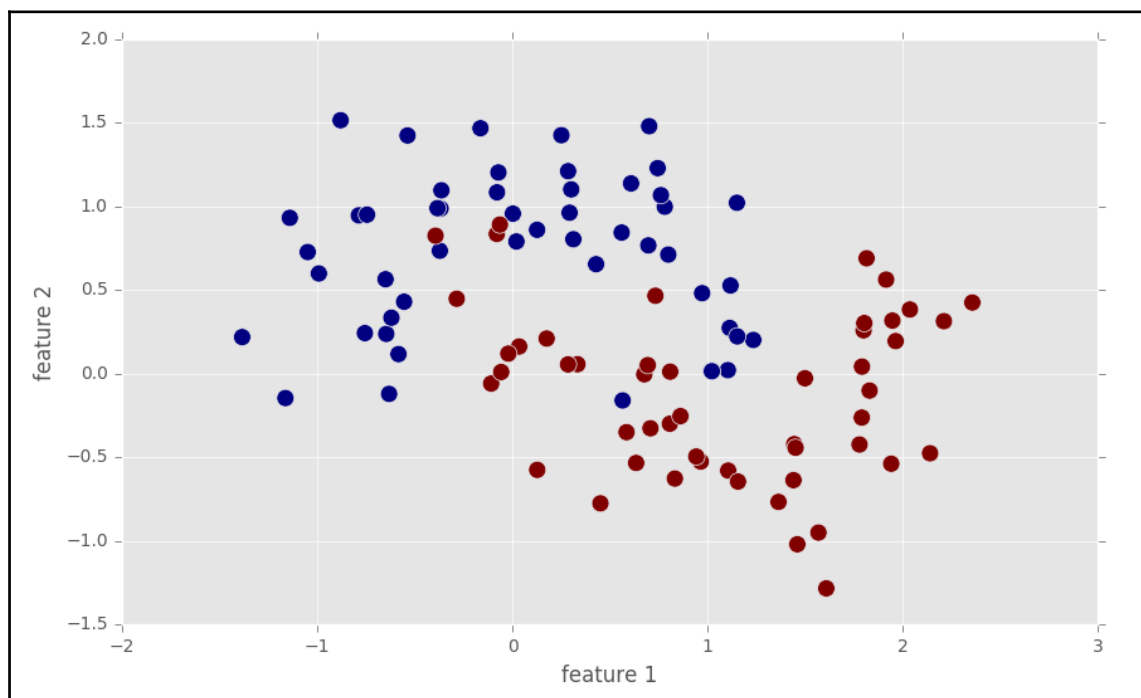
For this, we will return to the `make_moons` function from `scikit-learn`'s `datasets` module, which we previously used in Chapter 8, *Discovering Hidden Structures with Unsupervised Learning* to organize data into two interleaving half circles. Here, we choose to generate 100 data samples belonging to two half circles, in combination with some Gaussian noise with standard deviation 0.25:

```
In [1]: from sklearn.datasets import make_moons
...     X, y = make_moons(n_samples=100, noise=0.25,
...                       random_state=100)
```

We can visualize this data using Matplotlib and the `scatter` function:

```
In [2]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     plt.style.use('ggplot')
In [3]: plt.scatter(X[:, 0], X[:, 1], s=100, c=y)
...     plt.xlabel('feature 1')
...     plt.ylabel('feature 2');
```

The resulting plot has the data colored according to the class they belong to:



Example of a `make_moons` dataset with added noise

Because of all the noise we added, the two half moons might not be apparent at first glance. That's a perfect scenario for our current intentions, which is to show that decision trees are tempted to overlook the general arrangement of data points (that is, the fact that they are organized in half circles) and instead focus on the noise in the data.

To illustrate this point, we first need to split the data into training and test sets. We choose a comfortable 75-25 split (by not specifying `train_size`), as we have done a number of times before:

```
In [4]: from sklearn.model_selection import train_test_split
...      X_train, X_test, y_train, y_test = train_test_split(
...          X, y, random_state=100
...      )
```

Now let's have some fun. What we want to do is to study how the decision boundary of a decision tree changes as we make it deeper and deeper.

For this, we will bring back the `plot_decision_boundary` function from Chapter 6, *Detecting Pedestrians with Support Vector Machines* among others:

```
In [5]: import numpy as np
...      def plot_decision_boundary(classifier, X_test, y_test):
```

The function itself involves a number of processing steps:

1. Create a dense 2-D grid of data points between $(x_{\min}, y_{\min})$, and $(x_{\max}, y_{\max})$:

```
...      h = 0.02 # step size in mesh
...      x_min = X_test[:, 0].min() - 1,
...      x_max = X_test[:, 0].max() + 1
...      y_min = X_test[:, 1].min() - 1
...      y_max = X_test[:, 1].max() + 1
...      xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
...                           np.arange(y_min, y_max, h))
...      
```

2. Classify every point on the grid:

```
...      X_hypo = np.c_[xx.ravel().astype(np.float32),
...                     yy.ravel().astype(np.float32)]
...      zz = classifier.predict(X_hypo)
...      zz = zz.reshape(xx.shape)
```

3. If the `predict` method returns a tuple, we are dealing with an OpenCV classifier, else we are dealing with scikit-learn:

```
...      if isinstance(ret, tuple):
...          zz = ret[1]
...      else:
...          zz = ret
...      zz = zz.reshape(xx.shape)
```

4. Use the predicted target labels (`zz`) to color the decision landscape:

```
...         plt.contourf(xx, yy, zz, cmap=plt.cm.coolwarm,
                        alpha=0.8)
```

5. Draw all the data points from the test set on top of the colored landscape:

```
...         plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test,
                        s=200)
```

Then we can code up a for loop, where at each iteration, we fit a tree of a different depth:

```
In [6]: from sklearn.tree import DecisionTreeClassifier
...     for depth in range(1, 9):
```

In order to plot the decision landscape for every created decision tree, we need to create a plot with a total of eight subplots. The iterator `depth` can then serve as an index for the current subplot:

```
...         plt.subplot(2, 4, depth)
```

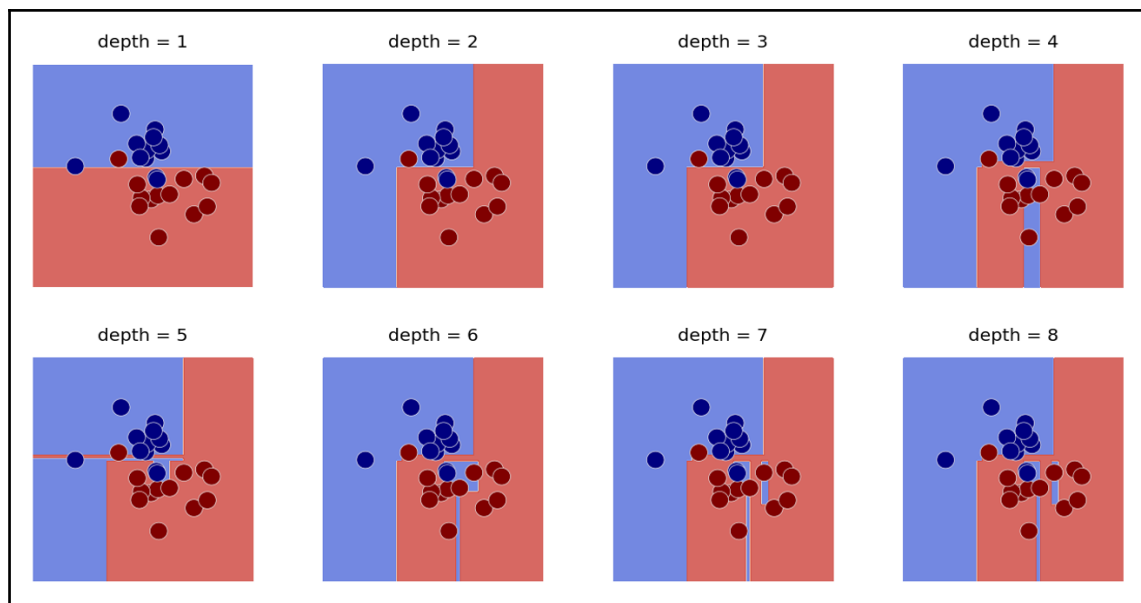
After fitting the tree to the data, we pass the predicted test labels to the `plot_decision_boundary` function:

```
...         tree = DecisionTreeClassifier(max_depth=depth)
...         tree.fit(X_train, y_train)
...         y_test = tree.predict(X_test)
...         plot_decision_boundary(tree, X_test, y_test)
```

For the sake of clarity, we also turn off the axes and decorate each plot with a title:

```
...         plt.axis('off')
...         plt.title('depth = %d' % depth)
```

This produces the following plot:



Classification results for decision trees with different depths

Recall the procedure of a decision tree: at every node in the tree, the algorithm picks a feature to split, either along the x -axis or along the y -axis. For example, a tree of `depth=1` has only one node and, thus, makes only one decision. This particular tree (upper row, leftmost panel) chose to split the second feature right down the middle, leading to a horizontal decision boundary that separates the decision landscape into an upper and lower half.

Similarly, a decision tree of `depth=2` will go a step further. After making the first decision of splitting the second feature in layer #1, the algorithm ends up with two additional nodes in layer #2. This particular tree (upper row, second panel from the left) chose to split the first feature in layer #2, leading to two additional vertical lines.

As we continue to build deeper and deeper trees, we notice something strange: the deeper the tree, the more likely it is to get strangely shaped decision regions, such as the tall and skinny patches in the rightmost panel of the lower row. It's clear that these patches are more a result of the noise in the data rather than some characteristic of the underlying data distribution. This is an indication that most of the trees are overfitting the data. After all, we know for a fact that the data is organized into two half circles! As such, the trees with `depth=3` or `depth=5` are probably closest to the real data distribution.

There are at least two different ways to make a decision tree less powerful:

- Train the tree only on a subset of the data
- Train the tree only on a subset of the features

Random forests do just that. In addition, they repeat the experiment many times by building an ensemble of trees, each of which is trained on a randomly chosen subset of data samples and/or features.

Random forests are a powerful ensemble classifier that come with several advantages:

- Both training and evaluation are fast. Not only are the underlying decision trees relatively simple data structures, but every tree in the forest is independent, making it easy to train them in parallel.
- Multiple trees allow for probabilistic classification. Later in this chapter, we will talk about voting procedures, which allow an ensemble to predict the probability with which a data point belongs to a particular class.

That being said, one of the disadvantages of random forests is that they might be hard to interpret, and the same goes for individual trees that are relatively deep. Although cumbersome, it is still possible to investigate the decision path within a particular tree. You might want to check out [Chapter 5, Using Decision Trees to Make a Medical Diagnosis](#) for a refresher.

Implementing our first random forest

In OpenCV, random forests can be built using the `RTrees_create` function from the `ml` module:

```
In [7]: import cv2
...     rtree = cv2.ml.RTrees_create()
```

The tree object provides a number of options, the most important of which are the following:

- `setMaxDepth`: This sets the maximum possible depth of each tree in the ensemble. The actual obtained depth may be smaller if other termination criteria are met first.
- `setMinSampleCount`: This sets the minimum number of samples that a node can contain for it to get split.

- `setMaxCategories`: This sets the maximum number of categories allowed. Setting the number of categories to a smaller value than the actual number of classes in the data performs subset estimation.
- `setTermCriteria`: This sets the termination criteria of the algorithm. This is also where you set the number of trees in the forest.



Although we might have hoped for a `setNumTrees` method to set the number of trees in the forest (kind of the most important parameter of them all, no?), we instead need to rely on the `setTermCriteria` method. Confusingly, the number of trees is conflated with `cv2.TERM_CRITERIA_MAX_ITER`, which is usually reserved for the number of iterations that an algorithm is run for, not for the number of estimators in an ensemble.

We can specify the number of trees in the forest by passing an integer `n_trees` to the `setTermCriteria` method. Here, we also want to tell the algorithm to quit once the score does not increase by at least `eps` from one iteration to the next:

```
In [8]: n_trees = 10
...      eps = 0.01
...      criteria = (cv2.TERM_CRITERIA_MAX_ITER + cv2.TERM_CRITERIA_EPS,
...                  n_trees, eps)
...      rtree.setTermCriteria(criteria)
```

Then we are ready to train the classifier on the data from the preceding code:

```
In [9]: rtree.train(X_train.astype(np.float32), cv2.ml.ROW_SAMPLE,
...                 y_train);
```

The test labels can be predicted with the `predict` method:

```
In [10]: _, y_hat = rtree.predict(X_test.astype(np.float32))
```

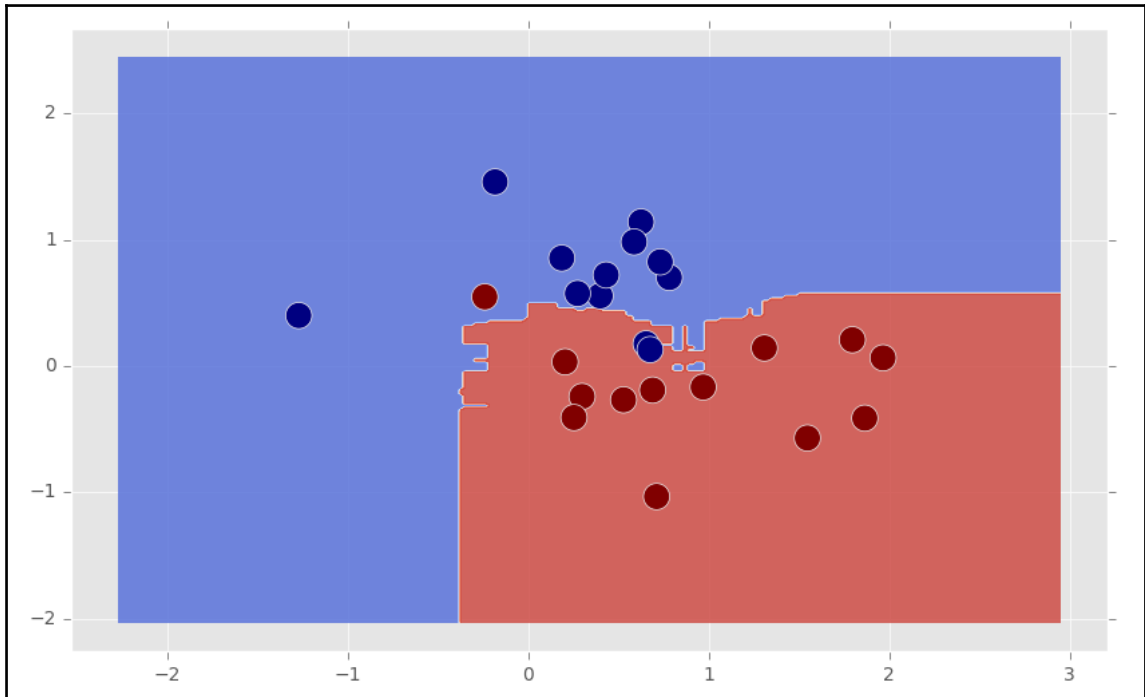
Using scikit-learn's `accuracy_score`, we can evaluate the model on the test set:

```
In [11]: from sklearn.metrics import accuracy_score
...       accuracy_score(y_test, y_hat)
Out[11]: 0.8399999999999997
```

After training, we can pass the predicted labels to the `plot_decision_boundary` function:

```
In [12]: plot_decision_boundary(rtree, X_test, y_test)
```

This will produce the following plot:



Decision landscape of a random forest classifier

Implementing a random forest with scikit-learn

Alternatively, we can implement random forests using scikit-learn:

```
In [13]: from sklearn.ensemble import RandomForestClassifier
...      forest = RandomForestClassifier(n_estimators=10, random_state=200)
```

Here, we have a number of options to customize the ensemble:

- `n_estimators`: This specifies the number of trees in the forest.
- `criterion`: This specifies the node splitting criterion. Setting `criterion='gini'` implements the Gini impurity, whereas setting `criterion='entropy'` implements information gain.

- `max_features`: This specifies the number (or fraction) of features to consider at each node split.
- `max_depth`: This specifies the maximum depth of each tree.
- `min_samples`: This specifies the minimum number of samples required to split a node.

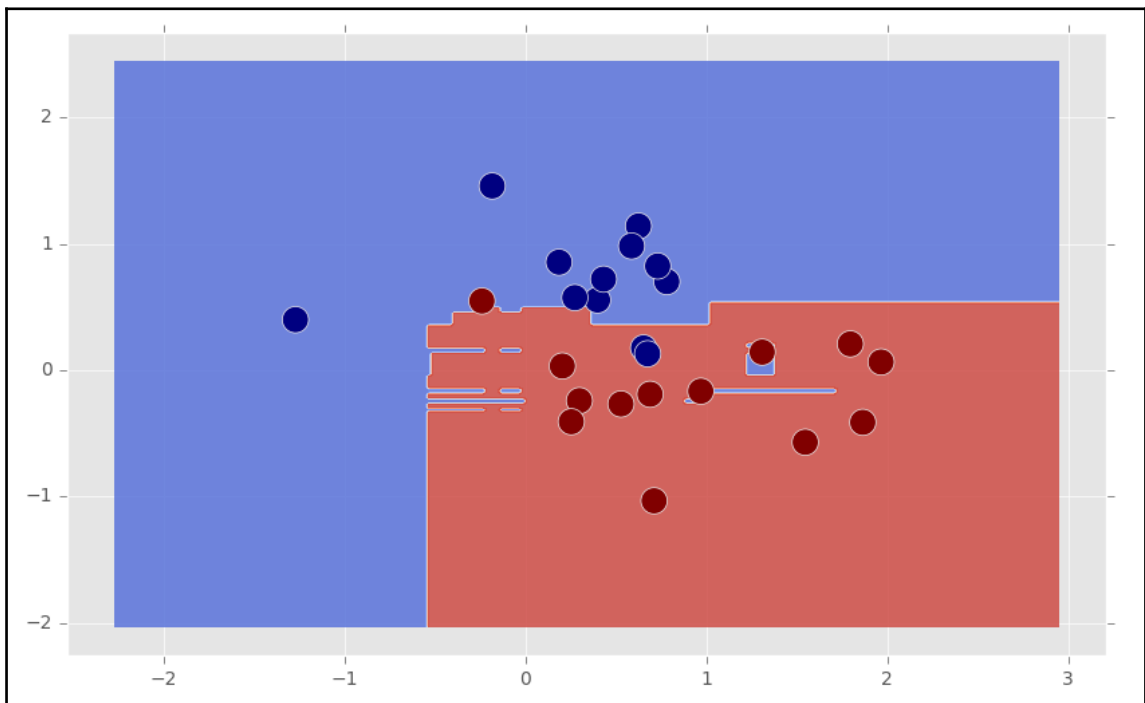
We can then fit the random forest to the data and score it like any other estimator:

```
In [14]: forest.fit(X_train, y_train)
...      forest.score(X_test, y_test)
Out[14]: 0.8399999999999997
```

This gives roughly the same result as in OpenCV. We can use our helper function to plot the decision boundary:

```
In [15]: plot_decision_boundary(forest, X_test, y_test)
```

The output looks like this:



Decision boundary of a random forest

Implementing extremely randomized trees

Random forests are already pretty arbitrary. But what if we wanted to take the randomness to its extreme?

In **extremely randomized trees** (see `ExtraTreesClassifier` and `ExtraTreesRegressor` classes), the randomness is taken even further than in random forests. Remember how decision trees usually choose a threshold for every feature so that the purity of the node split is maximized. Extremely randomized trees, on the other hand, choose these thresholds at random. The best one of these randomly-generated thresholds is then used as the splitting rule.

We can build an extremely randomized tree as follows:

```
In [16]: from sklearn.ensemble import ExtraTreesClassifier
...      extra_tree = ExtraTreesClassifier(n_estimators=10,
...                                       random_state=100)
```

To illustrate the difference between a single decision tree, a random forest, and extremely randomized trees, let's consider a simple dataset, such as the Iris dataset:

```
In [17]: from sklearn.datasets import load_iris
...      iris = load_iris()
...      X = iris.data[:, [0, 2]]
...      y = iris.target
In [18]: X_train, X_test, y_train, y_test = train_test_split(
...      X, y, random_state=100
...      )
```

We can then fit and score the tree object the same way we did before:

```
In [19]: extra_tree.fit(X_train, y_train)
...      extra_tree.score(X_test, y_test)
Out[19]: 0.92105263157894735
```

For comparison, using a random forest would have resulted in the same performance:

```
In [20]: forest = RandomForestClassifier(n_estimators=10,
...                                     random_state=100)
...      forest.fit(X_train, y_train)
...      forest.score(X_test, y_test)
Out[20]: 0.92105263157894735
```

In fact, the same is true for a single tree:

```
In [21]: tree = DecisionTreeClassifier()
...      tree.fit(X_train, y_train)
```

```
...     tree.score(X_test, y_test)
Out[21]: 0.92105263157894735
```

So what's the difference between them? To answer this question, we have to look at the decision boundaries. Fortunately, we have already imported our `plot_decision_boundary` helper function in the preceding section, so all we need to do is pass the different classifier objects to it.

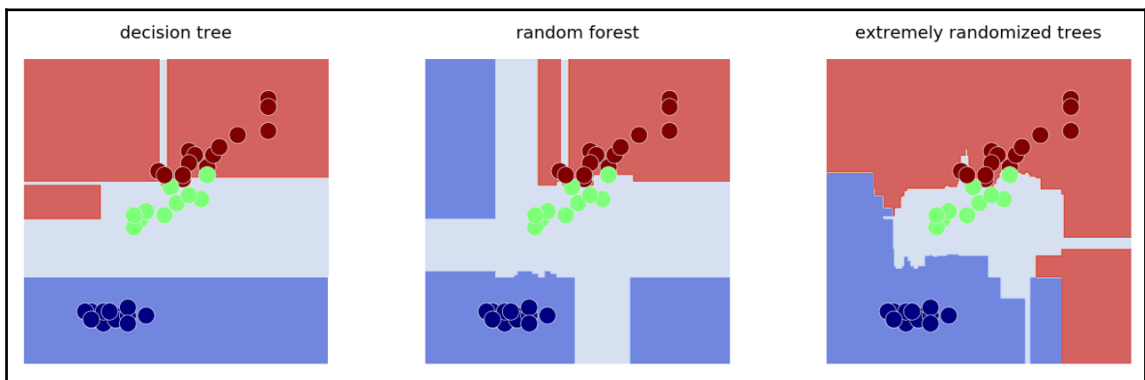
We will build a list of classifiers, where each entry in the list is a tuple that contains an index, a name for the classifier, and the classifier object:

```
In [22]: classifiers = [
...     (1, 'decision tree', tree),
...     (2, 'random forest', forest),
...     (3, 'extremely randomized trees', extra_tree)
... ]
```

Then it's easy to pass the list of classifiers to our helper function such that the decision landscape of every classifier is drawn in its own subplot:

```
In [23]: for sp, name, model in classifiers:
...     plt.subplot(1, 3, sp)
...     plot_decision_boundary(model, X_test, y_test)
...     plt.title(name)
...     plt.axis('off')
```

The result looks like this:



Comparing decision boundaries for various tree based classifiers

Now, the differences between the three classifiers become clearer. We see the single tree drawing by far the simplest decision boundaries, splitting the landscape using horizontal decision boundaries. The random forest is able to more clearly separate the cloud of data points in the lower left of the decision landscape. However, only extremely randomized trees were able to corner the cloud of data points towards the center of the landscape from all sides.

Now that we know about all the different variations of tree ensembles, let's move on to a real-world dataset.

Using random forests for face recognition

A popular dataset that we haven't talked much about yet is the **Olivetti face dataset**.

The Olivetti face dataset was collected in 1990 by AT&T Laboratories Cambridge. The dataset comprises facial images of 40 distinct subjects, taken at different times and under different lighting conditions. In addition, subjects varied their facial expression (open/closed eyes, smiling/not smiling) and their facial details (glasses/no glasses).

Images were then quantized to 256 grayscale levels and stored as unsigned 8-bit integers. Because there are 40 distinct subjects, the dataset comes with 40 distinct target labels. Recognizing faces thus constitutes an example of a **multiclass classification** task.

Loading the dataset

Like many other classic datasets, the Olivetti face dataset can be loaded using scikit-learn:

```
In [1]: from sklearn.datasets import fetch_olivetti_faces
...      dataset = fetch_olivetti_faces()
In [2]: X = dataset.data
...      y = dataset.target
```

Although the original images consisted of 92×112 pixel images, the version available through scikit-learn contains images downscaled to 64×64 pixels.

To get a sense of the dataset, we can plot some example images. Let's pick eight indices from the dataset in a random order:

```
In [3]: import numpy as np
...      np.random.seed(21)
...      idx_rand = np.random.randint(len(X), size=8)
```

We can plot these example images using Matplotlib, but we need to make sure we reshape the column vectors to 64 x 64 pixel images before plotting:

```
In [4]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     for p, i in enumerate(idx_rand):
...         plt.subplot(2, 4, p + 1)
...     plt.imshow(X[i, :].reshape((64, 64)), cmap='gray')
...     plt.axis('off')
```

The preceding code produces the following picture:



Example face images from the Olivetti dataset

You can see how all the faces are taken against a dark background and are upright. The facial expression varies drastically from image to image, making this an interesting classification problem. Try not to laugh at some of them!

Preprocessing the dataset

Before we can pass the dataset to the classifier, we need to preprocess it following the best practices from [Chapter 4, Representing Data and Engineering Features](#).

Specifically, we want to make sure that all example images have the same mean grayscale level:

```
In [5]: n_samples, n_features = X.shape[:2]
...      X -= X.mean(axis=0)
```

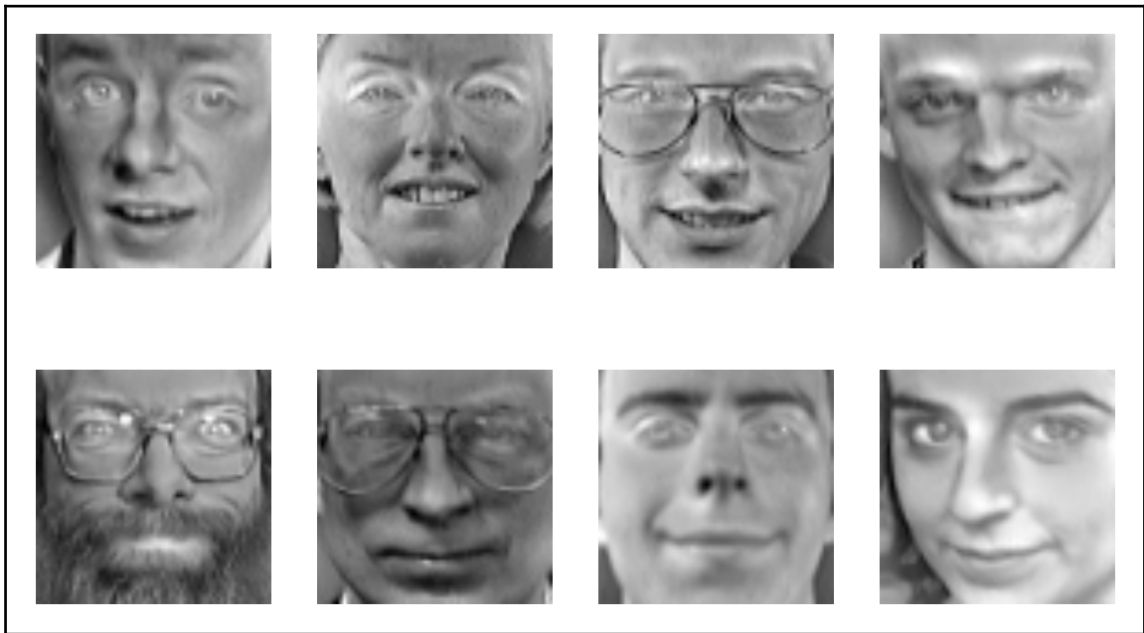
We repeat this procedure for every image to make sure the feature values of every data point (that is, a row in X) are centered around zero:

```
In [6]: X -= X.mean(axis=1).reshape(n_samples, -1)
```

The preprocessed data can be visualized using the preceding code:

```
In [7]: for p, i in enumerate(idx_rand):
...      plt.subplot(2, 4, p + 1)
...      plt.imshow(X[i, :].reshape((64, 64)), cmap='gray')
...      plt.axis('off')
```

This produces the following figure:



Examples of preprocessed face images from the Olivetti dataset

Training and testing the random forest

We continue to follow our best practice to split the data into training and test sets:

```
In [8]: from sklearn.model_selection import train_test_split
...      X_train, X_test, y_train, y_test = train_test_split(
...          X, y, random_state=21
...      )
```

Then we are ready to apply a random forest to the data:

```
In [9]: import cv2
...      rtree = cv2.ml.RTrees_create()
```

Here we want to create an ensemble with 50 decision trees:

```
In [10]: n_trees = 50
...        eps = 0.01
...        criteria = (cv2.TERM_CRITERIA_MAX_ITER + cv2.TERM_CRITERIA_EPS,
...                     n_trees, eps)
...        rtree.setTermCriteria(criteria)
```

Because we have a large number of categories (that is, 40), we want to make sure the random forest is set up to handle them accordingly:

```
In [10]: rtree.setMaxCategories(len(np.unique(y)))
```

We can play with other optional arguments, such as the number of data points required in a node before it can be split:

```
In [11]: rtree.setMinSampleCount(2)
```

However, we might not want to limit the depth of each tree. This is again, a parameter we will have to experiment with in the end. But for now, let's set it to a large integer value, making the depth effectively unconstrained:

```
In [12]: rtree.setMaxDepth(1000)
```

Then we can fit the classifier to the training data:

```
In [13]: rtree.train(X_train, cv2.ml.ROW_SAMPLE, y_train);
```

We can check the resulting depth of the tree using the following function:

```
In [13]: rtree.getMaxDepth()
Out[13]: 25
```

This means that although we allowed the tree to go up to depth 1000, in the end only 25 layers were needed.

The evaluation of the classifier is done once again by predicting the labels first (`y_hat`) and then passing them to the `accuracy_score` function:

```
In [14]: _, y_hat = tree.predict(X_test)
In [15]: from sklearn.metrics import accuracy_score
...       accuracy_score(y_test, y_hat)
Out[15]: 0.87
```

We find 87% accuracy, which turns out to be much better than with a single decision tree:

```
In [16]: from sklearn.tree import DecisionTreeClassifier
...       tree = DecisionTreeClassifier(random_state=21, max_depth=25)
...       tree.fit(X_train, y_train)
...       tree.score(X_test, y_test)
Out[16]: 0.46999999999999997
```

Not bad! We can play with the optional parameters to see if we get better. The most important one seems to be the number of trees in the forest. We can repeat the experiment with a forest made from 100 trees:

```
In [18]: num_trees = 100
...       eps = 0.01
...       criteria = (cv2.TERM_CRITERIA_MAX_ITER + cv2.TERM_CRITERIA_EPS,
...                   num_trees, eps)
...       rtree.setTermCriteria(criteria)
...       rtree.train(X_train, cv2.ml.ROW_SAMPLE, y_train);
...       _, y_hat = rtree.predict(X_test)
...       accuracy_score(y_test, y_hat)
Out[18]: 0.91000000000000003
```

With this configuration, we get 91% accuracy!



Here we tried to improve the performance of our model through **creative trial-and-error**: We vary some of the parameters we deem important, and observe the resulting change in performance, until we find a configuration that satisfies our expectations. We will learn more sophisticated techniques for improving a model in Chapter 11, *Selecting the Right Model with Hyperparameter Tuning*.

Another interesting use case of decision tree ensembles is Adaptive Boosting or AdaBoost.

Implementing AdaBoost

When the trees in the forest are trees of depth 1 (also known as **decision stumps**) and we perform boosting instead of bagging, the resulting algorithm is called **AdaBoost**.

AdaBoost adjusts the dataset at each iteration by performing the following actions:

- Selecting a decision stump
- Increasing the weighting of cases that the decision stump labeled incorrectly while reducing the weighting of correctly labeled cases

This iterative weight adjustment causes each new classifier in the ensemble to prioritize training the incorrectly labeled cases. As a result, the model adjusts by targeting highly-weighted data points.

Eventually, the stumps are combined to form a final classifier.

Implementing AdaBoost in OpenCV

Although OpenCV provides a very efficient implementation of AdaBoost, it is hidden under the **Haar cascade** classifier. Haar cascade classifiers are a very popular tool for face detection, which we can illustrate through the example of the Lena image:

```
In [1]: img_bgr = cv2.imread('data/lena.jpg', cv2.IMREAD_COLOR)
...     img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
```

After loading the image in both color and grayscale, we load a pretrained Haar cascade:

```
In [2]: import cv2
...     filename = 'data/haarcascade_frontalface_default.xml'
...     face_cascade = cv2.CascadeClassifier(filename)
```

The classifier will then detect faces present in the image using the following function call:

```
In [3]: faces = face_cascade.detectMultiScale(img_gray, 1.1, 5)
```

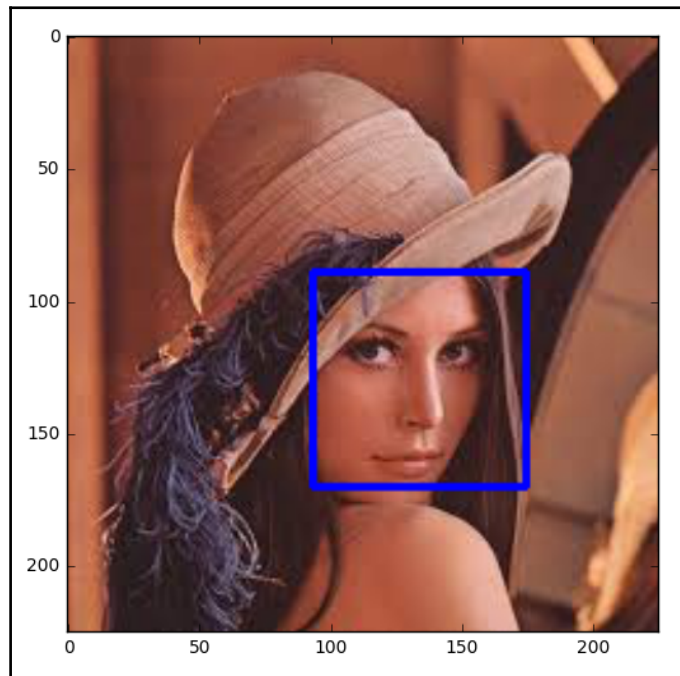
Note that the algorithm operates only on grayscale images. That's why we saved two pictures of Lena, one to which we can apply the classifier (`img_gray`), and one on which we can draw the resulting bounding box (`img_bgr`):

```
In [4]: color = (255, 0, 0)
...     thickness = 2
...     for (x, y, w, h) in faces:
...         cv2.rectangle(img_bgr, (x, y), (x + w, y + h),
...                        color, thickness)
```

Then we can plot the image using the following code:

```
In [5]: import matplotlib.pyplot as plt
...     %matplotlib inline
...     plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB));
```

This results in the following image, with the location of the face indicated by a blue bounding box:



Detecting faces in the Lena image

Obviously, this picture contains only a single face. However, the preceding code will work even on images where multiple faces could be detected. Try it out!

Implementing AdaBoost in scikit-learn

In scikit-learn, AdaBoost is just another ensemble estimator. We can create an ensemble from 100 decision stumps as follows:

```
In [6]: from sklearn.ensemble import AdaBoostClassifier
...      ada = AdaBoostClassifier(n_estimators=100,
...                               random_state=456)
```

We can load the breast cancer set once more and split it 75-25:

```
In [7]: from sklearn.datasets import load_breast_cancer
...      cancer = load_breast_cancer()
...      X = cancer.data
...      y = cancer.target
In [8]: from sklearn.model_selection import train_test_split
...      X_train, X_test, y_train, y_test = train_test_split(
...          X, y, random_state=456
...      )
```

Then fit and score AdaBoost using the familiar procedure:

```
In [9]: ada.fit(X_train, y_train)
...      ada.score(X_test, y_test)
Out[9]: 0.97902097902097907
```

The result is remarkable, 97.9% accuracy!

We might want to compare this result to a random forest. However, to be fair, we should make the trees in the forest all decision stumps. Then we will know the difference between bagging and boosting:

```
In [10]: from sklearn.ensemble import RandomForestClassifier
...        forest = RandomForestClassifier(n_estimators=100,
...                                       max_depth=1,
...                                       random_state=456)
...        forest.fit(X_train, y_train)
...        forest.score(X_test, y_test)
Out[10]: 0.93706293706293708
```

Of course, if we let the trees be as deep as needed, we might get a better score:

```
In [11]: forest = RandomForestClassifier(n_estimators=100,
...                                     random_state=456)
...        forest.fit(X_train, y_train)
...        forest.score(X_test, y_test)
Out[11]: 0.98601398601398604
```

As a last step in this chapter, let's talk about how to combine different types of models into an ensemble.

Combining different models into a voting classifier

So far, we saw how to combine different instances of the same classifier or regressor into an ensemble. In this chapter, we are going to take this idea a step further and combine conceptually different classifiers into what is known as a **voting classifier**.

The idea behind voting classifiers is that the individual learners in the ensemble don't necessarily need to be of the same type. After all, no matter how the individual classifiers arrived at their prediction, in the end, we are going to apply a decision rule that integrates all the votes of the individual classifiers. This is also known as a **voting scheme**.

Understanding different voting schemes

Two different voting schemes are common among voting classifiers:

- In **hard voting** (also known as **majority voting**), every individual classifier votes for a class, and the majority wins. In statistical terms, the predicted target label of the ensemble is the mode of the distribution of individually predicted labels.
- In **soft voting**, every individual classifier provides a probability value that a specific data point belongs to a particular target class. The predictions are weighted by the classifier's importance and summed up. Then the target label with the greatest sum of weighted probabilities wins the vote.

For example, let's assume we have three different classifiers in the ensemble that perform a binary classification task. Under the hard voting scheme, every classifier would predict a target label for a particular data point:

classifier	predicted target label
classifier #1	class 1
classifier #2	class 1
classifier #3	class 0

The voting classifier would then tally up the votes and go with the majority. In this case, the ensemble classifier would predict class 1.

In a soft voting scheme, the math is slightly more involved. In soft voting, every classifier is assigned a weight coefficient, which stands for the importance of the classifier in the voting procedure. For the sake of simplicity, let's assume all three classifiers have the same weight: $w_1 = w_2 = w_3 = 1$.

The three classifiers would then go on to predict the probability of a particular data point belonging to each of the available class labels:

classifier	class 0	class 1
classifier #1	$0.3 w_1$	$0.7 w_1$
classifier #2	$0.5 w_2$	$0.5 w_2$
classifier #3	$0.4 w_3$	$0.6 w_3$

In this example, classifier #1 is 70% sure we're looking at an example of class 1. Classifier #2 is fifty-fifty, and classifier #3 tends to agree with classifier #1. Every probability score gets combined with the weight coefficient of the classifier.

The voting classifier would then compute the weighted average for each class label:

- For class 0, we get a weighted average of $(0.3 w_1 + 0.5 w_2 + 0.4 w_3) / 3 = 0.4$
- For class 1, we get a weighted average of $(0.7 w_1 + 0.5 w_2 + 0.6 w_3) / 3 = 0.6$

Because the weighted average for class 1 is higher than for class 0, the ensemble classifier would go on to predict class 1.

Implementing a voting classifier

Let's look at a simple example of a voting classifier that combines three different algorithms:

- A logistic regression classifier from [Chapter 3, First Steps in Supervised Learning](#)
- A Gaussian naive Bayes classifier from [Chapter 7, Implementing a Spam Filter with Bayesian Learning](#)
- A random forest classifier from this chapter

We can combine these three algorithms into a voting classifier and apply it to the breast cancer dataset with the following steps:

1. Load the dataset, and split it into training and test sets:

```
In [1]: from sklearn.datasets import load_breast_cancer
...     cancer = load_breast_cancer()
...     X = cancer.data
...     y = cancer.target
In [2]: from sklearn.model_selection import train_test_split
...     X_train, X_test, y_train, y_test = train_test_split(
...         X, y, random_state=13
...     )
```

2. Instantiate the individual classifiers:

```
In [3]: from sklearn.linear_model import LogisticRegression
...     model1 = LogisticRegression(random_state=13)
In [4]: from sklearn.naive_bayes import GaussianNB
...     model2 = GaussianNB()
In [5]: from sklearn.ensemble import RandomForestClassifier
...     model3 = RandomForestClassifier(random_state=13)
```

3. Assign the individual classifiers to the voting ensemble. Here, we need to pass a list of tuples (estimators), where every tuple consists of the name of the classifier (a string of our choosing) and the model object. The voting scheme can be either `voting='hard'` or `voting='soft'`:

```
In [6]: from sklearn.ensemble import VotingClassifier
...     vote = VotingClassifier(estimators=[('lr', model1),
...                                         ('gnb', model2),
...                                         ('rfc', model3)],
...                             voting='hard')
```

4. Fit the ensemble to the training data and score it on the test data:

```
In [7]: vote.fit(X_train, y_train)
...     vote.score(X_test, y_test)
Out[7]: 0.95104895104895104
```

In order to convince us that 95.1% is a great accuracy score, we can compare the ensemble's performance to the theoretical performance of each individual classifier. We do this by fitting the individual classifiers to the data. Then we will see that the logistic regression model achieves 94.4% accuracy on its own:

```
In [8]: model1.fit(X_train, y_train)
...     model1.score(X_test, y_test)
Out[8]: 0.94405594405594406
```

Similarly, the naive Bayes classifier achieves 93.0% accuracy:

```
In [9]: model2.fit(X_train, y_train)
...     model2.score(X_test, y_test)
Out[9]: 0.93006993006993011
```

Last but not least, the random forest classifier also achieved 94.4% accuracy:

```
In [10]: model3.fit(X_train, y_train)
...      model3.score(X_test, y_test)
Out[10]: 0.94405594405594406
```

All in all, we were just able to gain a good percent in performance by combining three unrelated classifiers into an ensemble. Each of these classifiers might have made different mistakes on the training set, but that's OK because on average, we need just two out of three classifiers to be correct.

Summary

In this chapter, we talked about how to improve various classifiers by combining them into an ensemble. We discussed how to average the predictions of different classifiers using bagging and how to have different classifiers correct each other's mistakes using boosting. A lot of time was spent discussing all possible ways to combine decision trees, be it decision stumps (AdaBoost), a bag of full of trees, random forests, or extremely randomized trees. Finally, we learned how to combine even different types of classifiers in an ensemble by building a voting classifier.

In the next chapter, we will talk more about how to compare the results of different classifiers by diving into the world of model selection and hyperparameter tuning.

11

Selecting the Right Model with Hyperparameter Tuning

Now that we have visited a wide variety of machine learning algorithms, I am sure you have realized that most of them come with a great number of settings to choose from. These settings or tuning knobs, the so-called **hyperparameters**, help us control the behavior of the algorithm when we try to maximize performance.

For example, we might want to choose the depth or split criterion in a decision tree or tune the number of neurons in a neural network. Finding the values of important parameters of a model is a tricky task but necessary for almost all models and datasets.

In this chapter, we will thus dive deeper into **model evaluation** and **hyperparameter tuning**. Assume that we have two different models that might apply to our task. How can we know which one is better? Answering this question often involves repeatedly fitting different versions of our model to different subsets of the data, such as in **cross-validation** and **bootstrapping**. In combination with different scoring functions, we can obtain reliable estimates of the generalization performance of our models.

But what if two different models give similar results? Can we be sure that the two models are equivalent, or is it possible that one of them just got lucky? How can we know whether one of them is significantly better than the other? Answering these questions will lead us to discussing some useful statistical tests such as **Students t-test** and **McNemar's test**.

As we will get familiar with these techniques, we will also want to answer the following questions:

- What's the best strategy to tweak the hyperparameters of a model?
- How can we compare the performance of different models in a fair way?
- How do we select the right machine learning tool for the task at hand?

Evaluating a model

Model evaluation strategies come in many different forms and shapes. In the following sections, we will, therefore, highlight three of the most commonly used techniques to compare models against each other:

- k -fold cross-validation
- bootstrapping
- McNemar's test

In principle, model evaluation is simple: after training a model on some data, we can estimate its effectiveness by comparing model predictions to some ground truth values. We learned early on that we should split the data into a training and a test set, and we tried to follow this instruction whenever possible. But why exactly did we do that again?

Evaluating a model the wrong way

The reason we never evaluate a model on the training set is that, in principle, any dataset can be learned if we throw a strong enough model at it.

A quick demonstration of this can be given with help of the Iris dataset, which we talked about extensively in [Chapter 3, *First Steps in Supervised Learning*](#). There the goal was to classify species of Iris flowers based on their physical dimensions. We can load the Iris dataset using scikit-learn:

```
In [1]: from sklearn.datasets import load_iris
...     iris = load_iris()
```

An innocent approach to this problem would be to store all data points in matrix x and all class labels in the vector y :

```
In [2]: import numpy as np
...     x = iris.data.astype(np.float32)
...     y = iris.target
```

Next, we choose a model and its hyperparameters. For example, let's use the k -NN algorithm from [Chapter 3, *First Steps in Supervised Learning*](#), which provides only a single hyperparameter: the number of neighbors, k . With $k=1$, we get a very simple model that classifies the label of an unknown point as belonging to the same class as its closest neighbor.

In OpenCV, k -NN instantiates as follows:

```
In [3]: import cv2
...     knn = cv2.ml.KNearest_create()
...     knn.setDefaultK(1)
```

Then we train the model and use it to predict labels for the data that we already know:

```
In [4]: knn.train(X, cv2.ml.ROW_SAMPLE, y)
...     _, y_hat = knn.predict(X)
```

Finally, we compute the fraction of correctly labeled points:

```
In [5]: from sklearn.metrics import accuracy_score
...     accuracy_score(y, y_hat)
Out[5]: 1.0
```

We see an accuracy score of 1.0, which indicates that 100% of points were correctly labeled by our model!



If a model gets 100% accuracy on the training set, we say the model **memorized** the data.

But is this truly measuring the expected accuracy? Have we really come up with a model that we expect to be correct 100% of the time?

As you may have gathered, the answer is no. This example shows that even a simple algorithm is capable of memorizing a real-world dataset. Imagine how easy this task would have been for a deep neural network! Usually, the more parameters a model has, the more powerful it is. We will come back to this shortly.

Evaluating a model in the right way

A better sense of a model's performance can be found using what's known as a test set, but you already knew this. When presented with data held out from the training procedure, we can check whether a model has learned some dependencies in the data that hold across the board or whether it just memorized the training set.

We can split the data into training and test sets using the familiar `train_test_split` from scikit-learn's `model_selection` module:

```
In [6]: from sklearn.model_selection import train_test_split
```

But how do we choose the right train-test ratio? Is there even such a thing as a right ratio? Or is this considered another hyperparameter of the model?

There are two competing concerns here:

- If our training set is too small, our model might not be able to extract the relevant data dependencies. As a result, our model performance might differ significantly from run to run, that is, if we repeat our experiment multiple times with different random number seeds. As an extreme example, consider a training set with a single data point from the Iris dataset. In this case, there would be no way for the model to even learn that there are multiple species in the dataset!
- If our test set is too small, our performance metric might differ significantly from run to run. As a result, we would have to rerun our experiment multiple times to get an idea of how well our model does on average. As an extreme example, consider a test set with a single data point. Since there are three different classes in the Iris dataset, we might get either 0, 33%, 66%, or 100% correct.

A good starting point is usually a 80-20 training-test split. However, it all depends on the amount of data available. For relatively small datasets, a 50-50 split might be more suitable:

```
In [7]: X_train, X_test, y_train, y_test = train_test_split(
...     X, y, random_state=37, train_size=0.5
...     )
```

Then we retrain the preceding model on the training set:

```
In [8]: knn = cv2.ml.KNearest_create()
...     knn.setDefaultK(1)
...     knn.train(X_train, cv2.ml.ROW_SAMPLE, y_train)
```

When we test the model on the test set, we suddenly get a different result:

```
In [9]: _, y_test_hat = knn.predict(X_test)
...     accuracy_score(y_test, y_test_hat)
Out[9]: 0.9666666666666667
```

We see a more reasonable result here, although 97% accuracy is still a formidable result. But is this the best possible result—and how can we know for sure?

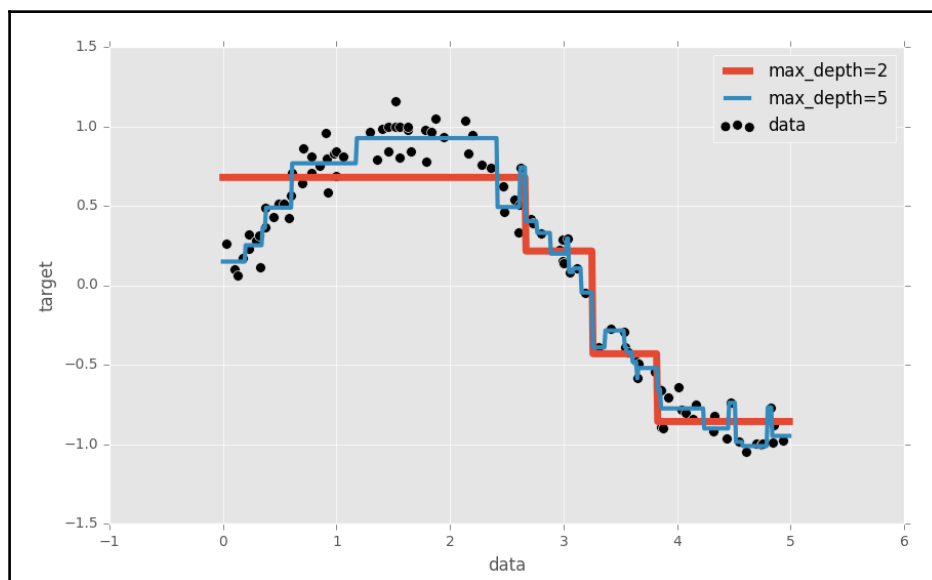
To answer this question is, we have to dig a little deeper.

Selecting the best model

When a model is under-performing, it is often not clear how to make it better. Throughout this book, I have declared several decisions a secret of the trade, for example, how to select the number of layers in a neural network. Even worse, the answer is often counter intuitive! For example, adding another layer to the network might make the results worse, and adding more training data might not change performance at all.

You can see why these issues are some of the most important aspects of machine learning. At the end of the day, the ability to determine what steps will or will not improve our model is what separates the successful machine learning practitioner from all others.

Let's have a look at a specific example. Remember [Chapter 5, Using Decision Trees to Make a Medical Diagnosis](#), where we used decision trees in a regression task? We were fitting two different trees to a sin function—one with depth 2 and one with depth 5. As a reminder, the regression result looked like this:



Approximating a sin function using decision trees

It should be clear that neither of these fits are particularly good. However, the two decision trees fail in two different ways!

The decision tree with depth 2 (thick line in the preceding figure) attempts to fit four straight lines through the data. Because the data is intrinsically more complicated than a few straight lines, this model fails. We could train it as much as we wanted, on as many training samples as we could generate—it would never be able to describe this dataset well. Such a model is said to **underfit** the data. In other words, the model does not have enough flexibility to account for all the features in the data. Another way of saying this is that the model has **high bias**.

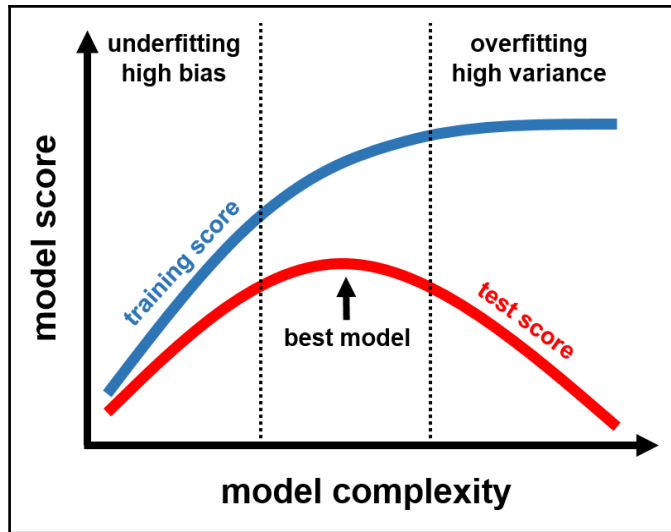
The other decision tree (thin line, depth 5) makes a different mistake. This model has enough flexibility to nearly perfectly account for the fine structures in the data. However, at some points, the model seems to follow the particular pattern of the noise; we added to the sin function rather than the sin function itself. You can see that on the right-hand side of the graph, where the blue curve (thin line) would jitter a lot. Such a model is said to **overfit** the data. In other words, the model has so much flexibility that it ends up accounting for random errors in the data. Another way of saying this is that model has **high variance**.

To give you a long story short—here's the secret sauce: Fundamentally, selecting the right model comes down to finding a sweet spot in the trade-off between bias and variance.



The amount of flexibility a model has (also known as the **model complexity**) is mostly dictated by its hyperparameters. That is why it is so important to tune them!

Let's return to the k -NN algorithm and the Iris dataset. If we repeated the procedure of fitting the model to the Iris data for all possible values of k and calculated both training and test scores, we would expect the result to look something like the following:



Model score as a function of model complexity

If there is one thing I would want you to remember from this chapter, it would be this diagram. Let's unpack it.

The diagram describes the model score (either training or test score) as a function of model complexity. As mentioned in the preceding diagram, the model complexity of a neural network roughly grows with the number of neurons in the network. In the case of k -NN, the opposite logic applies—the larger the value for k , the smoother the decision boundary, and thus, the lower the complexity. In other words, k -NN with $k=1$ would be all the way to the right in the preceding diagram, where the training score is perfect. No wonder we got 100% accuracy on the training set!

From the preceding diagram, we can gather that there are three regimes in the model complexity landscape:

- For very low model complexity (a high-bias model), the training data is underfit. In this regime, the model achieves only low scores on both the training and test set, no matter for how long we trained it.
- For very high model complexity (a high-variance model), the training data is overfit, which means that the model predicts the training data very well but fails for any previously unseen data. In this regime, the model has started to learn intricacies or peculiarities that only appear in the training data. Since these peculiarities do not apply to unseen data, the training score gets lower and lower.

- For some intermediate value, the test score is maximal. It is this intermediate regime, where the test score is maximal, that we are trying to find. This is the sweet spot in the trade-off between bias and variance!

This means that we can find the best algorithm for the task at hand by mapping out the model complexity landscape. Specifically, we can use the following indicators to know which regime we are currently in:

- If both training and test scores are below our expectations, we are probably in the leftmost regime in the preceding diagram, where the model is underfitting the data. In this case, a good idea might be to increase the model complexity and try again.
- If the training score is much higher than the test score, we are probably in the rightmost regime in the preceding diagram, where model is overfitting the data. In this case, a good idea might be to decrease the model complexity and try again.

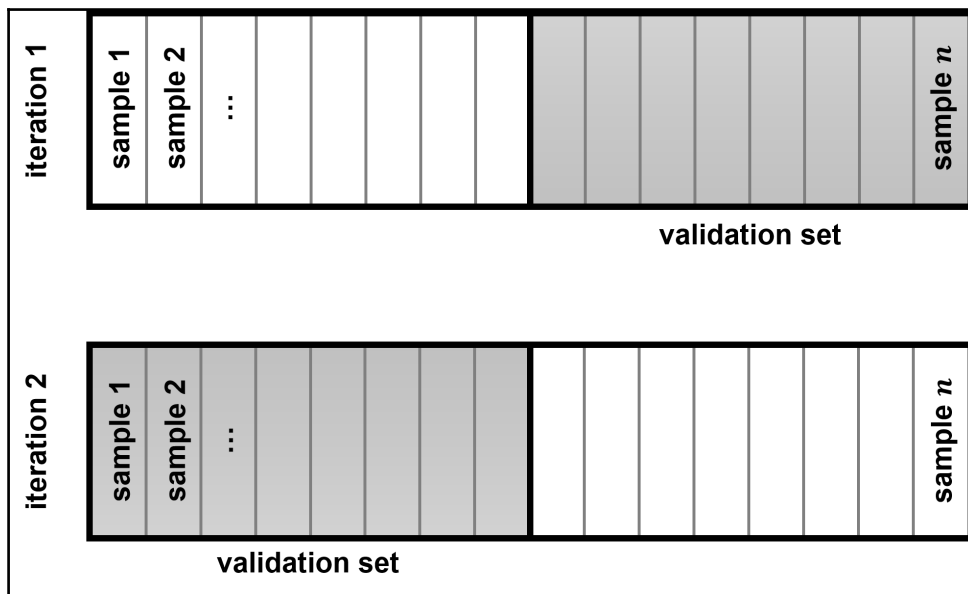
Although this procedure works in general, there are more sophisticated strategies for model evaluation that proved to be more thorough than a simple train-test split, which we will talk about in the following sections.

Understanding cross-validation

Cross-validation is a method of evaluating the generalization performance of a model that is generally more stable and thorough than splitting the dataset into training and test sets.

The most commonly used version of cross-validation is ***k*-fold cross-validation**, where *k* is a number specified by the user (usually five or ten). Here, the dataset is partitioned into *k* parts of more or less equal size, called **fold**s. For a dataset that contains *N* data points, each fold should thus have approximately N / k samples. Then a series of models is trained on the data, using *k* - 1 folds for training and one remaining fold for testing. The procedure is repeated for *k* iterations, each time choosing a different fold for testing, until every fold has served as a test set once

An example of two-fold cross-validation is shown in the following figure:

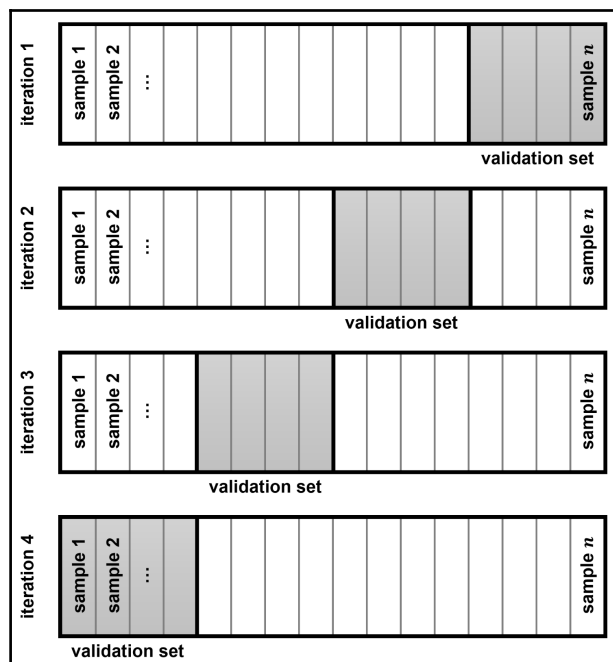


Two-fold cross-validation



At first glance, this might look like our regular procedure of splitting the data into a training and test set. One difference is that cross-validation is run twice in this case: once where the model is trained on what we usually call the training set (and tested on the test set), and once where the model is trained on what we usually call the test set (and tested on the training set).

Analogously, in a cross-validation procedure with four folds, we would start by using the first three folds for training and the fourth fold for testing (iteration 1). In iteration 2, we would choose the third fold for testing (called **validation set** shown in the following figure) and the remaining folds (folds 1, 2, and 4) for testing, and so on:



Four-fold cross-validation

During each of these iterations, we compute the accuracy on the validation set. At the end of the day, we take the average of all accuracy scores to get a final mean score of the model. This will give us a much more robust estimate of the generalization performance of our model because we repeated the train-test procedure multiple times on multiple random splits of the data.



Note that the purpose of cross-validation is not to build a final model, that is, we don't use these four instances of our trained model to do any real prediction. The purpose of cross-validation is only to evaluate how well a given algorithm will generalize when trained on a specific dataset.

Having multiple splits of the data also provides some information about how sensitive our model is to the selection of the training dataset. Since we are dealing with random data splits, we have to consider the possibility that our procedure is subject to luck:

- We might get **lucky** in one iteration so that for some reason, all data points in the validation set are really easy to classify. Then the reported accuracy would be unreasonably high.
- We might get **unlucky** in one iteration so that the data points in the validation set are practically impossible to classify correctly. Then the reported accuracy would be unreasonably low.

By repeating the procedure for k folds, we make sure that the observed result does not strongly depend on the exact train-test split. As a bonus, we can get an idea of how the model might perform in the worst case and best case scenarios when applied to new data by looking at the differences in accuracy from one fold to the next.

Another benefit of cross-validation over a single train-test split is that we use our data more effectively. Usually, the more data we can use to fit the model, the more accurate the model will end up being. However, if we did a 80-20 train-test split using the `train_test_split` function, we would have to reserve 20% of all data points for testing. In five-fold cross-validation, we might set 20% of the data points for validation, but by cycling through the different iterations, we eventually end up using every data point for training. Even better—in ten-fold cross-validation, we can use 90% of the data for training!

Manually implementing cross-validation in OpenCV

The easiest way to perform cross-validation in OpenCV is to do the data splits by hand.

For example, in order to implement two-fold cross-validation, we would follow the following procedure:

1. Load the dataset:

```
In [1]: from sklearn.datasets import load_iris
...     import numpy as np
...     iris = load_iris()
...     X = iris.data.astype(np.float32)
...     y = iris.target
```

2. Split the data into two equally sized parts:

```
In [2]: from sklearn.model_selection import model_selection
...     X_fold1, X_fold2, y_fold1, y_fold2 = train_test_split(
...         X, y, random_state=37, train_size=0.5
...     )
```

3. Instantiate the classifier:

```
In [3]: import cv2
...      knn = cv2.ml.KNearest_create()
...      knn.setDefaultK(1)
```

4. Train the classifier on the first fold, then predict the labels of the second fold:

```
In [4]: knn.train(X_fold1, cv2.ml.ROW_SAMPLE, y_fold1)
...      _, y_hat_fold2 = knn.predict(X_fold2)
```

5. Train the classifier on the second fold, then predict the labels of the first fold:

```
In [5]: knn.train(X_fold2, cv2.ml.ROW_SAMPLE, y_fold2)
...      _, y_hat_fold1 = knn.predict(X_fold1)
```

6. Compute accuracy scores for both folds:

```
In [6]: from sklearn.metrics import accuracy_score
...      accuracy_score(y_fold1, y_hat_fold1)
Out[6]: 0.92000000000000004
In [7]: accuracy_score(y_fold2, y_hat_fold2)
Out[7]: 0.88
```

This procedure will yield two accuracy scores, one for the first fold (92% accuracy), and one for the second fold (88% accuracy). On average, our classifier thus achieved 90% accuracy on unseen data.

Using scikit-learn for k-fold cross-validation

In scikit-learn, cross-validation can be performed in three steps:

1. Load the dataset. Since we already did this earlier, we don't have to do it again.
2. Instantiate the classifier:

```
In [8]: from sklearn.neighbors import KNeighborsClassifier
...      model = KNeighborsClassifier(n_neighbors=1)
```

3. Perform cross-validation with the `cross_val_score` function. This function takes as input a model, the full dataset (`x`), the target labels (`y`) and an integer value for the number of folds (`cv`). It is not necessary to split the data by hand—the function will do that automatically depending on the number of folds. After the cross-validation is completed, the function returns the test scores:

```
In [9]: from sklearn.model_selection import cross_val_score
...     scores = cross_val_score(model, X, y, cv=5)
...     scores
Out[9]: array([ 0.96666667, 0.96666667, 0.93333333, 0.93333333,
               1.  ])
```

In order to get a sense how the model did on average, we can look at the mean and standard deviation of the five scores:

```
In [10]: scores.mean(), scores.std()
Out[10]: (0.95999999999999996, 0.024944382578492935)
```

With five folds, we have a much better idea about how robust the classifier is on average. We see that k -NN with $k=1$ achieves on average 96% accuracy, and this value fluctuates from run to run with a standard deviation of roughly 2.5%.

Implementing leave-one-out cross-validation

Another popular way to implement cross-validation is to choose the number of folds equal to the number of data points in the dataset. In other words, if there are N data points, we set $k=N$. This means that we will end up having to do N iterations of cross-validation, but in every iteration, the training set will consist of only a single data point. The advantage of this procedure is that we get to use all-but-one data point for training. Hence, this procedure is also known as **leave-one-out** cross-validation.

In scikit-learn, this functionality is provided by the `LeaveOneOut` method from the `model_selection` module:

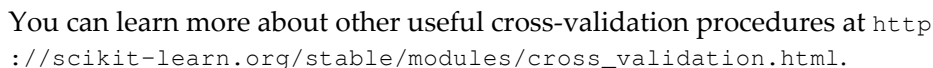
```
In [11]: from sklearn.model_selection import LeaveOneOut
```

This object can be passed directly to the `cross_val_score` function in the following way:

```
In [12]: scores = cross_val_score(model, X, y, cv=LeaveOneOut())
```

If we want to know the average performance of the classifier, we would still compute the mean and standard deviation of the scores:

We can see this scoring scheme returns very similar results to five-fold cross-validation.



An alternative procedure to k -fold cross-validation is **bootstrapping**.

Instead of splitting the data into folds, bootstrapping builds a training set by drawing samples randomly from the dataset. Typically, a bootstrap is formed by drawing samples with replacement. Imagine putting all of the data points into a bag and then drawing randomly from the bag. After drawing a sample, we would put it back in the bag. This allows for some samples to show up multiple times in the training set, which is something cross-validation does not allow.

The classifier is then tested on all samples that are not part of the bootstrap (the so-called **out-of-bag** examples), and the procedure is repeated a large number of times (say, 10,000 times). Thus, we get a distribution of the model's score that allows us to estimate the robustness of the model.

Manually implementing bootstrapping in OpenCV

Bootstrapping can be implemented with the following procedure:

1. Load the dataset. Since we already did this earlier, we don't have to do it again.
2. Instantiate the classifier:

```
In [15]: knn = cv2.ml.KNearest_create()
...      knn.setDefaultK(1)
```

3. From our dataset with N samples, randomly choose N samples with replacement to form a bootstrap. This can be done most easily with the `choice` function from NumPy's `random` module. We tell the function to draw `len(X)` samples in the range `[0, len(X)-1]` with replacement (`replace=True`). The function then returns a list of indices, from which we form our bootstrap:

```
In [16]: idx_boot = np.random.choice(len(X), size=len(X),
...                                  replace=True)
...      X_boot = X[idx_boot, :]
...      y_boot = y[idx_boot]
```

4. Put all samples that do not show in the bootstrap in the out-of-bag set:

```
In [17]: idx_oob = np.array([x not in idx_boot
...                           for x in np.arange(len(X))], dtype=np.bool)
...      X_oob = X[idx_oob, :]
...      y_oob = y[idx_oob]
```

5. Train the classifier on the bootstrap samples:

```
In [18]: knn.train(X_train, cv2.ml.ROW_SAMPLE, y_boot)
Out[18]: True
```

6. Test the classifier on the out-of-bag samples:

```
In [19]: _, y_hat = knn.predict(X_oob)
...      accuracy_score(y_oob, y_hat)
Out[19]: 0.9285714285714286
```

7. Steps 3-6 form one it
8. Iteration of the bootstrap. Repeat these steps up to 10,000 times to get 10,000 accuracy scores, then average the scores to get an idea of the classifier's mean performance.

For our convenience, we can build a function from steps 3-6 so that it is easy to run the procedure for some `n_iter` number of times. We also pass a model (our *k*-NN classifier, `model`), the feature matrix (`X`), and the vector with all class labels (`y`):

```
In [20]: def yield_bootstrap(model, X, y, n_iter=10000):  
...       for _ in range(n_iter):
```

The steps within the for loop are essentially steps 3-6 from the code mentioned earlier. This involved training the classifier on the bootstrap and testing it on the out-of-bag examples:

```
...         # train the classifier on bootstrap  
...         idx_boot = np.random.choice(len(X), size=len(X),  
...                                     replace=True)  
...         X_boot = X[idx_boot, :]  
...         y_boot = y[idx_boot]  
...         knn.train(X_boot, cv2.ml.ROW_SAMPLE, y_boot)  
...  
...         # test classifier on out-of-bag examples  
...         idx_oob = np.array([x not in idx_boot  
...                             for x in np.arange(len(X))],  
...                             dtype=np.bool)  
...         X_oob = X[idx_oob, :]  
...         y_oob = y[idx_oob]  
...         _, y_hat = knn.predict(X_oob)
```

Then we need to return the accuracy score. You might expect a `return` statement here. However, a more elegant way is to use the `yield` statement, which turns the function automatically into a generator. This means we don't have to initialize an empty list (`acc = []`) and then append the new accuracy score at each iteration (`acc.append(accuracy_score(...))`). The bookkeeping is done automatically:

```
...         yield accuracy_score(y_oob, y_hat)
```

To make sure we all get the same result, let's fix the seed of the random number generator:

```
In [21]: np.random.seed(42)
```

Now, let's run the procedure for `n_iter=10` times by converting the function output to a list:

```
In [22]: list(yield_bootstrap(knn, X, y, n_iter=10))
Out[22]: [0.98333333333333328,
          0.93650793650793651,
          0.92452830188679247,
          0.92307692307692313,
          0.94545454545454544,
          0.94736842105263153,
          0.98148148148148151,
          0.96078431372549022,
          0.93220338983050843,
          0.96610169491525422]
```

As you can see, for this small sample we get accuracy scores anywhere between 92% and 98%. To get a more reliable estimate of the model's performance, we repeat the procedure 1,000 times and calculate both mean and standard deviation of the resulting scores:

```
In [23]: acc = list(yield_bootstrap(knn, X, y, n_iter=1000))
...      np.mean(acc), np.std(acc)
Out[23]: (0.95524155136419198, 0.022040380995646654)
```

You are always welcome to increase the number of repetitions. But once `n_iter` is large enough, the procedure should be robust to the randomness of the sampling procedure. In this case, we do not expect to see any more changes to the distribution of score values as we keep increasing `n_iter` to, for example, 10,000 iterations:

```
In [24]: acc = list(yield_bootstrap(knn, X, y, n_iter=10000))
...      np.mean(acc), np.std(acc)
Out[24]: (0.95501528733009422, 0.021778543317079499)
```

Typically, the scores obtained with bootstrapping would be used in a **statistical test** to assess the **significance** of our result. Let's have a look at how that is done.

Assessing the significance of our results

Assume for a moment that we implemented the cross-validation procedure for two versions of our k -NN classifier. The resulting test scores are— 92.34% for Model A, and 92.73% for Model B. How do we know which model is better?

Following our logic introduced here, we might argue for Model B because it has a better test score. But what if the two models are not significantly different? These could have two underlying causes, which are both a consequence of the randomness of our testing procedure:

- For all we know, Model B just got lucky. Perhaps we chose a really low k for our cross-validation procedure. Perhaps Model B ended up with a beneficial train-test split so that the model had no problem classifying the data. After all, we didn't run tens of thousands of iterations like in bootstrapping to make sure the result holds in general.
- Variability in the test scores is so high that we cannot know for sure whether the two results are essentially the same. This could be the case even over 10,000 iterations in bootstrapping! If the testing procedure is inherently noisy, the resulting test scores will be noisy, too.

So how can we know for sure that the two models are different?

Implementing Student's t -test

One of the most famous statistical tests is **Student's t -test**. You might have heard of it before: it allows us to determine whether two sets of data are significantly different from one another. This was a really important test for William Sealy Gosset, the inventor of the test, who worked at the Guinness brewery and wanted to know whether two batches of stout differed in quality.



Note that Student here is capitalized. Although Gosset wasn't allowed to publish his test due to company policy, he did so anyway under his pen name Student.

In practice, the t -test allows us to determine whether two data samples come from underlying distributions with the same mean or **expected value**.

For our purposes, this means that we can use the t -test to determine whether the test scores of two independent classifiers have the same mean value. We start by hypothesizing that the two sets of test scores are identical. We call this the **null hypothesis** because this is the hypothesis we want to nullify, that is, we are looking for evidence to **reject** the hypothesis because we want to ensure that one classifier is significantly better than the other.

We accept or reject a null hypothesis based on a parameter known as the *p*-value that the *t*-test returns. The *p*-value takes on values between 0 and 1. A *p*-value of 0.05 would mean that the null hypothesis is right only 5 out of 100 times. A small *p*-value thus indicates strong evidence that the hypothesis can be safely rejected. It is customary to use $p=0.05$ as a cut-off value below which we reject the null hypothesis.

If this is all too confusing, think of it this way: when we run a *t*-test for the purpose of comparing classifier test scores, we are looking to obtain a small *p*-value because that means that the two classifiers give significantly different results.

We can implement Student's *t*-test with SciPy's `ttest_ind` function from the `stats` module:

```
In [25]: from scipy.stats import ttest_ind
```

Let's start with a simple example. Assume we ran five-fold cross-validation on two classifiers and obtained the following scores:

```
In [26]: scores_a = [1, 1, 1, 1, 1]
...      scores_b = [0, 0, 0, 0, 0]
```

This means that Model A achieved 100% accuracy in all five folds, whereas Model B got 0% accuracy. In this case, it is clear that the two results are significantly different. If we run the *t*-test on this data, we should thus find a really small *p*-value:

```
In [27]: ttest_ind(scores_a, scores_b)
Out[27]: Ttest_indResult(statistic=inf, pvalue=0.0)
```

And we do! We actually get the smallest possible *p*-value, $p=0.0$.

On the other hand, what if the two classifiers got exactly the same numbers, except during different folds. In this case, we would expect the two classifiers to be equivalent, which is indicated by a really large *p*-value:

```
In [28]: scores_a = [0.9, 0.9, 0.9, 0.8, 0.8]
...      scores_b = [0.8, 0.8, 0.9, 0.9, 0.9]
...      ttest_ind(scores_a, scores_b)
Out[28]: Ttest_indResult(statistic=0.0, pvalue=1.0)
```

Analogous to the aforementioned, we get the largest possible *p*-value, $p=1.0$.

To see what happens in a more realistic example, let's return to our k -NN classifier from earlier example. Using the test scores obtained from the ten-fold cross-validation procedure, we can compare two different k -NN classifiers with the following procedure:

1. Obtain a set of test scores for Model A. We choose Model A to be the k -NN classifier from earlier ($k=1$):

```
In [29]: k1 = KNeighborsClassifier(n_neighbors=1)
...      scores_k1 = cross_val_score(k1, X, y, cv=10)
...      np.mean(scores_k1), np.std(scores_k1)
Out[29]: (0.95999999999999996, 0.053333333333333323)
```

2. Obtain a set of test scores for Model B. Let's choose Model B to be a k -NN classifier with $k=3$:

```
In [30]: k3 = KNeighborsClassifier(n_neighbors=3)
...      scores_k3 = cross_val_score(k3, X, y, cv=10)
...      np.mean(scores_k3), np.std(scores_k3)
Out[30]: (0.96666666666666656, 0.044721359549995787)
```

3. Apply the t -test to both sets of scores:

```
In [31]: ttest_ind(scores_k1, scores_k3)
Out[31]: Ttest_indResult(statistic=-0.2873478855663425,
pvalue=0.77712784875052965)
```

As you can see, this is a good example of two classifiers giving different cross-validation scores (96.0% and 96.7%) that turn out to be not significantly different! Because we get a large p -value ($p=0.777$), we expect the two classifiers to be equivalent 77 out of 100 times.

Implementing McNemar's test

A more advanced statistical technique is **McNemar's test**. This test can be used on paired data to determine whether there are any differences between the two samples. As in the case of the t -test, we can use McNemar's test to determine whether two models give significantly different classification results.

McNemar's test operates on pairs of data points. This means that we need to know, for both classifiers, how they classified each data point. Based on the number of data points that the first classifier got right but the second got wrong and vice versa, we can determine whether the two classifiers are equivalent.

Let's assume the preceding Model A and Model B were applied to the same five data points. Whereas Model A classified every data point correctly (denoted with a 1), Model B got all of them wrong (denoted with a 0):

```
In [33]: scores_a = np.array([1, 1, 1, 1, 1])
...      scores_b = np.array([0, 0, 0, 0, 0])
```

McNemar's test wants to know two things:

- How many data points did Model A get right but Model B get wrong?
- How many data points did Model A get wrong but Model B get right?

We can check which data points Model A got right but Model B got wrong as follows:

```
In [34]: a1_b0 = scores_a * (1 - scores_b)
...      a1_b0
Out[34]: array([1, 1, 1, 1, 1])
```

Of course, this applies to all of the data points. The opposite is true for the data points that Model B got right and Model A got wrong:

```
In [35]: a0_b1 = (1 - scores_a) * scores_b
...      a0_b1
Out[35]: array([0, 0, 0, 0, 0])
```

Feeding these numbers to McNemar's test should return a small p -value because the two classifiers are obviously different:

```
In [36]: mcnemar_midp(a1_b0.sum(), a0_b1.sum())
Out[36]: 0.03125
```

And it does!

We can apply McNemar's test to a more complicated example, but we cannot operate on cross-validation scores anymore. The reason is that we need to know the classification result for every data point, not just an average. Hence, it makes more sense to apply McNemar's test to the leave-one-out cross-validation.

Going back to k -NN with $k=1$ and $k=3$, we can calculate their scores as follows:

```
In [37]: scores_k1 = cross_val_score(k1, X, y, cv=LeaveOneOut())
...      scores_k3 = cross_val_score(k3, X, y, cv=LeaveOneOut())
```

The number of data points that one of the classifiers got right but the other got wrong are as follows:

```
In [38]: np.sum(scores_k1 * (1 - scores_k3))
```

```
Out [38]: 0.0
In [39]: np.sum((1 - scores_k1) * scores_k3)
Out [39]: 0.0
```

We got no differences whatsoever! Now it becomes clear why the *t*-test led us to believe that the two classifiers are identical. As a result, if we feed the two sums into McNemar's test function, we get the largest possible p value, $p=1.0$:

```
In [40]: mcnemar_midp(np.sum(scores_k1 * (1 - scores_k3)),
...                  np.sum((1 - scores_k1) * scores_k3))
Out [40]: 1.0
```

Now that we know how to assess the significance of our results, we can take the next step and improve the model's performance by tuning its hyperparameters.

Tuning hyperparameters with grid search

The most commonly used tool for hyperparameter tuning is **grid search**, which is basically a fancy term for saying we will try all possible parameter combinations with a for loop.

Let's have a look at how that is done in practice.

Implementing a simple grid search

Returning to our *k*-NN classifier, we find that we have only one hyperparameter to tune: *k*. Typically, you would have a much larger number of open parameters to mess with, but the *k*-NN algorithm is simple enough for us to manually implement grid search.

Before we get started, we need to split the dataset as we have done before into training and test sets. Here we choose a 75-25 split:

```
In [1]: from sklearn.datasets import load_iris
...      import numpy as np
...      iris = load_iris()
...      X = iris.data.astype(np.float32)
...      y = iris.target
In [2]: X_train, X_test, y_train, y_test = train_test_split(
...      X, y, random_state=37
...      )
```

Then the goal is to loop over all possible values of k . As we do this, we want to keep track of the best accuracy we observed as well as the value for k that gave rise to this result:

```
In [3]: best_acc = 0.0
...     best_k = 0
```

Grid search then looks like an outer loop around the entire train and test procedure:

```
In [4]: import cv2
...     from sklearn.metrics import accuracy_score
...     for k in range(1, 20):
...         knn = cv2.ml.KNearest_create()
...         knn.setDefaultK(k)
...         knn.train(X_train, cv2.ml.ROW_SAMPLE, y_train)
...         _, y_test_hat = knn.predict(X_test)
```

After calculating the accuracy on the test set (`acc`), we compare it to the best accuracy found so far (`best_acc`). If the new value is better, we update our bookkeeping variables and move on to the next iteration:

```
...         acc = accuracy_score(y_test, y_test_hat)
...         if acc > best_acc:
...             best_acc = acc
...             best_k = k
```

When we are done, we can have a look at the best accuracy:

```
In [5]: best_acc, best_k
Out[5]: (0.97368421052631582, 1)
```

Turns out, we can get 97.4% accuracy using $k = 1$.



With more variables, we would naturally extend this procedure by wrapping our code in a nested for loop. However, as you can imagine, this can quickly become computationally expensive.

Understanding the value of a validation set

Following our best practice of splitting the data into training and test sets, we might be tempted to tell people that we have found a model that performs with 97.4% accuracy on the dataset. However, our result might not necessarily generalize to new data. The argument is the same as earlier on in the book when we warranted the train-test split that we need an independent dataset for evaluation.

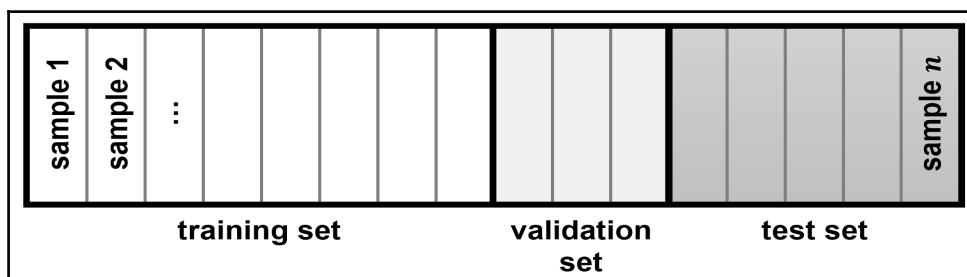
However, when we implemented grid search in the last section, we used the test set to evaluate the outcome of the grid search and update the hyperparameter k . This means we can no longer use the test set to evaluate the final data! Any model choices made based on the test set accuracy would leak information from the test set into the model.

One way to resolve this data is to split the data again and introduce what is known as a **validation set**. The validation set is different from the training and test set and is used exclusively for selecting the best parameters of the model. It is a good practice to do all exploratory analysis and model selection on this validation set and keep a separate test set, which is only used for the final evaluation.

In other words, we should end up splitting the data into three different sets:

- a training set, which is used to build the model
- a validation set, which is used to select the parameters of the model
- a test set, which is used to evaluate the performance of the final model

Such a three-way split is illustrated in the following figure:



Example of how to split a dataset into training, validation, and test sets

In practice, the three-way split is achieved in two steps:

1. Split into two chunks: one that contains training and validation sets and another that contains the test set:

```
In [6]: X_trainval, X_test, y_trainval, y_test =  
...      train_test_split(X, y, random_state=37)  
In [7]: X_trainval.shape  
Out[7]: (112, 4)
```

2. Split `X_trainval` again into proper training and validation sets:

```
In [8]: X_train, X_valid, y_train, y_valid = train_test_split(  
...      X_trainval, y_trainval, random_state=37
```

```
...     )
In [9]: X_train.shape
Out[9]: (84, 4)
```

Then we repeat the manual grid search from the preceding code, but this time, we will use the validation set to find the best k (see code highlights):

```
In [10]: best_acc = 0.0
...     best_k = 0
...     for k in range(1, 20):
...         knn = cv2.ml.KNearest_create()
...         knn.setDefaultK(k)
...         knn.train(X_train, cv2.ml.ROW_SAMPLE, y_train)
...         _, y_valid_hat = knn.predict(X_valid)
...         acc = accuracy_score(y_valid, y_valid_hat)
...         if acc >= best_acc:
...             best_acc = acc
...             best_k = k
...     best_acc, best_k
Out[10]: (1.0, 7)
```

We now find that a 100% validation score (`best_acc`) can be achieved with $k=7$ (`best_k`)! However, recall that this score might be overly optimistic. To find out how well the model really performs, we need to test it on held-out data from the test set.

In order to arrive at our final model, we can use the value for k we found during grid search and re-train the model on both the training and validation data. This way, we used as much data as possible to build the model while still honoring the train-test split principle.

This means we should retrain the model on `X_trainval`, which contains both the training and validation sets and score it on the test set:

```
In [25]: knn = cv2.ml.KNearest_create()
...     knn.setDefaultK(best_k)
...     knn.train(X_trainval, cv2.ml.ROW_SAMPLE, y_trainval)
...     _, y_test_hat = knn.predict(X_test)
...     accuracy_score(y_test, y_test_hat), best_k
Out[25]: (0.94736842105263153, 7)
```

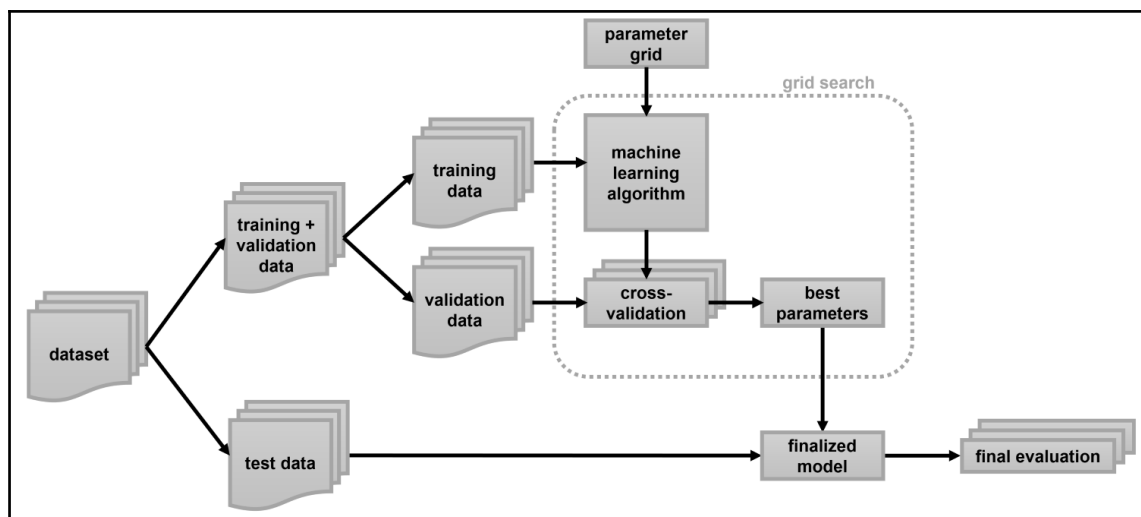
With this procedure, we find a formidable score of 94.7% accuracy on the test set. Because we honored the train-test split principle, we can now be sure that this is the performance we can expect from the classifier when applied to novel data. It is not as high as the 100% accuracy reported during validation, but it is still a very good score!

Combining grid search with cross-validation

One potential danger of the grid search we just implemented is that the outcome might be relatively sensitive to how exactly we split the data. After all, we might have accidentally chosen a split that put most of the easy-to-classify data points in the test set, resulting in an overly optimistic score. Although we would be happy at first, as soon as we tried the model on some new held-out data, we would find that the actual performance of the classifier is much lower than expected.

Instead, we can combine grid search with cross-validation. This way, the data is split multiple times into training and validation sets, and cross-validation is performed at every step of the grid search to evaluate every parameter combination.

The entire process is illustrated in the following figure:



Overview of grid search with cross-validation

As is evident from the figure, the test set is created right from the outset and kept separate from the grid search. The remainder of the data is then split into training and validation sets like we did before and passed to the grid search. Within the grid search box, we perform cross-validation on every possible combination of parameter values to find the best model. The selected parameters are then used to build a final model, which is evaluated on the test set.

Because grid search with cross-validation is such a commonly used method for hyperparameter tuning, scikit-learn provides the `GridSearchCV` class, which implements it in the form of an estimator.

We can specify all the parameters we want `GridSearchCV` to search over by using a dictionary. Every entry of the dictionary should be of the form `{name: values}`, where `name` is a string that should be equivalent to the parameter name usually passed to the classifier, and `values` is a list of values to try.

For example, in order to search for the best value of the parameter `n_neighbors` of the `KNeighborsClassifier` class, we would design the parameter dictionary as follows:

```
In [12]: param_grid = {'n_neighbors': range(1, 20)}
```

Here, we are searching for the best k in the range $[1, 19]$.

We then need to pass the parameter grid as well as the classifier (`KNeighborsClassifier`) to the `GridSearchCV` object:

```
In [13]: from sklearn.neighbors import KNeighborsClassifier
...      from sklearn.model_selection import GridSearchCV
...      grid_search = GridSearchCV(KNeighborsClassifier(), param_grid,
...                                cv=5)
```

Then we can train the classifier using the `fit` method. In return, scikit-learn will inform us about all the parameters used in the grid search:

```
In [14]: grid_search.fit(X_trainval, y_trainval)
Out[14]: GridSearchCV(cv=5, error_score='raise',
                      estimator=KNeighborsClassifier(algorithm='auto',
                                                    leaf_size=30, metric='minkowski',
                                                    metric_params=None, n_jobs=1, n_neighbors=5,
                                                    p=2, weights='uniform'),
                      fit_params={}, iid=True, n_jobs=1,
                      param_grid={'n_neighbors': range(1, 20)},
                      pre_dispatch='2*n_jobs',
                      refit=True, return_train_score=True, scoring=None,
                      verbose=0)
```

This will allow us to find the best validation score and the corresponding value for k :

```
In [15]: grid_search.best_score_, grid_search.best_params_  
Out[15]: (0.9642857142857143, {'n_neighbors': 3})
```

We thus get a validation score of 96.4% for $k = 3$. Since grid search with cross-validation is more robust than our earlier procedure, we would expect the validation scores to be more realistic than the 100% accuracy we found before.

However, from the previous section, we know that this score might still be overly optimistic, so we need to score the classifier on the test set instead:

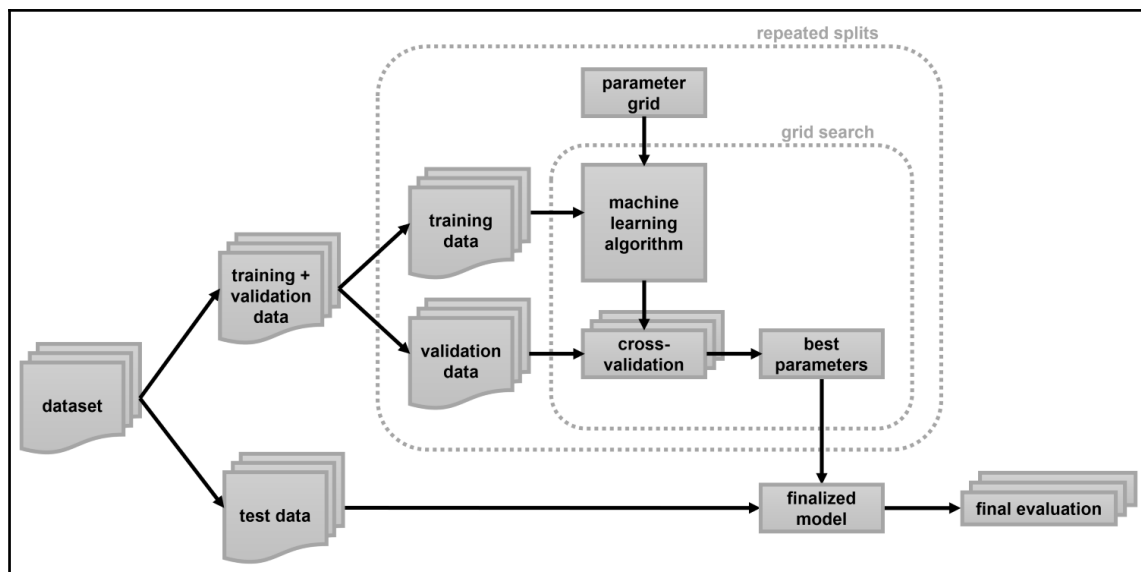
```
In [30]: grid_search.score(X_test, y_test)  
Out[30]: 0.97368421052631582
```

And to our surprise, the test score is even better.

Combining grid search with nested cross-validation

Although grid search with cross-validation makes for a much more robust model selection procedure, you might have noticed that we performed the split into training and validation set still only once. As a result, our results might still depend too much on the exact training-validation split of the data.

Instead of splitting the data into training and validation set once, we can go a step further and use multiple splits for cross-validation. This will result in what is known as **nested cross-validation**, and the process is illustrated in the following figure:



Overview of nested cross-validation

In nested cross-validation, there is an outer loop over the grid search box that repeatedly splits the data into training and validation sets. For each of these splits, a grid search is run, which will report back a set of best parameter values. Then, for each outer split, we get a test score using the best settings.



While running a grid search over many parameters and on large datasets can be computationally challenging, it is also embarrassingly parallel. This model using a particular parameter setting on a particular cross-validation split can be done completely independently from the other parameter settings and models. This makes grid search and cross-validation ideal candidates for parallelization over multiple CPU cores or over a cluster.

Now that we know how to find the best parameters of a model, let's take a closer look at the different evaluation metrics which we can use to score a model.

Scoring models using different evaluation metrics

So far, we have evaluated classification performance using accuracy (the fraction of correctly classified samples) and regression performance using R^2 . However, these are only two of the many possible ways to summarize how well a supervised model performs on a given dataset. In practice, these evaluation metrics might not be appropriate for our application, and it is important to choose the right metric when selecting between models and adjusting parameters.

When selecting a metric, we should always have the end goal of the machine learning application in mind. In practice, we are usually interested not just in making accurate predictions but also in using these predictions as part of a larger decision-making process. For example, minimizing false positives might be equally important as maximizing accuracy.



When choosing a model or adjusting parameters, we should pick the model or parameter values that have the most positive influence on the business metric.

Choosing the right classification metric

We talked about a number of essential scoring functions in [Chapter 3, First Steps in Supervised Learning](#). Among the most fundamental metrics for classification were the following:

- **Accuracy:** This counts the number of data points in the test set that have been predicted correctly and returns that number as a fraction of the test set size (`sklearn.metrics.accuracy_score`). This is the most basic scoring function for classifiers, and we have made extensive use of it throughout this book.
- **Precision:** This describes the ability of a classifier not to label a positive sample as a negative (`sklearn.metrics.precision_score`).
- **Recall (or sensitivity):** This describes the ability of a classifier to retrieve all the positive samples (`sklearn.metrics.recall_score`).

Although precision and recall are important measures, looking at only one of them will not give us a good idea of the big picture. One way to summarize the two measures is known as the **f-score** or **f-measure** (`sklearn.metrics.f1_score`), which computes the harmonic mean of precision and recall as $2(\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$.

Sometimes we need to do more than maximize accuracy. For example, if we are using machine learning in a commercial application, then the decision-making should be driven by the business goals. One of these goals might be to guarantee at least 90% recall. The challenge then becomes to develop a model that still has reasonable accuracy while satisfying all secondary requirements. Setting goals like this is often called **setting the operating point**.

However, when developing a new system, it is often not clear what the operating point should be. For this reason and to understand the problem better, it is instructive to investigate all possible trade-offs of precision and recall at once. This is possible using a tool called the **precision-recall curve** (`sklearn.metrics.precision_recall_curve`).



Another commonly used tool to analyze the behavior of classifiers is the **receiver operating characteristic (ROC) curve**. Similar to the precision-recall curve, the ROC curve considers all possible thresholds for a given classifier, but instead of reporting precision and recall, it shows the **false positive rate** against the **true positive rate**.

Choosing the right regression metric

Evaluation for regression can be done in similar detail as we did for classification. In Chapter 3, *First Steps in Supervised Learning*, we also talked about some fundamental metrics for regression:

- **Mean squared error:** The most commonly used error metric for regression problems is to measure the squared error between predicted and true target value for every data point in the training set, averaged across all data points (`sklearn.metrics.mean_squared_error`).
- **Explained variance:** A more sophisticated metric is to measure to what degree a model can explain the variation or dispersion of the test data (`sklearn.metrics.explained_variance_score`). Often, the amount of explained variance is measured using the correlation coefficient.
- The **R² score** (pronounced **R squared**): This is closely related to the explained variance score, but it uses an unbiased variance estimation (`sklearn.metrics.r2_score`). It is also known as the **coefficient of determination**.

In most applications we have encountered so far, using the default R^2 score is enough.

As we combine elaborate grid searchers with sophisticated evaluation metrics, our model selection code might become increasingly complex. Fortunately, scikit-learn offers a way to simplify model selection with a helpful construct known as a **pipeline**.

Chaining algorithms together to form a pipeline

Most machine learning problems we have discussed so far consist of at least a preprocessing step and a classification step. The more complicated the problem, the longer this **processing chain** might get. One convenient way to glue multiple processing steps together and even use them in grid search is by using the `Pipeline` class from scikit-learn.

Implementing pipelines in scikit-learn

The `Pipeline` class itself has a `fit`, a `predict`, and a `score` method, which behave just like any other estimator in scikit-learn. The most common used case of the `Pipeline` class is to chain different preprocessing steps together with a supervised model like a classifier.

Let's return to the breast cancer dataset from Chapter 5, *Using Decision Trees to Make a Medical Diagnosis*. Using scikit-learn, we import the dataset and split it into training and test sets:

```
In [1]: from sklearn.datasets import load_breast_cancer
...     import numpy as np
...     cancer = load_breast_cancer()
...     X = cancer.data.astype(np.float32)
...     y = cancer.target
In [2]: X_train, X_test, y_train, y_test = train_test_split(
...     X, y, random_state=37
...     )
```

Instead of the k -NN algorithm, we could fit a **support vector machine (SVM)** to the data:

```
In [3]: from sklearn.svm import SVC
...     svm = SVC()
...     svm.fit(X_train, y_train)
Out[3]: SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
...         decision_function_shape=None, degree=3, gamma='auto',
...         kernel='rbf', max_iter=-1, probability=False,
```

```
random_state=None, shrinking=True, tol=0.001,  
verbose=False)
```

Without straining our brains too hard, this algorithm achieves an accuracy score of 65%:

```
In [4]: svm.score(X_test, y_test)  
Out[4]: 0.65034965034965031
```

Now if we wanted to run the algorithm again using some preprocessing step (for example, by scaling the data first with `MinMaxScaler`), we would do the preprocessing step by hand and then feed the preprocessed data into the classifiers `fit` method.

An alternative is to use a pipeline object. Here, we want to specify a list of processing steps, where each step is a tuple containing a name (any string of our choosing) and an instance of an estimator:

```
In [5]: from sklearn.pipeline import Pipeline  
...     pipe = Pipeline([("scaler", MinMaxScaler()), ("svm", SVC())])
```

Here, we created two steps: the first, called "scaler", is an instance of `MinMaxScaler`, and the second, called "svm", is an instance of `SVC`. Now we can fit the pipeline like any other scikit-learn estimator:

```
In [6]: pipe.fit(X_train, y_train)  
...     Pipeline(steps=[('scaler', MinMaxScaler(copy=True,  
...     feature_range=(0, 1))), ('svm', SVC(C=1.0,  
...     cache_size=200, class_weight=None, coef0=0.0,  
...     decision_function_shape=None, degree=3, gamma='auto',  
...     kernel='rbf', max_iter=-1, probability=False,  
...     random_state=None, shrinking=True, tol=0.001,  
...     verbose=False))])
```

Here, the `fit` method first calls `fit` on the first step (the scaler), then it transforms the training data using the scaler, and finally it fits the SVM with the scaled data.

And voila! When we score the classifier on the test data, we see a drastic improvement in performance:

```
In [7]: pipe.score(X_test, y_test)  
Out[7]: 0.95104895104895104
```

Calling the `score` method on the pipeline first transforms the test data using the scaler and then calls the `score` method on the SVM using the scaled test data. And scikit-learn did all this with only four lines of code!

The main benefit of using the pipeline, however, is that we can now use this single estimator in `cross_val_score` or `GridSearchCV`.

Using pipelines in grid searches

Using a pipeline in a grid search works the same way as using any other estimator.

We define a parameter grid to search over and construct a `GridSearchCV` from the pipeline and the parameter grid. When specifying the parameter grid, there is, however, a slight change. We need to specify for each parameter which step of the pipeline it belongs to. Both parameters that we want to adjust, `C` and `gamma`, are parameters of `SVC`, the second step. In the preceding section, we gave this step the name `"svm"`. The syntax to define a parameter grid for a pipeline is to specify for each parameter the step name, followed by `__` (a double underscore), followed by the parameter name.

Hence, we would construct the parameter grid as follows:

```
In [8]: param_grid = {'svm__C': [0.001, 0.01, 0.1, 1, 10, 100],
...                  'svm__gamma': [0.001, 0.01, 0.1, 1, 10, 100]}
```

With this parameter grid, we can use `GridSearchCV` as usual:

```
In [9]: grid = GridSearchCV(pipe, param_grid=param_grid, cv=10)
...     grid.fit(X_train, y_train);
```

The best score in the grid is stored in `best_score_`:

```
In [10]: grid.best_score_
Out[10]: 0.97652582159624413
```

Similarly, the best parameters are stored in `best_params_`:

```
In [11]: grid.best_params_
Out[11]: {'svm__C': 1, 'svm__gamma': 1}
```

But recall that the cross-validation score might be overly optimistic. In order to know the true performance of the classifier, we need to score it on the test set:

```
In [12]: grid.score(X_test, y_test)
Out[12]: 0.965034965034965
```

In contrast to the grid search we did before, now for each split in the cross-validation, `MinMaxScaler` is refit with only the training splits, and no information is leaked from the test split into the parameter search.

This makes it easy to build a pipeline to chain together a whole variety of steps! You can mix and match estimators in the pipeline at will, you just need to make sure that every step in the pipeline provides a `transform` method (except for the last step). This allows an estimator in the pipeline to produce a new representation of the data, which, in turn, can be used as input to the next step.



The `Pipeline` class is not restricted to preprocessing and classification but can, in fact, join any number of estimators together. For example, we could build a pipeline containing feature extraction, feature selection, scaling, and classification, for a total of four steps. Similarly, the last step could be regression or clustering instead of classification.

Summary

In this chapter, we tried to complement our existing machine learning skills by discussing best practices in model selection and hyperparameter tuning. You learned how to tweak the hyperparameters of a model using grid search and cross-validation in both OpenCV and scikit-learn. We also talked about a wide variety of evaluation metrics and how to chain algorithms into a pipeline.

Now you are almost ready to start working on some real-world problems on your own. But before we part ways, the next chapter will give you some tips and tricks on how to approach a machine learning problem in the wild.

12

Wrapping Up

Congratulations! You have just made a big step toward becoming a machine learning practitioner. Not only are you familiar with a wide variety of fundamental machine learning algorithms, you also know how to apply them to both supervised and unsupervised learning problems.

Before we part ways, I want to give you some final words of advice, point you toward some additional resources, and give you some suggestions on how you can further improve your machine learning and data science skills.

Approaching a machine learning problem

When you see a new machine learning problem in the wild, you might be tempted to jump ahead and throw your favorite algorithm at the problem—perhaps the one you understood best or had the most fun implementing. But knowing beforehand which algorithm will perform best on your specific problem is not often possible.

Instead, you need to take a step back and look at the big picture. Before you get in too deep, you will want to make sure to define the actual problem you are trying to solve. For example, do you already have a specific goal in mind, or are you just looking to do some exploratory analysis and find something interesting in the data? Often, you will start with a general goal, such as detecting spam email messages, making movie recommendations, or automatically tagging your friends in pictures uploaded to a social media platform. However, as we have seen throughout the book, there are often several ways to solve a problem. For example, we have recognized handwritten digits using logistic regression, k-means clustering, and deep learning. Defining the problem will help you ask the right questions and make the right choices along the way.

As a rule of thumb, you can use the following five-step procedure to approach machine learning problems in the wild:

1. **Categorize the problem:** This is a two-step process:
 - **Categorize by input:** Simply speaking, if you have labeled data, it's a supervised learning problem. If you have unlabeled data and want to find structure, it's an unsupervised learning problem. If you want to optimize an objective function by interacting with an environment, it's a reinforcement learning problem.
 - **Categorize by output:** If the output of your model is a number, it's a regression problem. If the output of your model is a class (or category), it's a classification problem. If the output of your model is a set of input groups, it's a clustering problem.
2. **Find the available algorithms:** Now that you have categorized the problem, you can identify the algorithms that are applicable and practical to implement using the tools at our disposal. Microsoft has created a handy algorithm cheat sheet that shows which algorithms can be used for which category of problems. Although the cheat sheet is tailored towards the **Microsoft Azure** software, you might find it generally helpful.



The Machine Learning algorithm cheat sheet PDF (by Microsoft Azure) can be downloaded from <http://aka.ms/MLCheatSheet>.

3. **Implement all of the applicable algorithms (prototyping):** For any given problem, there are usually a handful of candidate algorithms that could do the job. So how do you know which one to pick? Often, the answer to this problem is not straightforward, so you have to resort to trial and error. Prototyping is best done in two steps:
 - You should aim for a quick and dirty implementation of several algorithms with minimal feature engineering. At this stage, you should mainly be interested in seeing which algorithm behaves better at a coarse scale. This step is a bit like hiring: you're looking for any reason to shorten your list of candidate algorithms. Once you have reduced the list to a few candidate algorithms, the real prototyping begins.

- Ideally, you would want to set up a machine learning pipeline that compares the performance of each algorithm on the dataset using a set of carefully selected evaluation criteria (see Chapter 11, *Selecting the Right Model with Hyper-Parameter Tuning*). At this stage, you should only be dealing with a handful of algorithms, so you can turn your attention to where the real magic lies: feature engineering.
4. **Feature engineering:** Perhaps even more important than choosing the right algorithm is choosing the right features to represent the data. You can read all about feature engineering in Chapter 4, *Representing Data and Engineering Features*.
 5. **Optimize hyperparameters:** Finally, you also want to optimize an algorithm's hyperparameters. Examples might include the number of principal components of PCA, the parameter k in the k -nearest neighbor algorithm, or the number of layers and learning rate in a neural network. You can look at Chapter 11, *Selecting the Right Model with Hyper-Parameter Tuning*, for inspiration.

Building your own estimator

In this book, we visited a whole variety of machine learning tools and algorithms that OpenCV provides straight out of the box. And if, for some reason, OpenCV does not provide exactly what we are looking for, we can always fall back on scikit-learn.

However, when tackling more advanced problems, you might find yourself wanting to perform some very specific data processing that neither OpenCV nor scikit-learn provide, or you might want to make slight adjustments to an existing algorithm. In this case, you may want to create your own estimator.

Writing your own OpenCV-based classifier in C++

Since OpenCV is one of those Python libraries that does not contain a single line of Python code under the hood (I'm kidding, but it's close), you will have to implement your custom estimator in C++. This can be done in four steps:

1. Implement a C++ source file that contains the main source code. You need to include two header files, one that contains all core functionality of OpenCV (`opencv.hpp`) and another that contains the machine learning module (`ml.hpp`):

```
#include <opencv2/opencv.hpp>
#include <opencv2/ml/ml.hpp>
```

```
#include <stdio.h>
```

Then an estimator class can be created by inheriting from the `StatModel` class:

```
class MyClass : public cv::ml::StatModel
{
    public:
```

Next, you define constructor and destructor of the class:

```
MyClass()
{
    print("MyClass constructor\n");
}
~MyClass() {}
```

Then you also have to define some methods. These are what you would fill in to make the classifier actually do some work:

```
int getVarCount() const
{
    // returns the number of variables in training samples
    return 0;
}

bool empty() const
{
    return true;
}

bool isTrained() const
{
    // returns true if the model is trained
    return false;
}

bool isClassifier() const
{
    // returns true if the model is a classifier
    return true;
}
```

The main work is done in the `train` method, which comes in two flavors (either accepting `cv::ml::TrainData` or `cv::InputArray` as input):

```
bool train(const cv::Ptr<cv::ml::TrainData>& trainData,
           int flags=0) const
```

```
{
    // trains the model
    return false;
}

bool train(cv::InputArray samples, int layout,
           cv::InputArray responses)
{
    // trains the model
    return false;
}
```

You also need to provide a predict method and a scoring function:

```
float predict(cv::InputArray samples,
              cv::OutputArray results=cv::noArray(),
              int flags=0) const
{
    // predicts responses for the provided samples
    return 0.0f;
}

float calcError(const cv::Ptr<cv::ml::TrainData>& data,
                bool test, cv::OutputArray resp)
{
    // calculates the error on the training or test dataset
    return 0.0f;
}
};
```

The last thing to do is to include a main function that instantiates the class:

```
int main()
{
    MyClass myclass;
    return 0;
}
```

2. Write a CMake file called CMakeLists.txt:

```
cmake_minimum_required(VERSION 2.8)
project(MyClass)
find_package(OpenCV REQUIRED)
add_executable(MyClass MyClass.cpp)
target_link_libraries(MyClass ${OpenCV_LIBS})
```

3. Compile the file on the command line by typing the following:

```
$ cmake
$ make
```

4. Run the executable `MyClass` method which was generated by the last command, which should lead to the following output:

```
$ ./MyClass
MyClass constructor
```

Writing your own scikit-learn-based classifier in Python

Alternatively, you can write your own classifier using the scikit-learn library.

You can do this by importing `BaseEstimator` and `ClassifierMixin`. The latter will provide a corresponding `score` method, which works for all classifiers. Optionally, you can overwrite the `score` method to provide your own:

```
In [1]: import numpy as np
...      from sklearn.base import BaseEstimator, ClassifierMixin
```

Then you can define a class that inherits from both `BaseEstimator` and `ClassifierMixin`:

```
In [2]: class MyClassifier(BaseEstimator, ClassifierMixin):
...      """An example classifier"""
```

You need to provide a constructor, a `fit` method, and a `predict` method. The constructor defines all parameters that the classifier needs, here given by some arbitrary example parameters `param1` and `param2` that don't do anything:

```
...     def __init__(self, param1=1, param2=2):
...         """Called when initializing the classifier
...
...         Parameters
...         -----
...         param1 : int, optional, default: 1
...             The first parameter
...         param2 : int, optional, default: 2
...             The second parameter
...         """
...         self.param1 = param1
```

```
...         self.param2 = param2
```

The classifier should then be fit to data in the `fit` method:

```
...     def fit(self, X, y=None):
...         """Fits the classifier to data
...
...         Parameters
...         -----
...         X : array-like
...             The training data, where the first dimension is
...             the number of training samples, and the second
...             dimension is the number of features.
...         y : array-like, optional, default: None
...             Vector of class labels
...
...         Returns
...         -----
...         The fit method returns the classifier object it
...         belongs to.
...         """
...         return self
```

Finally, the classifier should also provide a `predict` method, which will predict the target labels of some data `x`:

```
...     def predict(self, X):
...         """Predicts target labels
...
...         Parameters
...         -----
...         X : array-like
...             Data samples for which to predict the target labels.
...
...         Returns
...         -----
...         y_pred : array-like
...             Target labels for every data sample in `X`
...         """
...         return np.zeros(X.shape[0])
```

Then you can instantiate the model like any other class:

```
In [3]: myclass = MyClassifier()
```

You can then fit the model to some arbitrary data:

```
In [4]: X = np.random.rand(10, 3)
...     myclass.fit(X)
Out[4]: MyClassifier(param1=1, param2=2)
```

And then you can proceed to predicting the target responses:

```
In [5]: myclass.predict(X)
Out[5]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]
```

Implementing a regressor, clustering algorithm, or transformer works similarly, but instead of the `ClassifierMixin` keyword, you would choose one of the following:

- `RegressorMixin` if you are writing a regressor (this will provide a basic `score` method suitable for regressors).
- `ClusterMixin` if you are writing a clustering algorithm (this will provide a basic `fit_predict` method suitable for clustering algorithms).
- `TransformerMixin` if you are writing a transformer (this will provide a basic `fit_predict` method suitable for transformers). Also, instead of `predict`, you would implement `transform`.



This is also a great way to disguise an OpenCV classifier as a scikit-learn estimator. This will allow you to use all of scikit-learn's convenience functions—for example, to make your classifier part of a pipeline—while OpenCV performs the underlying computation.

Where to go from here?

The goal of this book was to introduce you to the world of machine learning and prepare you to become a machine learning practitioner. Now that you know everything about the fundamental algorithms, you might want to investigate some topics in more depth.

Although it is not necessary to understand all the details of all the algorithms we implemented in this book, knowing some of the theory behind them might just make you a better data scientist.

If you are looking for a more advanced lecture, then you might want to consider some of the following classics:



- Stephen Marsland, *Machine Learning: An Algorithmic Perspective. Second Edition*, Chapman and Hall/Crc, ISBN 978-146658328-3, 2014
- Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, ISBN 978-038731073-2, 2007
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Second Edition*, Springer, ISBN 978-038784857-0, 2016

When it comes to software libraries, we already learned about two essential ones—OpenCV and scikit-learn. Often, using Python is great for trying out and evaluating models, but larger web services and applications are more commonly written in Java or C++. For example, the C++ package is **vowpal wabbit (vw)**, which comes with its own command-line interface. For running machine learning algorithms on a cluster, people often use `mllib`, a **Scala** library built on top of **Spark**. If you are not married to Python, you might also consider using R, another common language of data scientists. R is a language designed specifically for statistical analysis and is famous for its visualization capabilities and the availability of many (often highly specialized) statistical modeling packages.

No matter which software you choose going forward, I guess the most important advice is to keep practicing your skills. But you already knew that. There are a number of excellent datasets out there that are just waiting for you to analyze them:

- Throughout this book, we made great use of the example datasets that are built in to scikit-learn. In addition, scikit-learn provides a way to load datasets from external services, such as `mldata.org`. Refer to <http://scikit-learn.org/stable/datasets/index.html> for more information.
- Kaggle is a company that hosts a wide range of datasets as well as competitions on their website, <http://www.kaggle.com>. Competitions are often hosted by a variety of companies, nonprofit organizations, and universities, and the winner can take home some serious monetary prizes. A disadvantage of competitions is that they already provide a particular metric to optimize and usually a fixed, preprocessed dataset.

- The OpenML platform (<http://www.openml.org>) hosts over 20,000 datasets with over 50,000 associated machine learning tasks.
- Another popular choice is the UC Irvine machine learning repository (<http://archive.ics.uci.edu/ml/index.php>), hosting over 370 popular and well-maintained datasets through a searchable interface.

Finally, if you are looking for more example code in Python, a number of excellent books nowadays come with their own GitHub repository:



- Jake VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly, ISBN 978-149191205-8, 2016, <https://github.com/jakevdp/PythonDataScienceHandbook>
- Andreas Muller and Sarah Guido, *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly, ISBN 978-144936941-5, 2016, https://github.com/amueller/introduction_to_ml_with_python
- Sebastian Raschka, *Python Machine Learning*. Packt, ISBN 978-178355513-0, 2015, <https://github.com/rasbt/python-machine-learning-book>

Summary

In this book, we covered a lot of theory and practice.

We discussed a wide variety of fundamental machine learning algorithms, be it supervised or unsupervised, illustrated best practices as well as ways to avoid common pitfalls, and we touched upon a variety of commands and packages for data analysis, machine learning, and visualization.

If you made it this far, you have already made a big step toward machine learning mastery. From here on out, I am confident you will do just fine on your own.

All that's left to say is farewell! I hope you enjoyed the ride; I certainly did.

Index

A

- accuracy
 - about 325
 - used, for scoring classifiers 47
- activation function 225
- AdaBoost
 - implementing 289
 - implementing, in OpenCV 289
 - implementing, in scikit-learn 291
- adaptive boosting algorithm 124
- additive mixing 101
- affinity propagation 206
- agglomerative hierarchical clustering
 - about 220
 - implementing 221
- aggregation 83
- algorithms
 - chaining 327
- Anaconda 15
- Apache SpamAssassin public corpus
 - reference 185
- application programming interface (API) 27
- artificial intelligence 8
- ASCII table
 - reference 117
- attributes 46
- averaging ensembles 266
- averaging methods 264
- axon 224
- axon terminals 224

B

- backpropagation
 - multi-layer perceptrons, training with 242
- bag of words 191
- bagging classifier

- implementing 266, 267
- bagging methods 266
- bagging regressor
 - implementing 268
- BaggingClassifier class
 - options 267
- base class 46
- basorexia 113
- batch-based k-means 210
- Bayes' theorem 171, 173, 174
- Bayesian classifier
 - implementing 176
- Bayesian models
 - reference, for example 171
- bell curve 150, 245
- binarizing 85
- binary classification 45
- binary classification task 140
- binary encoding 97
- Binder project
 - reference 14
- blending methods 264
- blobs 150
- boosting classifier
 - implementing 270
- boosting ensembles 269
- boosting methods 264
- boosting regressor
 - implementing 271
- bootstrapping
 - about 162, 309
 - implementing, manually in OpenCV 310
 - used, for estimating robustness 309
- Boston housing prices dataset 67
- Breast Cancer Wisconsin dataset
 - reference 128
- BRIEF descriptor 108

C

C++

- OpenCV based classifier, writing in 333

Caffe

- reference 250

- categorical features 97

- categorical label 11

- categorical variables

- representing 97, 98

- class labels

- predicting, classification models used 55

- classes 46

- classification 45

- classification metric

- accuracy 325

- explained variance 326

- precision 325

- R squared 326

- recall 325

- selecting 325

- classification models

- used, for predicting class labels 55

- classification performance

- measuring 47, 49, 51

- classification problem 11

- classification task 115

- classifiers

- about 264

- scoring, accuracy used 47

- scoring, precision used 47

- scoring, recall used 48

- cluster analysis 195

- clusters

- organizing, as hierarchical tree 220

- coefficient of determination 52

- cognitive load 13

- Cognitive Toolkit

- about 250

- reference 250

- color palette

- reducing, k-means used 215

- color pickers 103

- color spaces, compressing with k-means

- about 210

- true-color palette, visualizing 210, 212

- color spaces

- using 101

- Compute Unified Device Architecture (CUDA) 13

- computer vision 8

- conda environment

- OpenCV, installing in 18

- Conda-Forge channel

- reference 18

- conditional probability 173

- confusion matrix 49, 219

- continuous outcomes

- predicting, regression models used 65

- continuous random variable 169

- Continuum

- reference 16

- convolutional neural networks 11

- core layers, Keras

- Activation 251

- Dense 251

- Reshape 251

- corners

- detecting, in images 103

- cross-validation procedures

- reference 309

- cross-validation

- about 303, 305

- grid search, combining with 321

- manual implementation, in OpenCV 306

- curse of dimensionality 89

- Cython 14

D

- dask 14

- data cleaning 82

- data formatting 82

- data mining 10

- data preprocessing

- about 82

- feature extraction 26

- feature selection 26

- data representation 81

- data sampling 82

- data transformation 196

- data

- classifying, with naive Bayes classifier 181
 - classifying, with normal Bayes classifier 178, 179, 180
 - perceptron, fitting to 232
 - preprocessing 119
 - visualizing, Matplotlib used 36
- decision boundaries 140
- decision nodes 111
- decision rule 123
- decision rules 126, 264
- decision rules, hand-coding
 - disadvantages 10
- decision stump 124, 269, 289
- decision tree 9
- decision trees, for diagnosing breast cancer
 - about 128
 - dataset, loading 129
 - decision tree, building 130, 132, 133
- decision trees
 - about 110, 111
 - basics 123
 - building 113
 - combining, into random forest 273
 - complexity, controlling 127
 - constructing 120
 - shortcomings 273, 274, 276
 - using, for regression 135
- decomposition 83
- deep learning
 - about 250
 - layers 252
- deep neural net, training with Keras
 - about 258
 - convolutional neural network, creating 260
 - MNIST dataset, preprocessing 259
 - model, fitting 261
- dendritic tree 224
- dendrogram 220
- density-based spatial clustering of applications with noise (DBSCAN) 206
- dimensionality reduction
 - about 88, 195
 - Independent Component Analysis (ICA) 94
 - Non-negative Matrix Factorization (NMF), implementing 95

- Principal Component Analysis (PCA) 90
- discrete 169
- discrete features 97
- discriminative models 169, 170
- divisive hierarchical clustering 220

E

- edge directions 157
- elbow method 205
- emails, classifying with naive Bayes classifier
 - about 184
 - data matrix, building with Pandas 188
 - data, preprocessing 189
 - dataset, loading 185, 186
 - full dataset, training 190
 - n-grams, used for improving results 191
 - normal Bayes classifier, training 190
 - tf-idf, used for improving results 192
- Enron-Spam dataset
 - reference 185
- ensemble methods
 - about 264
 - averaging methods 264
 - boosting methods 264
 - stacking methods 264
- entropy 127
- estimator
 - building 333
- Euclidean distance 72, 84
- evaluation metrics
 - used, for scoring models 325
- evidence 174
- expectation step 199
- expectation-maximization solution
 - implementing 200, 201
- expectation-maximization
 - about 199, 200
 - limitations 202, 203, 204, 205, 206, 207, 208, 210
- expected value 313
- explained variance
 - about 326
 - used, for scoring regressors 52
- external datasets
 - loading, in Python 34

extremely randomized trees
 implementing 282

F

f-measure 326
f-score 326
factor analysis 195
false negative 49
false positive 49
Fast Approximate Nearest Neighbor (FLANN) 221
FAST keypoint detector 108
feature engineering
 about 26, 81
 feature extraction 82
 feature selection 81
 missing data, handling 86
feature extraction 26, 82
feature selection 26, 81
feature space 26, 82
features
 binarizing 85
 normalizing 84
 scaling, to range 85
 significance 124
 standardizing 83
feedforward neural network 237
folds 303
fraction of variance unexplained 54

G

Gaussian function 150
Gaussian mixture models (GMMs)
 about 200
 reference 200
generative models 169, 170
Gini impurity 126
Git
 about 14
 reference 14
gradient descent 238, 239
gray colormap 40
grid search
 combining, with cross-validation 321
 combining, with nested cross-validation 323
 pipelines, using in 329

H

Haar cascade classifiers 289
hand-coded expert system 9
handwritten digits, classifying with k-means
 about 217
 dataset, loading 217
 k-means, running 217
handwritten digits, classifying
 about 254
 MLP, training with OpenCV 257
 MNIST dataset, loading 254, 255
 MNIST dataset, preprocessing 256
hard voting 292
Harris corner detection 103
Harris operator 26, 82
Hewlett-Packard spam dataset
 reference 184
hidden layer 237
hierarchical clustering
 about 220
 agglomerative hierarchical clustering 220
 divisive hierarchical clustering 220
hierarchical tree
 clusters, organizing as 220
high bias 301
high variance 301
histogram of oriented gradients (HOG) 26, 156
HLS space
 images, encoding in 103
hotspots 150
HSV space
 images, encoding in 103
Hue Saturation Lightness (HSL) 101
Hue Saturation Value (HSV) 101
hyperbolic tangent 245
hyperparameter tuning 166
hyperparameter, tuning with grid search
 about 317
 grid search, combining with cross-validation 321
 simple search grid, implementing 317
 value, of validation set 318, 319
hyperplane 66, 149

I

- image features 101
- image representation 11
- images
 - corners, detecting in 103
 - encoding, in HLS space 103
 - encoding, in HSV space 103
 - encoding, in RGB space 101
 - representing 101
- imputation 86
- Independent Component Analysis (ICA)
 - implementing 94
- information gain 127
- internal parameters 24
- Internet of Things 12
- IPython session
 - starting 27
- IPython
 - reference 28
- iris species
 - classifying, logistic regression used 73
- iterative algorithm 67

J

- joblib 14
- joint probability distribution 170
- Jupyter Notebook 13, 16, 19
- Jupyter session
 - starting 27

K

- k-fold cross-validation
 - about 303
 - scikit-learn, using for 307
- k-means clustering
 - about 196
 - used, for reducing color palette 214, 215
- k-means example
 - implementing 196, 197, 199
- k-nearest neighbors algorithm 56
- k-nearest neighbors implementation, in OpenCV
 - about 57
 - classifier, training 61
 - label, predicting of data point 62, 64

- training data, generating 57, 59

Kaggle

- about 339
- reference 339

Keras

- about 251
- core layers 251
- layers 252
- loss functions 253
- optimizers 252
- reference 253

- kernel functions 150

- kernel trick 150

kernels

- SVM kernels 151

L

- L1 norm 72, 84

- L1 regularization 72

- L2 norm 72, 84

- L2 regularization 72

- labels 26, 45

- lasso regression 72

layers, deep learning

- Dropout 252

- Embedding 252

- GaussianNoise 252

layers, Keras

- convolutional layers 252

- pooling layers 252

- leaf node 123

- learning rate 240

leave-one-out cross-validation

- implementing 308

- likelihood 174, 181

- linear combination 74

- linear decision boundary 149

- linear regression 65

- linear regression, for predicting Boston housing prices
 - about 67

- dataset, loading 68

- model, testing 69, 70

- model, training 68

- Ling-Spam corpus

- reference 184
- list comprehension 115
- logistic function 73
- logistic regression
 - about 73
 - binary classification problem, making 75
 - classifier, testing 78
 - classifier, training 77
 - data, inspecting 76
 - data, splitting into test sets 77
 - data, splitting into training set 77
 - training data, loading 74
 - used, for classifying iris species 73
- loss function 238
- loss functions, Keras
 - hinge loss 253
 - mean squared error 253

M

- machine learning model
 - setting up, in OpenCV 46
- machine learning problem
 - approaching 331, 332
- machine learning problems, phases
 - test phase 25
 - training phase 25
- machine learning workflow 24, 25
- majority voting 292
- Manhattan distance 72, 84
- margins 140
- Matplotlib
 - data, visualizing from external dataset 38
 - importing 36
 - reference 36
 - simple plot, producing 36
 - used, for visualizing data 36
- maximization step 199
- maximum a posteriori (MAP) decision rule 176
- maximum-margin classifiers 140
- McCulloch-Pitts neuron 224, 225, 226
- McNemar's test
 - implementing 315, 316
- mean absolute error (MAE) 136
- mean concave points 132
- mean squared error (MSE) 136

- mean squared error
 - about 67, 326
 - used, for scoring regressors 51
- members 46
- meta-estimators 266
- methods 46
- mini-batch learning 240
- misclassification error 141
- MNIST dataset 254
- model complexity 301
- model performance
 - measuring, with scoring functions 47
- models
 - best model, selecting 300, 301, 302
 - combining, into voting classifier 292
 - evaluating 297
 - evaluating, right way 298
 - evaluating, wrong way 297
 - scoring, evaluation metrics used 325
- multi-class classification 45
- multiclass classification task 284
- multidimensional arrays
 - creating 33
- multilayer perceptron (MLP)
 - about 237
 - training, with backpropagation 242
- multilayer perceptron implementation, in OpenCV
 - about 243
 - data, preprocessing 244
 - MLP classifier, creating 244
 - MLP classifier, customizing 245, 246
 - MLP classifier, testing 249
 - MLP classifier, training 247
- multivariate Gaussian 90

N

- n-gram counts 191
- naïve Bayes classifier
 - data, classifying with 181
- negatives 48
- nested cross-validation
 - grid search, combining with 323
- net input 225
- Netflix Prize
 - reference 272

- neural network, training requisites
 - cost function 226
 - learning rule 227
 - training data 226
- Non-negative Matrix Factorization (NMF)
 - implementing 95
- nonlinear decision boundaries
 - dealing with 149
- nonlinear support vector machines
 - implementing 152, 153
- normal Bayes classifier
 - data, classifying with 178, 179, 180
- Not a Number (NaN) 86
- null hypothesis 313
- Numerical Python (NumPy) 27
- NumPy arrays 30
- NumPy
 - importing 29
 - multidimensional arrays, creating 33
 - reference 29
 - single array elements, accessing by indexing 32

O

- object-oriented programming (OOP) 46
- objects 46
- Olivetti face dataset 284
- on-line gradient descent 240
- one-hot encoding 97
- open-source deep learning frameworks
 - Caffe 250
 - Cognitive Toolkit 250
 - TensorFlow 250
 - Theano 251
 - Torch 251
- OpenCV based classifier
 - writing, in C++ 333
- OpenCV's TrainData container, in C++
 - used, for dealing with data 41
- OpenCV
 - AdaBoost, implementing in 289
 - cross-validation, implementing manually 306
 - installation, verifying 18
 - installing, in conda environment 18
 - machine learning model, setting up 46
 - manual implementation, of bootstrapping 310

- ML module 21
- Principal Component Analysis (PCA),
 - implementing in 90, 92
 - supervised learning 46
- OpenML platform
 - reference 340
- optimal decision boundaries
 - learning 140, 141, 142
- optimization problem 67
- optimizers, Keras
 - adaptive moment estimation 252
 - root mean square propagation 252
 - stochastic gradient descent 252
- origin 66
- overfit 301
- overfitting 71, 161
- overfitting, avoiding
 - post-pruning 127
 - pre-pruning 127

P

- p-value 314
- Pandas
 - about 188
 - reference 35
- partial derivative 242
- partitions 140
- pasting methods 266
- pedestrians detection, in wild
 - about 154
 - dataset, obtaining 154, 155
 - histogram of oriented gradients (HOG) 156, 158
 - model, bootstrapping 162
 - negatives, generating 158, 160
 - support vector machine, implementing 161
- pedestrians
 - detecting, in large image 163
- perceptron classifier
 - evaluating 233
 - toy dataset, generating 231
- perceptron learning rule 227
- perceptron
 - about 227, 228
 - applying, to data 235, 236
 - fitting, to data 232

- implementing 230
- pipelines
 - about 327
 - implementing, in scikit-learn 327
 - using, in grid search 329
- plane 149
- positives 48
- post-pruning 127
- posterior 174
- pre-pruning 127
- precision-recall 326
- precision
 - about 325
 - used, for scoring classifiers 47
- predictions 65
- principal component 93
- Principal Component Analysis (PCA)
 - implementing, in OpenCV 90, 92
- prior 174
- probability 74, 169
- probability density function 170
- probability distribution 169, 170
- probability mass function 170
- probability theory 169
- processing chain 327
- pruning 127
- Python's NumPy package
 - used, for dealing with data 29
- Python
 - external datasets, loading in 34
 - Scikit-Learn based classifier, writing in 336

R

- R squared
 - about 326
 - used, for scoring regressors 52
- radial basis functions (RBFs) 150
- random forest
 - about 127
 - decision trees, combining into 273
 - implementing 278
 - implementing, with scikit-learn 280
- random forests, for face recognition
 - about 284
 - dataset, loading 284
 - dataset, preprocessing 285
 - random forest, testing 287
 - random forest, training 287
- random number generator 48
- random patches methods 266
- random subspace methods 266
- random variable 169
- range
 - features, scaling to 85
- recall
 - about 325
 - used, for scoring classifiers 48
- receiver operating characteristic (ROC) 326
- rectilinear distance 72
- recurrent neural network 237
- region of interest (ROI) 159
- regression metric
 - mean squared error 326
 - selecting 326
- regression models
 - used, for predicting continuous outcomes 65
- regression problem 11
- regression
 - about 45
 - decision trees, using for 135
- regressors
 - about 264
 - scoring, explained variance used 52
 - scoring, mean squared error used 51
 - scoring, R squared used 52
- regularization 72
- regularizers
 - L1 regularization 72
 - L2 regularization 72
- reinforcement learning 12
- ResearchGate
 - reference 163
- results
 - significance, assessing 312
- reward signal 12
- RGB space
 - images, encoding in 101
- ridge regression 72
- robustness
 - enabling, bootstrapping used 309

root node 123

S

Scala 339

scale-invariant feature transform (SIFT) 26

Scale-Invariant Feature Transform (SIFT)

- about 81

- using 105

scaling 82

Scientific Python (SciPy) 27

Scikit-Learn based classifier

- writing, in Python 336

scikit-learn

- about 13, 16, 34

- AdaBoost, implementing in 291

- pipelines, implementing in 327

- random forest, implementing with 280

- reference 100, 339

- using, for k-fold cross-validation 307

SciPy 15

scoring functions

- used, for measuring model performance 47

Shi-Tomasi corner detection 104

sigmoid 73

Silhouette analysis 206

silhouette plot 207

simple linear regression 65

snake distance 72

soft voting 292

soma 224

Spark 339

spectral clustering 208

speeded up robust features (SURF) 26

Speeded Up Robust Features (SURF)

- about 82

- using 107

squared error 51

squashing function 74

stacking ensembles 272

stacking methods 264

standardization 83

statistical learning models 46

stochastic gradient descent 240

Student's t-test

- implementing 313, 315

supervised learning

- about 11, 45

- classification 45

- in OpenCV 46

- regression 45

support vector machine implementation

- about 142

- dataset, generating 143

- dataset, preprocessing 144

- dataset, visualizing 143

- decision boundary, visualizing 146, 147, 148

support vector machine

- building 145

SVM kernels 151

T

target labels 115

target values 45

TensorFlow

- about 250

- reference 250

term frequency-inverse document frequency (TF-IDF)

- about 100

- reference 100

term-inverse document frequency (tf-idf) 192

test data 11

test dataset 25

text features

- representing 99

Theano

- about 251

- reference 251

thresholding 85

Torch

- about 251

- reference 251

toy dataset

- creating 176

trained decision tree

- visualizing 121

training data 11

training dataset 25

true negative 49

true positive 49

true-color image 210
two-class classification 45
type I error 49
type II error 49

U

UC Irvine machine learning repository
 reference 340
uncertainty 127
underfit 301
universal approximation property 238
unsupervised learning
 about 11, 195
 cluster analysis 195
 dimensionality reduction 195
 factor analysis 195

V

validation dataset 25
validation set 319
variance 54
variance reduction 136
version control system 14
voting classifier
 about 292
 implementing 293
 models, combining into 292
voting schemes
 hard voting 292
 soft voting 292
vowpal wabbit (vw) 339

W

weight coefficients 65, 225
weight update rule 227
weighted sum 225
wisdom of the crowd concept 264